UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di Fisica “G. Occhialini”

Corso di Laurea Triennale



Sviluppo di un decoder basato su Reinforcement Learning per Quantum Error Correction

Relatore:
Prof. Andrea Giachero

Correlatore:
Dott. Danilo Labranca

Relatore Esterno:
Dott. Michele Grossi

Candidato:
Simone Pietro Razzetti
Matricola 898006

ANNO ACCADEMICO 2024/2025

Sommario

Il *Quantum Computing* nasce come promettente alternativa alle tecnologie classiche per l'esecuzione di calcoli e simulazioni computazionalmente complessi. In particolare, l'impiego dei qubit come unità fondamentali del calcolo consente di sfruttare proprietà notevoli della meccanica quantistica per un'elaborazione dell'informazione significativamente più efficiente rispetto a quella tradizionale. Tuttavia, indipendentemente dalla piattaforma utilizzata, la fragile natura quantistica di questi sistemi li rende estremamente sensibili alle interazioni con l'ambiente esterno, il che provoca fenomeni di decoerenza e errori nei processi di calcolo. Questo rappresenta uno dei principali limiti per la realizzazione pratica di algoritmi quantistici, per cui diventa indispensabile l'utilizzo di tecniche di correzione degli errori.

Negli anni sono state scoperte diverse strategie di *Quantum Error Correction* (QEC), che sostanzialmente sono basate sull'utilizzo di strutture apposite in grado di distribuire l'informazione logica su un insieme di qubit fisici. Questo consente di proteggere l'informazione complessiva da eventi singoli e, nel contempo, di rilevare, identificare e correggere gli errori marginali che si manifestano. In tale contesto, il *Surface Code* rappresenta una delle tecnologie più promettenti e applicabili: la sua struttura bidimensionale infatti, oltre a garantire un alto livello di tolleranza rispetto a errori isolati, si inserisce bene nel contesto delle principali tecnologie hardware attuali. In generale, la diagnosi degli errori in QEC avviene per mezzo di misure indirette di operatori di parità, noti come stabilizzatori, il cui risultato complessivo costituisce la sindrome d'errore. La sindrome è ambigua e pertanto necessita di essere decifrata per poter compiere le giuste azioni correttive. Il compito di interpretare tali segnali è affidato al decoder, ovvero l'algoritmo incaricato di identificare la configurazione di errore più probabile che può essersi verificata e dunque di suggerire le operazioni correttive più appropriate.

Questo progetto di tesi si inserisce nel contesto del problema di decodifica di sindrome d'errore generata da *Surface Code*, in particolare affrontato tramite il paradigma di *Reinforcement Learning* (RL). Negli ultimi anni sono infatti emerse diverse proposte che impiegano metodi di *Machine Learning*, in particolare legate all'utilizzo di reti neurali, per lo sviluppo di decoder in QEC. Un approccio di questo tipo risulta intrinsecamente generalizzabile e soprattutto adatto ad includere nel processo di decodifica l'eventualità di errori di misura degli stabilizzatori. I principali algoritmi classici proposti, per gestire *faulty syndrome* realistiche, richiedono una computazione onerosa e lenta che li rende difficilmente applicabili nella pratica, specie se si considera che il decoder dovrebbe operare *online* in tempo reale durante l'esecuzione di un algoritmo. L'utilizzo di reti neurali artificiali, o di metodi di *Deep RL* come nel caso di questa tesi, consente invece di realizzare — pagando *offline* il costo di un addestramento intensivo in termini di tempo e risorse — dei modelli *ad hoc* significativamente più rapidi e dunque applicabili verosimilmente anche in un contesto reale. Il principale limite di

un approccio di questo tipo è legato al fatto che, sebbene la struttura della rete e quindi l'algoritmo sia generalizzabile, l'addestramento è specifico per il modello d'errore studiato, il che rende presto inefficace il decoder in presenza di un pattern di errore che può cambiare nel tempo. A questo proposito però, se si considerano modelli di rumore simili, è possibile sfruttare alcune tecniche di *continual learning* come *Probabilistic Policy Reuse* (PPR) per sfruttare le conoscenze sviluppate in un determinato ambiente e migliorare la capacità di adattamento a quello nuovo.

Nella tesi viene realizzato un decoder basato su *Deep RL* — in particolare sull'algoritmo *Double Deep Q-Network* (DDQN) — addestrato in un ambiente adatto a simulare sindromi d'errore *faulty* relative a *Surface Code* e specifiche per un determinato modello di rumore. L'algoritmo implementa reti neurali convoluzionali (CNN) per interpretare la struttura tridimensionale di dati forniti dall'ambiente, nota come *syndrome volume*. Tale struttura è costituita da matrici bidimensionali generate nel tempo, che corrispondono all'insieme delle misure sequenziali degli stabilizzatori soggetti a errori; questo consente di costruire una minima statistica che permette all'agente di eseguire una decodifica anche in presenza di una strumentazione non ideale. Ogni matrice generata corrisponde ad un istante di tempo discreto in cui un qubit o uno stabilizzatore può essere soggetto ad un errore con una certa probabilità. Pertanto, il numero di singole sindromi osservate durante un episodio prima che lo stato logico raggiunga una condizione terminale definisce la *lifetime* del sistema, confrontabile con la vita media di un qubit fisico. In particolare, l'agente sviluppato è stato addestrato in ambienti caratterizzati da modelli di errore di inversione dello stato logico del qubit (*bit-flip*), con probabilità progressivamente crescente. In questo modo — testando le politiche ottenute in ambienti di simulazione coerenti e raccogliendo le principali metriche — è stato possibile valutare gli incrementi di *lifetime* media per ciascun modello studiato, e costruire una libreria di *policy* necessaria per l'approccio di *continual learning*. Successivamente è stata sviluppata una funzione di training ispirata all'algoritmo DDQN con PPR, utilizzata per addestrare l'agente negli stessi ambienti di simulazione precedenti, ma con il supporto delle politiche preaddestrate della libreria.

I dati sperimentali ottenuti mostrano come un decoder di questo tipo costituisca una valida alternativa per il problema di decodifica di *faulty-syndrome*. I test effettuati sulle politiche addestrate hanno permesso di ottenere valori di *logical lifetime* media significativamente maggiori, in proporzione con la difficoltà dell'ambiente affrontato, rispetto alla vita media dei singoli qubit fisici soggetti alle stesse condizioni di rumore. Inoltre, il confronto tra i grafici di *lifetime* media mobile in funzione del numero di interazioni agente-ambiente (*timestep*) — relativi ai metodi DDQN e DDQN+PPR — ha permesso di dimostrare l'adattabilità più efficiente al modello di errore del secondo metodo di addestramento. In media infatti, per ciascun ambiente simulato, gli stessi livelli prestazionali sono stati raggiunti dall'agente DDQN+PPR con il 20–30% di *timestep* in meno rispetto al DDQN tradizionale. Questo conferma le ipotesi di efficacia di un approccio di *continual learning* in questo contesto e getta le basi per un sistema in grado di aggiornarsi anche *online* al variare del modello di rumore.

Questo lavoro di ricerca è ispirato dalle proposte di Sweke et al. (2020) e di Matekole et al. (2022), da cui in particolare è stato tratto l'ambiente di simulazione implementato nel progetto. Il codice è stato interamente sviluppato in *Python*, utilizzando principalmente le librerie di *PyTorch* e *Gymnasium*. Le simulazioni sono state condotte su hardware classico, in cui i qubit erano simulati e il sistema di *Machine Learning* operava senza l'ausilio di infrastrutture di *High Performance Computing* (HPC). Tale configu-

razione ha imposto limiti alla potenza di calcolo e vincoli nella sperimentazione dei parametri. I risultati ottenuti rappresentano quindi un primo risultato promettente, che conferma la validità dei metodi proposti e potrà essere ulteriormente ottimizzato sotto diversi aspetti. In prospettiva, l'approccio sviluppato potrà beneficiare dell'impiego di risorse HPC e, potenzialmente, dell'utilizzo di qubit superconduttivi reali.

Indice

Sommario **I**

Elenco delle figure **VI**

Introduzione **VII**

1 Quantum Error Correction e Surface Code **1**
1.1 Richiami sulla computazione quantistica 1
1.1.1 Qubit 1
1.1.2 Quantum Gate 2
1.1.3 Rumore ed errori quantistici 3
1.2 Principi della Quantum Error Correction 4
1.2.1 Qubit fisici e qubit logici 4
1.2.2 Formalismo degli stabilizzatori 6
1.2.3 Il teorema della soglia 7
1.3 Surface Code 8
1.3.1 Struttura, stabilizzatori e rilevamento degli errori 9
1.3.2 Scalabilità e vantaggi 10
1.4 Decoder per Surface Code 10
1.4.1 Formalismo omologico e classi di equivalenza 11
1.4.2 Sindromi rumorose e decodifica *Fault-tolerant* 11
1.4.3 Approcci di decodifica 12

2 Deep Reinforcement Learning **14**
2.1 Fondamenti di Reinforcement Learning 14
2.1.1 Agente, ambiente, stato, azione e ricompensa 15
2.1.2 Processi Decisionali di Markov 16
2.1.3 Politiche, funzioni valore ed equazioni di Bellman 17
2.2 Algoritmi *value-based* per il Reinforcement Learning 19
2.2.1 Q-learning 19
2.2.2 Deep Q-Network 21
2.2.3 Double Deep Q-Network 22
2.3 Reti Neurali artificiali 22
2.3.1 Architettura e funzioni di attivazione 23
2.3.2 Addestramento di una rete 24
2.3.3 Tipologie di rete 27
2.4 Probabilistic Policy Reuse 28
2.4.1 Esplorazione tramite π-reuse 28
2.4.2 Addestramento con Probabilistic Policy Reuse 29

3 Progetto e risultati sperimentali **31**
3.1 Setup sperimentale 31
3.1.1 Ambiente di simulazione 32
3.1.2 Architettura del decoder 36
3.2 Addestramento dell'agente 41
3.2.1 Funzioni di training DDQN e DDQN + PPR 41
3.2.2 Parametri e impostazioni di addestramento 44
3.2.3 Funzione di test e valutazione della policy 46
3.3 Risultati sperimentali 47
3.3.1 Confronto tra DDQN e DDQN + PPR 49
3.3.2 Analisi della lifetime logica 52

Conclusioni **54**

A Algoritmi di Reinforcement Learning **57**

B Implementazione e frammenti di codice **60**

C Grafici ulteriori **68**

Elenco delle figure

1.1 Rappresentazione di un qubit sulla *sfera di Bloch* 2
1.2 Esempio diagramma di circuito stato di Bell $|\Phi^+\rangle$. 3
1.3 Esempio di uno schema tipico di *Quantum Error Correction*. 4
1.4 Esempio di circuito per misura indiretta degli stabilizzatori Z_1Z_2. . . . 5
1.5 Rappresentazione di un *Surface Code* 5×5 e degli operatori logici X_L e Z_L [9]. 9
1.6 Risposta degli stabilizzatori a ciascuna tipologia di errore quantistico [9]. 9
1.7 Esempi di catene di errori con sindrome equivalente [9]. 10
1.8 Conseguenze di una misura sequenziale degli stabilizzatori per la gestione di *faulty-syndrome* in un decoder [9]. 12

2.1 Esempio di un problema didattico per RL: *Frozen Lake* [26]. 16
2.2 Ciclo interattivo agente-ambiente in un Processo Decisionale di Markov [25]. 17
2.3 Esempio della struttura a strati di una rete neurale artificiale [30]. . . . 23
2.4 Esempio di una rete neurale composta da strati CNN e FFN [32]. . . . 28

3.1 Matrici per la codifica di uno stato dell'*environment* [36]. 32
3.2 Processo ciclico di interazione con l'ambiente [9]. 35
3.3 Struttura della rete neurale implementata [9]. 37
3.4 Grafico *lifetime* media per episodio, ambiente $p_{\text{err}} = 0.001$. 48
3.5 Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.001$. 48
3.6 Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.003$. 49
3.7 Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.005$. 50
3.8 Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.007$. 50
3.9 Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.009$. 51
3.10 Distribuzione della *logical lifetime* per la politica PPR addestrata sull'ambiente $p_{\text{err}} = 0.009$. 52
3.11 Andamento della *logical lifetime* media all'aumentare del tasso di errore p_{err}. 53

Introduzione

Nel 1982, durante una conferenza intitolata *"Simulating Physics with Computers"*, Richard Feynman propose per la prima volta l'idea che la computazione classica non sarebbe mai stata in grado di simulare sistemi quantistici in maniera efficiente, suggerendo l'idea di costruire un computer basato sulle leggi della meccanica quantistica stessa [1]. Pochi anni dopo, nel 1985, David Deutsch formalizzò questa intuizione introducendo il concetto di *computer quantistico universale*, fornendo così le basi teoriche della moderna computazione quantistica [2].

Fin dalla sua nascita, il *Quantum Computing* (QC) ha suscitato grande interesse — oltre la semplice simulazione di sistemi complessi — per le sue potenzialità teoriche nel risolvere problemi computazionalmente complessi che, con le tecnologie classiche attualmente note, non sarebbero possibili in termini di tempo ragionevoli. Negli anni successivi infatti Peter Shor dimostrò che l'impiego di algoritmi quantistici consente la fattorizzazione di numeri di grandi dimensioni con un vantaggio quasi esponenziale rispetto ai corrispettivi algoritmi classici [3], mettendo così potenzialmente a rischio la sicurezza dei moderni sistemi di crittografia a chiave pubblica. Parallelamente, Lov Grover mostrò come un computer quantistico permetta di eseguire ricerche in database non strutturati con un guadagno quadratico in velocità rispetto ai metodi di computazione classica [4].
Tuttavia, l'implementazione pratica di simili algoritmi richiede dispositivi dotati di un elevato numero di qubit e circuiti profondi, condizioni necessarie per una reale applicazione di interesse. Inoltre, per garantire una corretta elaborazione dell'informazione è fondamentale mantenere i qubit in uno stato di coerenza e ciò rischia di essere un vero e proprio "collo di bottiglia" poiché essi risultano estremamente sensibili al rumore e quindi soggetti a errori durante l'esecuzione di calcoli.

In questa prospettiva, la computazione quantistica odierna si colloca in quella che viene definita era NISQ (*Noisy Intermediate-Scale Quantum*), espressione coniata da John Preskill nel 2018 per descrivere la fase attuale di sviluppo di dispositivi quantistici [5]. I processori NISQ dispongono di un numero limitato di qubit — generalmente tra poche decine e qualche centinaio — e sono intrinsecamente affetti da rumore e decoerenza. Sebbene questo tipo di dispositivi non consenta ancora una computazione pienamente *fault-tolerant*, essi rappresentano un passaggio cruciale verso questa direzione, poiché permettono la sperimentazione di nuove architetture, protocolli di mitigazione del rumore e soprattutto tecniche di correzione dell'errore sempre più sofisticate. In questo contesto, gli studi e le ricerche di tecniche e protocolli per la correzione degli errori — *Quantum Error Correction* (QEC) — rivestono un ruolo fondamentale per il raggiungimento di un livello di computazione *fault-tolerant*.

Questa nuova frontiera del *Quantum Computing*, a differenza della computazione classica in cui è sufficiente affidarsi ad una semplice ridondanza dell'informazione, si deve necessariamente confrontare con alcuni principi della meccanica quantistica che

complicano notevolmente la soluzione. In particolar modo, il teorema di non clonazione impedisce la copia di stati quantistici arbitrari, e dunque di garantire una memoria dello stato di un qubit, mentre come se non bastasse il collasso dello stato dovuto alla misura rende impossibile una diagnosi dell'errore tramite misure dirette sullo stato. Per questo motivo, le tecniche di *Quantum Error Correction* devono ricorrere a schemi di codifica indiretta che consentano di rilevare e correggere errori senza distruggere lo stato logico. Negli ultimi decenni, sono stati sviluppati diversi modelli teorici di QEC, tra cui i codici a stabilizzatori e, in particolare, i *Surface Codes*, oggi considerati tra i più promettenti per un'implementazione sperimentale su larga scala [6, 7].

Una componente cruciale per questo tipo di codici è rappresentata dal decoder, ossia l'algoritmo incaricato di interpretare le sindromi di errore prodotte dal codice quantistico e di determinare la correzione più appropriata. La complessità di simili codici — e di conseguenza il problema della decodifica — cresce rapidamente con la dimensione del codice e pertanto per una reale applicazione pratica negli ultimi anni sono stati proposti approcci basati su metodi di *Machine Learning* e, più recentemente, su *Reinforcement Learning* (RL), per sviluppare strategie di decodifica adattive e più efficienti [8, 9].
Tuttavia, un decoder basato su RL — sebbene sia intrinsecamente adatto per generalizzare il comportamento in ambienti con differenti caratteristiche di rumore — rimane limitato dai tempi di addestramento, che crescono rapidamente all'aumentare della dimensione e complessità del codice. Per superare questo limite, sono state proposte tecniche di *continual learning* per rendere il processo di addestramento più efficiente e migliorare l'adattamento del decoder ai variazioni nel modello di rumore, sfruttando le esperienze accumulate in ambienti simili [10].

Capitolo 1

Quantum Error Correction e Surface Code

In questo primo capitolo vengono introdotti gli aspetti teorici relativi al problema posto all'agente sviluppato in questo progetto. Si parte da un breve richiamo sugli elementi fondamentali di *Quantum Computing* (QC) per fornire il contesto necessario alla comprensione dei concetti successivi. Si prosegue quindi con la presentazione della *Quantum Error Correction* (QEC), illustrando i principi alla base della protezione dell'informazione quantistica e il formalismo matematico su cui si fondano i codici correttivi. Successivamente viene analizzato il caso del *Surface Code* — il modello specifico di codifica utilizzato in questo progetto — con particolare attenzione alla struttura e alla rilevazione degli errori. Il capitolo si conclude delineando il compito che verrà concretamente affidato all'agente *Reinforcement Learning*, incaricato di svolgere il ruolo di decoder nel contesto del *Surface Code*.

1.1 Richiami sulla computazione quantistica

In primo luogo vengono introdotti gli elementi di *Quantum Computing* necessari per la comprensione del contesto e degli obbiettivi della *Quantum Error Correction*.

1.1.1 Qubit

Il qubit è l'unità fondamentale di informazione nel calcolo quantistico [11]. Si tratta di un sistema a due livelli e cioè un oggetto che matematicamente può essere rappresentato da un vettore in uno spazio di Hilbert di dimensione due. Per questo motivo, utilizzando la notazione di Dirac e definendo i vettori della *base computazionale* $|0\rangle$ e $|1\rangle$, un *quantum bit* può essere rappresentato come una sovrapposizione dei due stati

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \quad |\alpha|^2 + |\beta|^2 = 1$$

a differenza di un bit classico che può assumere unicamente valori 0 e 1 distinti. I coefficienti complessi α e β rappresentano le ampiezze di probabilità: in accordo con gli assiomi della meccanica quantistica, l'operazione di misura comporta infatti il collasso del qubit in uno dei due autostati $|0\rangle$ o $|1\rangle$, rispettivamente con probabilità $|\alpha|^2$ e $|\beta|^2$.
Senza violare in alcun modo la normalizzazione, è sempre possibile esprimere le ampiezze di probabilità in termini di due parametri $\theta \in [0, \pi]$ e $\phi \in [0, 2\pi]$. In questo

modo, a meno di una fase globale trascurabile, lo stato del qubit può essere scritto come

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle$$

e dunque rappresentato come un punto sulla *sfera di Bloch* (fig. 1.1).

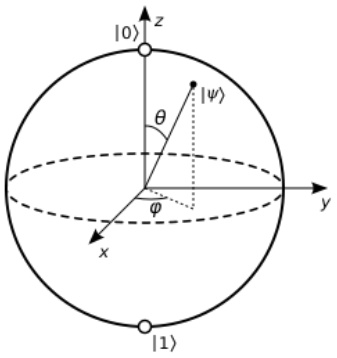


Figura 1.1: Rappresentazione di un qubit sulla *sfera di Bloch*

Fino ad ora si è considerato un sistema composto da un qubit singolo. In generale, dati n qubit, lo stato che descrive un sistema simile è una combinazione lineare di 2^n autostati possibili, ossia un vettore appartenente al prodotto tensoriale degli spazi di Hilbert $H^{\otimes n}$. Un tale oggetto è in grado di codificare una quantità esponenzialmente crescente di informazioni rispetto al numero di qubit coinvolti, gettando le basi per il parallelismo intrinseco che costituisce uno dei principali vantaggi del calcolo quantistico rispetto alla sua controparte classica.

Inoltre, considerare un sistema definito in questo modo da origine a fenomeni profondamente non classici che giocano un ruolo chiave in computazione quantistica, come l'*entanglement*.

1.1.2 Quantum Gate

Le operazioni logiche sui qubit sono rappresentate da operatori unitari analoghi alle porte logiche della computazione classica, ma in grado manipolare la sovrapposizione di stati. Per analizzare i principali operatori logici della computazione quantistica è conveniente fare riferimento agli strumenti dell'algebra lineare, in particolar modo alle rappresentazioni vettoriali e matriciali degli spazi di Hilbert.
Definiti i vettori della base $|0\rangle = (1\ 0)^T$ e $|1\rangle = (0\ 1)^T$, i principali esempi di operatori a singolo qubit sono riportati di seguito.

- **X-gate:**
  - Rappresentazione matriciale : $\mathbf{X} = \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
  - Agisce come un NOT quantistico: $X|0\rangle = |1\rangle$ e $X|1\rangle = |0\rangle$.
- **Z-gate:**
  - Rappresentazione matriciale: $\mathbf{Z} = \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

- Applica un cambio di fase a $|1\rangle$: $Z|0\rangle = |0\rangle$ e $Z|1\rangle = -|1\rangle$.

- **Hadamard-gate:**

  - Rappresentazione matriciale: $\mathbf{H} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
  - Crea la sovrapposizione degli stati: ad esempio $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

Oltre alle operazioni a singolo qubit, è importante menzionare l'esistenza di operatori logici multi-qubit, anch'essi costruiti mediante prodotto tensoriale di operatori a un qubit. Un esempio fondamentale è l'operatore **CX**, che rappresenta l'equivalente quantistico della porta logica classica XOR. Esso agisce applicando X al secondo qubit (*target*) solo se il primo (*control*) è $|1\rangle$. La sua rappresentazione matriciale nella *base computazionale* $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ è la seguente.

$$\mathbf{CX} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Un algoritmo di computazione quantistica consiste quindi nell'applicazione sequenziale di *quantum gate* su specifici qubit. È utile visualizzare il set di operatori logici applicati in successione tramite *diagrammi di circuito*, come quello mostrato di seguito[1] (fig. 1.2).

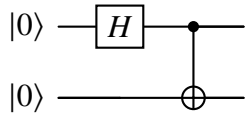


Figura 1.2: Esempio diagramma di circuito stato di Bell $|\Phi^+\rangle$.

1.1.3 Rumore ed errori quantistici

Fino ad ora i qubit sono stati considerati puramente come oggetti matematici perfetti. Nella realtà, seppur con caratteristiche differenti legate alla tecnologia con cui vengono realizzati [12], i qubit sono stati quantistici facilmente soggetti a fenomeni di decoerenza dovuti all'interazione con l'ambiente esterno o imperfezioni nei dispositivi con cui vengono realizzati. Questo tipo di interazioni alterano lo stato quantistico del sistema, introducendo errori che compromettono l'informazione codificata e, di conseguenza, il calcolo basato su di essa.
A differenza della computazione classica, dove l'unico tipo di errore che può verificarsi è il *bit-flip*, un sistema quantistico può essere soggetto a diversi tipi di rumore, i cui principali sono:

- **Bit-flip Noise**: analogamente al caso classico, corrisponde all'inversione dello stato di un qubit. È rappresentato da un'applicazione dell'operatore logico X.

[1] Il circuito riportato rappresenta la preparazione dello stato di Bell $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, l'esempio più semplice di entanglement.

- **Phase-flip Noise**: altera la fase dello stato quantistico. È rappresentato da un'applicazione dell'operatore logico Z.
- **Depolarizing Noise**: errore generale che combina i due precedenti, modellando in maniera efficace il rumore reale presente in un hardware quantistico.

Dunque per la realizzazione di una computazione *fault-tolerant* diventa necessario sviluppare tecniche di correzione di questi errori [13]. Queste tecniche però devono necessariamente confrontarsi con due principi fondamentali della meccanica quantistica: il collasso dello stato quantistico all'atto di misura e il teorema di non-clonazione quantistica (*no cloning theorem*). In virtù di questi principi infatti non è possibile misurare direttamente lo stato quantistico di un sistema — e quindi rilevare eventuali errori — senza comprometterne l'informazione.
Questa esigenza introduce il concetto di *Quantum Error Correction*.

1.2 Principi della Quantum Error Correction

La *Quantum Error Correction* (QEC) — correzione degli errori quantistici — è l'insieme delle tecniche utilizzate per proteggere l'informazione quantistica da eventuali errori dovuti a rumore, decoerenza o imperfezioni dei dispositivi.

In genere, viene costruito un insieme di qubit *logici* a partire da più qubit *fisici* entangled, gli errori locali vengono rilevati tramite misure indirette di opportuni operatori — *stabilizzatori* — e infine un decoder determina le correzioni da applicare per ripristinare lo stato originale (fig. 1.3).

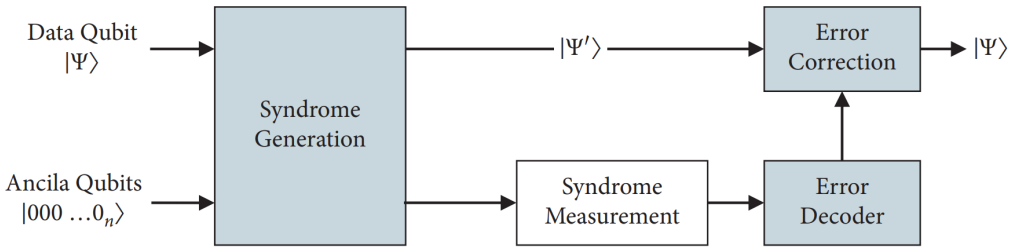


Figura 1.3: Esempio di uno schema tipico di *Quantum Error Correction*.

Questo ciclo di codifica, rilevamento e correzione può essere ripetuto periodicamente durante il calcolo, permettendo di mantenere la coerenza dell'informazione anche in presenza di rumore e decoerenza.

1.2.1 Qubit fisici e qubit logici

Le principali tecniche di *Quantum Error Correction* oggi utilizzate si basano su protocolli di codifica — e successiva decodifica — dell'informazione attraverso l'utilizzo di molteplici qubit. L'idea è di utilizzare più qubit *fisici* per la realizzazione di un singolo qubit *logico* e quindi di preservare non tanto lo stato di coerenza microscopico quanto più la coerenza dell'informazione a livello macroscopico.

Come anticipato, a differenza del caso classico in cui è sufficiente duplicare l'informazione ed immagazzinarla su più bit, nel caso quantistico questo non è possibile a causa del teorema di non-clonazione. Esistono invece diversi metodi per codificare

l'informazione in stati entangled di più qubit *fisici* in modo che sia distribuita tra essi e possa essere recuperata qualora si verifichi un errore locale.
Si consideri ad esempio uno stato logico dato dalla sovrapposizione entangled di tre qubit fisici (1.1), che di fatto rappresenta un'estensione quantistica del *repetition code* classico.

$$|\psi\rangle = \alpha\,|0\rangle_L + \beta\,|1\rangle_L = \alpha\,|000\rangle + \beta\,|111\rangle \tag{1.1}$$

Classicamente, se si verifica un errore ad esempio bit-flip, per ripristinare lo stato logico corretto è sufficiente osservare i bit e correggere quello errato seguendo la maggioranza. In meccanica quantistica, una misura dello stato comporterebbe la perdita irreversibile dell'informazione immagazzinata nelle ampiezze di probabilità del qubit. Per questa ragione, le tecniche di *Quantum Error Correction* si basano piuttosto su controlli della conservazione di parità (*parity checks*) per diagnosticare dove localmente si è verificato l'errore, evitando il collasso dello stato logico [14], [15].
Nel caso del *repetition code* a 3 qubit, è possibile effettuare una misura della parità mediante gli operatori Z_1Z_2 e Z_2Z_3 applicati allo stato logico corrotto. I risultati per ogni eventuale errore sono riportati in tab. 1.1.

Errore	**Stato corrotto**	Z_1Z_2	Z_2Z_3
Nessuno	$\alpha\|000\rangle + \beta\|111\rangle$	$+1$	$+1$
X_1	$\alpha\|100\rangle + \beta\|011\rangle$	-1	$+1$
X_2	$\alpha\|010\rangle + \beta\|101\rangle$	-1	-1
X_3	$\alpha\|001\rangle + \beta\|110\rangle$	$+1$	-1

Tabella 1.1: Misure degli stabilizzatori Z_1Z_2 e Z_2Z_3 per il codice a tre qubit

Il fatto che ad ogni esito della misura di questi operatori sia possibile associare un solo errore in maniera biunivoca permette quindi di identificare il qubit su cui si è verificato l'errore ed eventualmente correggerlo. Resta però da discutere come si possa effettuare una misura indiretta di questi operatori, evitando il collasso dello stato.
Intuitivamente si può verificare che un circuito come quello in fig. 1.4 può essere implementato per la misura indiretta dello stabilizzatore Z_1Z_2. Seguendo schemi simili, a patto di implementare qubit *ancilla* e limitare ad essi la misura dello stato, è sempre possibile ottenere una diagnosi dell'errore preservando l'informazione logica, anche se questo comporta una spesa maggiore in termini di qubit.

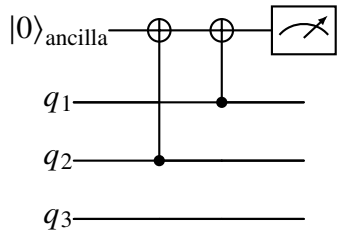


Figura 1.4: Esempio di circuito per misura indiretta degli stabilizzatori Z_1Z_2.

La logica presentata brevemente in questo esempio è quella su cui si basa il formalismo degli stabilizzatori introdotto da Gottesman [16] su cui sono costruiti la maggior parte degli schemi di *Quantum Error Correction*.

Si noti che il modello di *"quantum" repetition code* presentato in questo esempio non è adatto alla diagnosi di errori quantistici di tipo *phase-flip* tramite il formalismo introdotto. Una codifica sensibile a tali errori richiederebbe infatti uno stato logico basato sull'entaglement di qubit proiettati sugli autostati dell'operatore X , ovvero

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}},$$

piuttosto che sulla base computazionale. In tal caso, lo stato logico assumerebbe la forma:

$$|\psi'\rangle = \alpha\,|+++\rangle + \beta\,|---\rangle\,.$$

Con una simile codifica, gli stabilizzatori necessari per la diagnosi sarebbero, al pari di quanto visto in precedenza, X_1X_2 e X_2X_3. Tuttavia, in questo modo il sistema perderebbe la facoltà di correggere errori di *bit-flip*: risulta quindi evidente che un sistema composto da soli tre qubit fisici non consente di implementare una codifica robusta in grado di correggere tutte le tipologie di errore quantistico.

L'evoluzione naturale di questo modello è rappresentata dal codice di Shor [17] che utilizza una codifica degli stati logici più complessa (1.2) basata su 9 qubit fisici, ma capace di individuare e correggere sia errori di *bit-flip* sia di *phase-flip*.

$$\begin{aligned} |0\rangle_L &= \frac{(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)}{2\sqrt{2}} \\ |1\rangle_L &= \frac{(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)}{2\sqrt{2}} \end{aligned} \tag{1.2}$$

1.2.2 Formalismo degli stabilizzatori

Viene introdotto ora in maniera più rigorosa il formalismo degli stabilizzatori di Gottesman [16, 14], che costituisce il quadro matematico in cui vengono formulati la maggior parte degli schemi di *Quantum Error Correction*.

Gli *stabilizer codes* [11, 18] sono codici di correzione quantistica che impiegano determinati operatori, detti stabilizzatori, per effettuare dei *parity checks* sullo stato logico, eventualmente corrotto. La misura di tali operatori produce una sindrome (*syndrome*) che, se opportunamente decodificata, consente di ottenere le istruzioni sulle operazioni da effettuare per correggere lo stato logico.

Un gruppo di Pauli per un sistema di n qubit è l'insieme di tutti gli operatori che si possono ottenere combinando, per ogni qubit, gli operatori di Pauli I, X, Y, Z, tramite prodotto tensoriale, eventualmente moltiplicati per un fattore di fase ± 1 o $\pm i$.

$$P_n = \{\omega P_1 \otimes \cdots \otimes P_n : P_i \in \{I, X, Y, Z\}, \omega \in \{\pm 1, \pm i\}\} \tag{1.3}$$

Uno stato logico $|\psi\rangle$ si dice *stabilizzato* da un operatore $M \in P_n$ se è autostato di M con autovalore +1.

$$M\,|\psi\rangle = |\psi\rangle \tag{1.4}$$

L'insieme di tutti gli operatori di Pauli che stabilizzano uno stato forma uno *stabilizer group* $\mathcal{S}$ (gruppo stabilizzatore), che è un sottogruppo abeliano del gruppo di Pauli P_n.

$$\mathcal{S} = \{M_i \in P_n : M_i\,|\psi\rangle = |\psi\rangle\,, [M_i, M_j] = 0, \forall (i, j)\} \tag{1.5}$$

Il sottospazio costituito dai vettori simultaneamente autostati con autovalore +1 di tutti gli elementi di $\mathcal{S}$ è detto *codespace* C del codice stabilizzatore.

$$C = \{|\psi_i\rangle : S|\psi_i\rangle = |\psi_i\rangle, \forall S \in \mathcal{S}\} \tag{1.6}$$

In generale, il gruppo stabilizzatore $\mathcal{S}$ può essere descritto a partire da un insieme minimo di operatori indipendenti $\mathcal{G} = \{M_1, M_2, \cdots, M_{n-k}\}$, detti *generatori*, dove n è il numero di qubit fisici utilizzati per codificare k qubit logici.

$$S = \langle M_1, M_2, \cdots, M_{n-k}\rangle \tag{1.7}$$

Gli operatori dell'insieme $\mathcal{G}$ rappresentano l'insieme minimo di operatori che devono essere applicati in una reale implementazione fisica per determinare la sindrome d'errore.

In un codice quantistico gli errori vengono rappresentati da operatori di Pauli $E \in P_n$. Se un errore E anticommuta con uno o più generatori M_i, la misura di tali operatori restituisce un autovalore -1, producendo una sindrome che può essere utilizzata per identificare la posizione e la natura dell'errore. Dato lo stato corrotto $|\tilde{\psi}\rangle = E|\psi\rangle$:

$$\{M_i, E\} = M_iE + EM_i = 0 \tag{1.8}$$

$$M_i|\tilde{\psi}\rangle = M_iE|\psi\rangle = -EM_i|\psi\rangle = -E|\psi\rangle = -|\tilde{\psi}\rangle \tag{1.9}$$

In questo modo la misura dei generatori consente di rilevare e correggere le trasformazioni non desiderate senza collassare l'informazione logica codificata.

Esempio. Si consideri l'estensione quantistica del *repetition code* a 3 qubit per la correzione di bit-flip. Poiché $n = 3$ e $k = 1$, per costruire il gruppo stabilizzatore e una sindrome efficace sono necessari 2 operatori che, come è stato mostrato in precedenza, risultano essere $M_1 = Z_1Z_2$ e $M_2 = Z_2Z_3$. Dunque, secondo il formalismo introdotto:

$$C = \{\alpha|000\rangle + \beta|111\rangle : \alpha, \beta \in \mathbb{C}\}$$

$$\mathcal{G} = \{Z_1Z_2, Z_2Z_3\}$$

$$\mathcal{S} = \langle Z_1Z_2, Z_2Z_3\rangle = \{I, Z_1Z_2, Z_2Z_3, Z_1Z_3\}$$

Il compito di determinare, data una sindrome misurata, le operazioni più appropriate per riportare il sistema nello stato logico originale costituisce l'essenza del problema di decodifica (*decoding problem*) che verrà approfondito in seguito per il caso specifico del *Surface Code*.

1.2.3 Il teorema della soglia

Viene presentato brevemente uno dei risultati storici fondamentali per quanto riguarda la correzione degli errori quantistici, ovvero il teorema della soglia o (*Threshold theorem*) [19, 14].

Data una certa probabilità p di errore durante le operazioni logiche o le misure su un singolo qubit fisico, il teorema garantisce che, se p è sufficientemente piccolo, le tecniche di *Quantum Error Correction* possono ridurre la probabilità di errore logico p_L del qubit codificato. In particolare, il teorema afferma che esiste un valore di soglia p_{th} specifico del codice tale che, se $p < p_{th}$, è possibile rendere la probabilità

di errore logico p_L arbitrariamente piccola aumentando la profondità della codifica, e cioè utilizzando un numero maggiore di qubit fisici. In termini matematici,

$$p_L \sim \left(\frac{p}{p_{th}}\right)^d$$

dove d rappresenta la distanza del codice, cioè il numero minimo di errori indipendenti necessari per modificare lo stato logico.
Nel concreto, il teorema è una garanzia formale della scalabilità: se la probabilità di errore p è minore della soglia, aumentando la distanza del codice è possibile teoricamente un calcolo quantistico *fault-tolerant* con probabilità di errori logici p_L arbitrariamente piccola. Al di sopra della soglia invece il codice accumula più errori di quelli che riesce a correggere, rendendo il sistema totalmente instabile.

Questo consente all'atto pratico di valutare l'efficienza di diversi schemi di QEC. Metodi di codifica come il codice di Shor precedentemente introdotto appartengono ad una classe di codici chiamati *codici concatenati*. Questo tipo di codice è caratterizzato una soglia troppo bassa — $p_{th} \sim (10^{-4} - 10^{-5})$ — se confrontata con le probabilità di errore che caratterizzano le tecnologie di qubit fisici allo stato attuale, rendendo difficile la realizzazione sperimentale. Inoltre, aumentare la distanza di simili codici richiede una crescita esponenziale in termini di qubit necessari, il che rende la scalabilità un problema non indifferente. Nel caso del codice di Shor, ad esempio:

- per $d = 1$ sono necessari 9 qubit[2]
- per $d = 2$ sono necessari $9^2 = 81$ qubit
- per $d = 3$ sono necessari $9^3 = 729$ qubit e così via.

Un'ottima soluzione è quindi costituita da un insieme di codici detti *topologici*, tra cui un buon candidato è il *Surface Code* per motivi pratici e soprattutto per i suoi alti livelli di soglia $p_{th} \sim 1\%$ [20].

1.3 Surface Code

Il *Surface Code* rappresenta una particolare implementazione del *codice toroidale* di Kitaev [6], in cui le condizioni periodiche al contorno vengono sostituite con condizioni aperte. Appartiene alla più ampia classe dei *codici topologici* in cui l'informazione logica non è immagazzinata in un singolo qubit fisico, ma è distribuita nello spazio e protetta dalle proprietà globali del sistema. In questi codici, gli errori locali (che coinvolgono pochi qubit adiacenti) generano dei difetti confinati che non compromettono lo stato logico complessivo. Affinché lo stato logico subisca una trasformazione, sarebbe necessario un *loop* di errori che attraversi l'intera superficie, realizzando di fatto un operatore logico. Questo meccanismo conferisce al *Surface Code* una robustezza intrinseca contro gli errori locali, caratteristica tipica dei sistemi topologici.

[2] Si noti infatti che negli esempi precedentemente introdotti non è stato mai menzionato il caso di molteplici errori, poiché questo renderebbe *non biunivoca* la relazione tra una sindrome e i casi di errori possibili, e quindi di fatto il codice non adatto.

1.3.1 Struttura, stabilizzatori e rilevamento degli errori

Immaginiamo il *Surface Code* [9] come una griglia di dimensione $d \times d$ in cui ogni vertice v rappresenta un qubit fisico, come rappresentato in fig.1.5(a) per il caso $d = 5$.

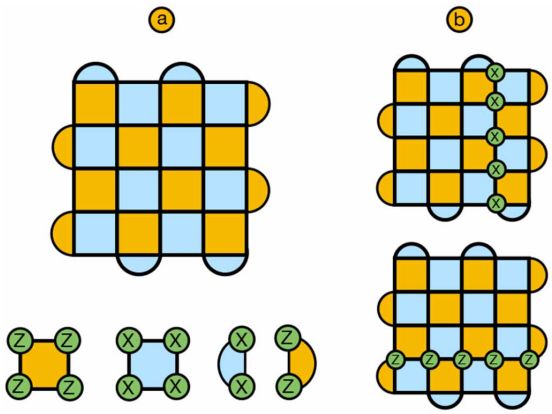


Figura 1.5: Rappresentazione di un *Surface Code* 5×5 e degli operatori logici X_L e Z_L [9].

Formalmente, lo stato collettivo di tutti i qubit sulla superficie appartiene allo spazio di Hilbert $H = \mathbb{C}^{2^{d \times d}}$. A ogni cella blu e a ogni cella arancione si associa uno stabilizzatore che agisce sui quattro qubit ai vertici della cella, applicando operatori rispettivamente di tipo X e Z, sensibili a errori di *phase-flip* e *bit-flip*. In particolare, se chiamiamo B_c e A_c gli insiemi di tutte le celle blu e arancioni, è possibile definire lo stabilizzatore S_c su ciascuna cella c come

$$S_c = \bigotimes_{v \in c} \sigma_v \quad con \quad \begin{cases} \sigma_v = X_v & se\ c \in B_c \\ \sigma_v = Z_v & se\ c \in A_c \end{cases} \tag{1.10}$$

Il sottospazio H_{sc} generato da tutti gli autostati simultanei +1 di questi stabilizzatori è uno spazio di Hilbert di dimensione 2 e costituisce il *code space* adatto per rappresentare un singolo qubit logico. Gli operatori logici sono operatori che preservano la coerenza dello stato e possono quindi essere utilizzati per manipolare lo stato del qubit logico. La fig. 1.5(b) mostra gli operatori logici X_L e Z_L, che sono stringhe continue di operatori X e Z che collegano rispettivamente i bordi opposti della superficie.

Nel momento in cui si verifica un errore, rappresentato dall'applicazione indesiderata di un operatore X o Z su un qubit fisico, gli stabilizzatori adiacenti ad esso assumono una autovalore -1, segnalando la presenza del difetto (cfr. fig. 1.6).



Figura 1.6: Risposta degli stabilizzatori a ciascuna tipologia di errore quantistico [9].

La sindrome sarà data dalla misura simultanea di tutti gli stabilizzatori della superficie. Si noti che nel caso in cui si verifichino più errori contemporaneamente, le sindromi possono sovrapporsi e la stessa configurazione di misure può corrispondere a diverse

catene di errori (cfr. sez. 1.4, fig. 1.7). Questo rende il processo di decodifica non banale, richiedendo algoritmi efficienti per individuare la catena più probabile compatibile con la sindrome osservata.

1.3.2 Scalabilità e vantaggi

È importante sottolineare che, a differenza dei *codici concatenati*, il *Surface Code* presenta una scalabilità molto più favorevole [7, 21]. Mentre nel codice di Shor si è visto che il numero di qubit cresce esponenzialmente con la distanza d del codice, in questo caso esso cresce quadraticamente con d, ovvero linearmente con la dimensione della superficie[3]. Questo significa che aumentando la distanza topologica del codice, si ottiene un errore logico che decresce esponenzialmente pur utilizzando un numero di qubit inferiore rispetto ad altre tipologie di codice, il che costituisce una caratteristica chiave per la scalabilità del calcolo quantistico *fault-tolerant*. Inoltre la natura 2D di questi schemi si integra perfettamente con le tecnologie attualmente più utilizzate — come qubit superconduttivi e trappole ioniche — in cui le interazioni tra qubit adiacenti sono realizzabili con alta fedeltà, mentre quelle a lunga distanza risultano molto più complesse.

Grazie a queste caratteristiche e a una soglia dell'errore elevata (fino a circa l'1%) il *Surface Code* rappresenta oggi una delle principali architetture di riferimento per la realizzazione di processori quantistici *fault-tolerant*.

1.4 Decoder per Surface Code

Per poter sfruttare appieno le proprietà di scalabilità del *Surface Code*, è necessario implementare meccanismi efficienti di estrazione e decodifica della sindrome, che permettano di individuare e correggere gli errori locali in tempo reale. Viene introdotto quindi il problema della decodifica [9], che formalmente può essere presentato nel seguente modo:

Decoding Problem. Immaginiamo che a $t = 0$ venga fornito un vettore di stato $|\psi\rangle = \alpha\,|0\rangle_L + \beta\,|1\rangle_L$. A un certo istante $t_1 > 0$, viene eseguita una misurazione della sindrome che indica che uno o più stabilizzatori sono violati, ovvero che si sono verificati errori sui qubit fisici. A partire dalla sindrome ottenuta, si determini un insieme di correzioni da applicare al reticolo del codice in modo che lo stato successivo $|\psi'\rangle$ sia uguale allo stato iniziale $|\psi\rangle$.

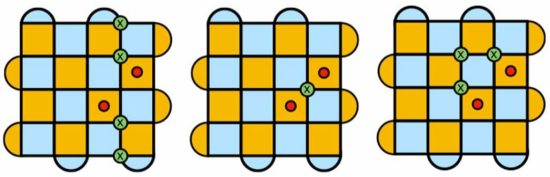


Figura 1.7: Esempi di catene di errori con sindrome equivalente [9].

[3] Per modificare lo stato logico è necessario un operatore logico e dunque una stringa di errori che attraversa la superficie di dimensione d.

Come anticipato, la difficoltà di questo problema risiede nel fatto che la sindrome e le possibili catene di errori che possono generarla non sono in corrispondenza biunivoca. Si considerino ad esempio gli esempi di fig. 1.7. Se la probabilità di errore p è relativamente bassa, è ragionevole immaginare che una sindrome del genere sia generata da un singolo errore e dunque con alta probabilità dal secondo caso della figura, il che porta ad applicare un operatore X sul qubit corrotto. Tuttavia, è importante studiare che conseguenze potrebbe avere questa scelta nel caso in cui la sindrome fosse generata da un diverso tipo di catena di errori. Prendiamo ad esempio il terzo caso di fig. 1.7: applicare X sul qubit in questione risulterebbe equivalente ad applicare uno stabilizzatore e, poiché si è detto che lo stato logico è costruito nel sottospazio degli autostati +1 di tutti gli stabilizzatori, questo correggerebbe comunque l'errore locale. Guardiamo invece al primo caso di fig. 1.7: qui l'operatore X chiuderebbe la catena di operatori che attraversa il reticolo da parte a parte, il che corrisponderebbe ad applicare un operatore logico X_L, compromettendo l'informazione globale codificata.

1.4.1 Formalismo omologico e classi di equivalenza

Da questo semplice esempio si può vedere che il più delle volte la risoluzione del problema di decodifica come affermato sopra implica decidere, data una sindrome intrinsecamente ambigua e una conoscenza — spesso imperfetta — del modello di errore, quale configurazione di errore si sia verificata con maggiore probabilità. La struttura matematica alla base del problema di decodifica per *Surface Code* è quella dell'omologia, che fornisce una rappresentazione formale della relazione tra errori e sindromi [22]. In questo linguaggio, un decoder determina la probabilità relativa di approssimare le diverse classi di omologia, e cioè classi di equivalenza di modelli di errore, a determinate configurazioni di errore catturate dalla sindrome.

In riferimento all'esempio presentato, i casi 2 e 3 della Figura 1.7 appartengono alla stessa classe di omologia, detta *triviale*. Pertanto, quando il decoder applica l'operazione di correzione più probabile, l'effetto combinato di errore e correzione si annulla, chiudendo entrambe le catene con un *loop triviale*, che ripristina lo stato logico. Diverso è il primo caso che, pur producendo la stessa sindrome, appartiene a una classe di omologia *non-triviale*. Pertanto, se si applica la stessa operazione di correzione, l'errore reale e la correzione si combinano per formare un *loop non-triviale*, ovvero una catena che attraversa il reticolo da bordo a bordo, risultando nell'applicazione di un operatore logico X_L che compromette l'informazione codificata [23].

1.4.2 Sindromi rumorose e decodifica *Fault-tolerant*

Un altro aspetto da considerare nella progettazione di un algoritmo di codifica è l'eventualità di una sindrome soggetta ad errori (*faulty-syndrome*). In precedenza è stato mostrato come, tramito l'utilizzo di qubit ancilla e appositi circuiti, è possibile praticare una misura indiretta degli stabilizzatori. Tuttavia questi circuiti e i qubit ancilla, sono oggetti quantistici soggetti ad errori al pari dei qubit di dati, e come tali introducono fonti di errore aggiuntivo. Di fronte a sindromi intrinsecamente inaffidabili, il decoder deve essere robusto e operare in un contesto di *fault-tolerance*. Un approccio essenziale è quello di raccogliere una sequenza di misure di sindrome nel tempo per costruire una statistica più accurata sulla diagnosi dei difetti. Questa metodologia è risolutiva, ma introduce una variabile cruciale: il tempo aggiuntivo necessario per effettuare più

misure sequenziali incrementa la probabilità che si verifichino nuovi errori reali sui qubit di dati, complicando ulteriormente la decodifica temporale, come si può intuire dalla fig. 1.8.

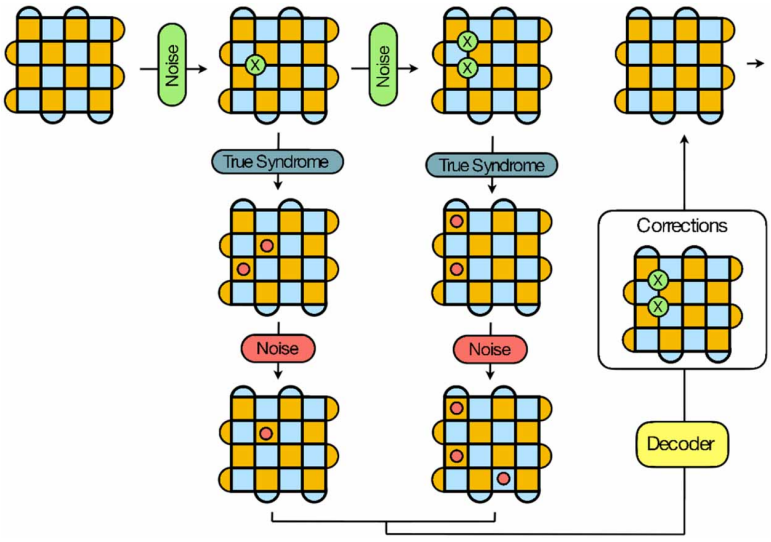


Figura 1.8: Conseguenze di una misura sequenziale degli stabilizzatori per la gestione di *faulty-syndrome* in un decoder [9].

In ultima istanza, una caratteristica fondamentale che il decoder deve possedere è la rapidità di calcolo. In un eventuale utilizzo reale, data una sindrome in input — eventualmente rumorosa e sequenziale — il decoder deve essere in grado di suggerire operazioni correttive in tempo reale. In particolare, affinché il codice abbia successo, il decoder deve essere in grado di completare il ciclo di diagnosi e correzione ad una velocità superiore a quella con cui gli errori si verificano, rispettando il vincolo imposto dalla soglia di *fault-tolerance* del sistema.

1.4.3 Approcci di decodifica

Attualmente esistono diversi esempi di algoritmi per *decoding problem*, distinti principalmente in metodi probabilistici classici e metodi basati sul *Machine Learning* (ML).

L'algoritmo classico che gode di maggior considerazione è il *Minimum Weight Perfect Matching* [24], noto come MWPM. Questo algoritmo modella il problema in modo spaziale (2D) cercando il percorso di errore più probabile — più breve — per i difetti rilevati nella sindrome [23]. Sebbene si tratti di un protocollo semplice concettualmente ed altamente efficiente nel regime di sindromi non rumorose, la sua maggiore limitazione è che non gestisce nativamente la sequenza di sindromi nel tempo. La sua estensione 3D per gestire un'eventuale sequenza di misure è computazionalmente onerosa, il che compromette la correzione in tempo reale.

Per superare questo limite e gestire il rumore in maniera più flessibile, sono emersi decoder promettenti basati sull'utilizzo di Reti Neurali (NN) e metodi di ML. Questi approcci offrono una soluzione per aggirare il problema della specificità del codice e dell'imperfetta modellazione dell'hardware quantistico, garantendo una migliore adattabilità a modelli di rumore differenti e potenziale per superare le prestazioni dei

metodi convenzionali. Le reti neurali, imparando direttamente dai dati grezzi di misura, possono semplificare il complesso processo di deduzione della sindrome e ottenere una decodifica più veloce [8]. Tuttavia, il limite principale di questa categoria risiede nella scalabilità: il costo e le tempistiche di *training* crescono notevolmente con la dimensione del codice topologico.

In questo scenario, il *Reinforcement Learnin*g (RL) si presenta come una metodologia promettente, ideale per ottimizzare decisioni sequenziali in ambienti dinamici, in cui è necessario un adattamento al modello di errore. In linea con questa direzione, questo progetto si concentra specificamente sullo sviluppo e l'analisi di un decoder simile.

Capitolo 2

Deep Reinforcement Learning

In questo secondo capitolo vengono introdotti i concetti teorici fondamentali per la comprensione dei metodi di *Machine Learning* (ML) applicati al problema di decodifica di *Surface Code* in *Quantum Error Correction*. Si inizia con una panoramica generale dei principi che governano il paradigma del *Reinforcement Learning* (RL), illustrandone i meccanismi di base e il formalismo matematico di riferimento. Successivamente vengono analizzati alcuni tra gli algoritmi di RL più noti e diffusi, propedeutici alla comprensione dell'algoritmo di *Deep learning* utilizzato per l'addestramento dell'agente in questo progetto, ossia il *Double Deep Q-Network* (DDQN). A tal fine viene fornita una breve descrizione del funzionamento delle reti neurali artificiali, elemento chiave negli algoritmi di *Deep Reinforcement Learning* e parte integrante dell'architettura adottata. Il capitolo si chiude con la presentazione di un approccio di *continual learning* impiegato in questo lavoro per dimostrare la maggiore adattabilità di un agente in ambienti caratterizzati da un rumore variabile: il *Probabilistic Policy Reuse* (PPR).

2.1 Fondamenti di Reinforcement Learning

Il *Reinforcement Learning* (RL) è un paradigma di apprendimento basato sull'esperienza, in cui un agente impara a prendere decisioni attraverso l'interazione diretta con un ambiente [25]. Esso costituisce un approccio indipendente all'interno dei principali metodi di *Machine Learning*, accanto al *supervised learning* e all'*unsupervised learning*.

Il *supervised learning* si fonda sull'apprendimento a partire da un set di esempi etichettati forniti da un supervisore esterno esperto. Ogni esempio è costituito dalla descrizione di una situazione particolare associata ad un'etichetta che contiene la specifica azione corretta da intraprendere in quel contesto, che spesso coincide con l'identificare la giusta categoria a cui la situazione appartiene. L'obiettivo di questo tipo di apprendimento è consentire al sistema di generalizzare le sue risposte, in modo tale da permettergli di agire correttamente anche in situazioni non presenti nel set di addestramento. Sebbene questo sia un paradigma fondamentale per il *Machine Learning*, risulta inadeguato per l'apprendimento dall'interazione con un ambiente dinamico: nei problemi interattivi infatti è poco pratico ottenere esempi di comportamento desiderato che siano rappresentativi di tutte le situazioni in cui l'agente potrebbe trovarsi a dover agire.

L'*unsupervised learning* invece consiste nella ricerca di strutture o regolarità nascoste all'interno di insiemi di dati non etichettati. Potrebbe essere naturale considerare il

Reinforcement Learning come una sorta di apprendimento non-supervisionato poiché anch'esso non si basa su esempi espliciti di comportamento corretto, come il caso del *supervised learning*. Tuttavia, la natura dei due paradigmi è profondamente diversa: RL infatti non cerca una struttura nascosta nei dati, bensì punta a massimizzare una ricompensa cumulativa che viene ottenuta come *feedback* delle azioni sequenziali compiute durante l'interazione con l'ambiente.

2.1.1 Agente, ambiente, stato, azione e ricompensa

Il processo di apprendimento per *Reinforcement Learning* avviene tramite uno schema di interazione ciclico che coinvolge agente e ambiente tramite stati, azioni e ricompense.

L'agente costituisce l'entità decisionale che con questo processo si vuole istruire affinché impari la miglior strategia comportamentale, detta politica (*policy*), volta a massimizzare la ricompensa cumulativa.
L'ambiente (*environment*) costituisce invece tutto ciò che circonda l'agente e con cui esso interagisce. Esso contiene tutte le informazioni necessarie a descrivere il problema: le *regole del gioco*[1], l'insieme di tutti i possibili stati in cui l'agente può trovarsi e l'insieme di tutti i possibili riscontri che può ottenere a seconda della qualità dell'azione che viene compiuta. In sostanza, determina la dinamica complessiva dell'esperienza di apprendimento.
Ogni interazione tra agente e ambiente si articola su tre elementi fondamentali: stato, azione e ricompensa. Lo stato rappresenta la descrizione della situazione corrente in cui si trova l'agente e pertanto contiene tutte le informazioni necessarie ad esso per scegliere un'azione adeguata. Le azioni costituiscono invece tutte le possibili decisioni o mosse che l'agente può compiere in risposta allo stato osservato. Si noti che stato e azione, a seconda della descrizione fornita dall'*environment*, possono appartenere rispettivamente ad un insieme finito o infinito, discreto o continuo di elementi. La ricompensa (*reward*) infine è un valore numerico che l'ambiente restituisce come valutazione della dell'utilità immediata dell'azione intrapresa: rappresenta il feedback che guida l'agente nel processo di apprendimento.

L'apprendimento può avvenire tramite episodi — talvolta chiamati epoche — che sono costruiti sul seguente ciclo di interazione. A partire da uno stato iniziale, che può essere determinato casualmente o in modo specifico a seconda dell'ambiente, l'agente seleziona un'azione da eseguire sulla base della sua politica corrente. L'azione viene applicata all'ambiente, che restituisce lo stato in cui l'agente si troverà e una ricompensa che valuta la bontà dell'azione eseguita. Queste informazioni vengono osservate dall'agente e utilizzate rispettivamente per aggiornare la politica e determinare una nuova azione da compiere. Il ciclo procede finché non viene raggiunto l'obiettivo prefissato o un'eventuale condizione di fine episodio: in tal caso il processo di apprendimento riprende con un nuovo episodio e così via, fino al raggiungimento di una strategia ottimale.

[1] L'espressione "regole del gioco" è usata in senso figurato per descrivere le leggi che governano l'interazione tra agente e ambiente. In molti casi è utile interpretare il processo di apprendimento per RL come un gioco in cui l'agente scopre attraverso l'esperienza come le proprie azioni influenzano lo stato e le ricompense ottenute. Non a caso molti dei sistemi di intelligenza artificiale per giochi, come *scacchi*, si basano su RL.

Esempio. Si consideri il caso di un problema didattico per RL, noto come *Frozen Lake*, disponibile nella libreria *Gymnasium*. Questo ambiente è particolarmente adatto ad essere risolto tramite l'algoritmo Q-learning tabellare, che verrà presentato in seguito. L'obiettivo è insegnare ad un agente il percorso ottimale per raggiungere una cella G (*Goal*), partendo da una posizione iniziale S (*Start*), attraverso solo celle F (*Frozen*) ed evitando celle H (*Hole*), che rappresentano buchi nel ghiaccio. Un esempio della configurazione dell'ambiente è presentato in fig. 2.1.

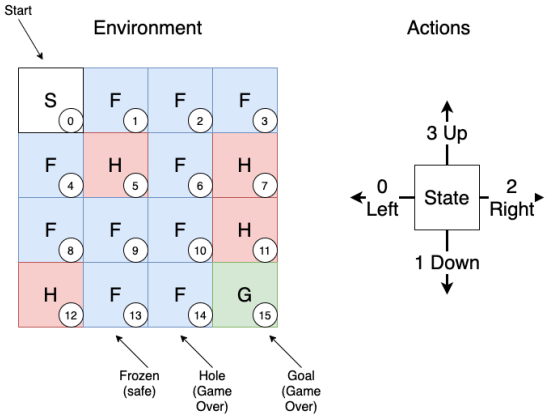


Figura 2.1: Esempio di un problema didattico per RL: *Frozen Lake* [26].

In questo contesto, un episodio consiste in un tentativo completo dell'agente di raggiungere l'obiettivo. La griglia rappresenta l'*environment* con cui l'agente interagisce: le celle rappresentano l'insieme di tutti gli stati accessibili, mentre le quattro direzioni su, giù, destra, sinistra costituiscono l'insieme delle azioni possibili per ogni stato (cfr. fig. 2.1). Per ogni azione, la ricompensa può essere modellata come segue: -1 se lo stato raggiunto è una cella[2] F, -10 e +99 se lo stato raggiunto è rispettivamente una cella H o G. Le transizioni verso celle G o H comportano la fine di un episodio, mentre quelle verso celle F sono semplici passi intermedi.

Per descrivere in maniera rigorosa la sequenza di interazioni tra agente e ambiente è utile introdurre il formalismo dei *Markov Decision Process* (MDP).

2.1.2 Processi Decisionali di Markov

Il formalismo matematico che descrive l'interazione tra agente e ambiente nel contesto di *Reinforcement Learning* è il modello dei Processi Decisionali di Markov (*Markov Decision Process*, MDP) [25]. Un MDP consente di rappresentare un generico problema di decisione sequenziale, in cui l'agente deve scegliere azioni tali da massimizzare una ricompensa cumulativa nel tempo.

In particolare, agente e ambiente interagiscono in una sequenza di passi temporali discreti. Per ogni passo temporale $t = 0, 1, 2, 3\ldots$, l'agente riceve una rappresentazione dello stato $S_t \in \mathcal{S}$ e sulla base di esso seleziona un'azione $A_t \in \mathcal{A}(s)$. L'ambiente risponde restituendo una ricompensa numerica $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ e il nuovo stato corrente S_{t+1}. Nel complesso, l'interazione tra agente e ambiente genera la seguente sequenza temporale:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3, \ldots$$

[2]Si noti che l'obiettivo è trovare il percorso ottimale, dunque anche il più breve: la ricompensa negativa è necessaria per esprimere questo particolare aspetto.

Una rappresentazione schematica di questo ciclo è presente in fig. 2.2.

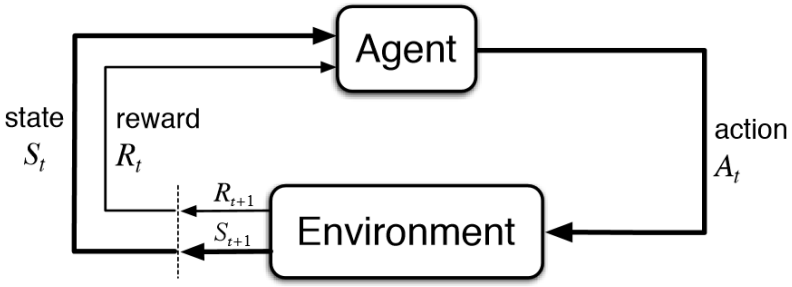


Figura 2.2: Ciclo interattivo agente-ambiente in un Processo Decisionale di Markov [25].

Si noti che in un Processo Decisionale di Markov *finito* — come quello considerato in questo elaborato — l'insieme degli stati, delle azioni e delle ricompense $(\mathcal{S}, \mathcal{A}, \mathcal{R})$ hanno un numero finito di elementi. Pertanto, le variabili R_t e S_t hanno distribuzioni di probabilità discrete ben definite che dipendono solo da stato e azione precedenti.

$$p(s', r \mid s, a) \doteq Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\} \tag{2.1}$$

La funzione p definisce tutta la dinamica del processo decisionale. La caratteristica fondamentale dei MDP è che questa probabilità dipende unicamente da stato e azione immediatamente precedenti e non dall'intera storia dell'interazione. Questa non è da considerarsi una restrizione sul processo decisionale, ma piuttosto una proprietà dello stato stesso. Lo stato deve includere tutte le informazioni rilevanti rispetto all'interazione passata agente-ambiente, necessarie per predire il futuro: in tal caso, si dice che lo stato possiede la *proprietà Markoviana*.

Infine, l'obiettivo dell'agente è quello di massimizzare la ricompensa cumulativa scontata, definita come

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.2}$$

dove γ è un parametro $0 \leq \gamma \leq 1$ chiamato fattore di sconto. Il ruolo di questo parametro è quello di controllare il bilanciamento tra l'importanza delle azioni immediate e future. La sommatoria è teoricamente infinita nel caso di apprendimento continuo (*continuing task*), mentre nel caso di apprendimento episodico il numero di step temporali risulta limitato alla durata dell'episodio. Questa funzione può essere riscritta in forma ricorsiva, come riportato di seguito.

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{2.3}$$

2.1.3 Politiche, funzioni valore ed equazioni di Bellman

Per formalizzare e definire il problema di decisione ottimale è necessario introdurre i concetti di politica e funzione valore. La maggior parte degli algoritmi di RL si basano sulla stima di funzioni valore, utili per interpretare "quanto è buono" uno stato o un'azione, secondo un criterio che è fornito dalla ricompensa cumulativa attesa che un determinato stato o un'azione possono generare.

Una politica π è una regola che specifica, in maniera deterministica o stocastica, quale azione verrà scelta dall'agente in ciascuno stato visitato. Formalmente, una *policy* stocastica è quindi una funzione $\mathcal{S} \times \mathcal{A}(s) \rightarrow [0, 1]$ che associa ad ogni stato la probabilità di eseguire ciascuna azione accessibile ad esso.

$$\pi(a \mid s) = Pr\{A_t = a \mid S_t = s\} \tag{2.4}$$

La politica dell'agente viene quindi modificata progressivamente in base all'esperienza accumulata durante l'interazione con l'ambiente.
Una funzione valore (*value function*) di uno stato s secondo la politica π è definita come il valore di aspettazione delle ricompense cumulative generate a partire dallo stato s seguendo la politica π, e cioè formalmente:

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right] \tag{2.5}$$

Analogamente, una funzione valore di azione (*action-value function*) (o funzione Q) è definita come il valore di aspettazione ottenuto seguendo una *policy* π dopo aver eseguito una specifica azione a in uno stato s, e cioè

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a \right] \tag{2.6}$$

Le funzioni valore nel *Reinforcement Learning* obbediscono a relazioni ricorsive, note come equazioni di Bellman. Questa proprietà è essenziale, poiché le equazioni di Bellman costituiscono il fondamento teorico su cui si basano tutti gli algoritmi di RL. Le equazioni ricorsive per *value function* e *action-value function* sono

$$V_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma V_\pi(s') \right] \tag{2.7}$$

$$Q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \sum_{a' \in \mathcal{A}(s')} \pi(a' \mid s') Q_\pi(s', a') \right] \tag{2.8}$$

in cui $s' \in \mathcal{S}$ e $a' \in \mathcal{A}(s')$ rappresentano rispettivamente stato e azione dello step temporale successivo.

L'obiettivo di un algoritmo di RL è dunque determinare la politica ottimale π_*, ossia la regola che — se adottata in modo deterministico — garantisce la massima ricompensa cumulativa attesa. Le corrispondenti funzioni valore ottimale sono definite come:

$$V_*(s) = \max_\pi V_\pi(s) \tag{2.9}$$

$$Q_*(s, a) = \max_\pi Q_\pi(s, a) \tag{2.10}$$

Analogamente alle equazioni di Bellman generiche, le funzioni valore ottimale soddisfano equazioni di Bellman ottimali in cui si massimizza sulle azioni disponibili:

$$V_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma V_*(s') \right], \tag{2.11}$$

$$Q_*(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a' \in A(s')} Q_*(s', a') \right]. \tag{2.12}$$

Una volta che $V_*(s)$ o $Q_*(s,a)$ sono note o al più ben approssimate tramite un algoritmo di RL, la *policy* ottimale può essere ricostruita scegliendo in ogni stato l'azione che massimizza Q_*, e cioè

$$\pi_*(s) = \arg\max_a Q_*(s,a) \tag{2.13}$$

In sintesi, l'obiettivo del *Reinforcement Learning* può essere interpretato come ricerca della miglior stima per le funzioni valore ottimali tramite l'esperienza. Una volta che queste stime sono sufficientemente accurate, la politica ottimale può essere ricavata direttamente scegliendo le azioni che massimizzano la funzione Q ottimale. Un approccio di questo tipo caratterizza una famiglia di algoritmi denominati *value-based*[3], che approssimano iterativamente Q_* tramite i dati raccolti dall'agente. A questa categoria appartengono alcuni algoritmi noti come *Q-Learning* e la sua estensione *Deep Q-Learning* (DQN), impiegata anche in questo progetto.

2.2 Algoritmi *value-based* per il Reinforcement Learning

In questa sezione vengono presentati alcuni tra i più noti algoritmi di RL *value-based*. Inizialmente viene presentato il caso semplice del *Q-learning*, un algoritmo di RL classico la cui comprensione risulta propedeutica alla successiva trattazione della sua estensione in *Deep learning*, rappresentata dal *Deep Q-Network*. Infine viene presentata una variante di quest'ultimo — *Double Deep Q-Network* — che corrisponde all'algoritmo implementato in questo progetto. Gli estratti di pseudocodice relativi agli algoritmi descritti sono riportati in Appendice A.

2.2.1 Q-learning

Il *Q-learning* è uno dei classici algoritmi *value-based* per il *Reinforcement Learning* [25]. Il suo obiettivo è stimare in maniera empirica la funzione Q_*, ossia la ricompensa cumulativa attesa derivante dall'eseguire una specifica azione in uno stato e successivamente seguire la politica ottimale.

Il *Q-learning* è noto per essere un algoritmo tabellare, ossia basato sull'utilizzo di una matrice $\mathcal{S} \times \mathcal{A}(s)$ — anche nota come *Q-table* — in cui ogni elemento $Q(s,a)$ rappresenta il valore stimato per una determinata azione a eseguita nello stato s. La *Q-table* dunque viene aggiornata iterativamente secondo la regola

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{2.14}$$

dove:

- α è chiamato tasso di apprendimento (*learning rate*) e controlla quanto l'aggiornamento corrente della stima influisce sul valore precedente;

[3] Gli algoritmi di *Reinforcement Learning* possono essere distinti in due principali categorie: (i) *value-based*, che stimano funzioni valore $V(s)$ o $Q(s,a)$ per derivare da esse la politica ottimale; (ii) *policy-based*, che apprendono direttamente la politica $\pi(a \mid s)$ senza stimare funzioni valore, ottimizzando i parametri della politica stessa al fine di massimizzare la ricompensa cumulativa attesa.

- γ è il solito fattore di sconto che bilancia il peso delle ricompense future rispetto a quelle immediate;
- $\max_a Q(S_{t+1}, a)$ rappresenta il valore stimato della miglior azione possibile nello stato successivo.

Si noti che questa regola rappresenta una realizzazione empirica delle equazioni di Bellman ottimali per Q_* precedentemente introdotte. In *Q-learning* infatti la dinamica dell'ambiente, formalmente rappresentata dalla distribuzione delle probabilità $p(s', a \mid s, a)$, non è nota. L'algoritmo dunque considera una singola interazione come un campione rappresentativo della transizione, e pertanto utilizza ciascuna esperienza come dato empirico per aggiornare la stima di Q.
In questo contesto, l'aggiornamento della tabella Q è un esempio di *Temporal Difference* (TD) *learning*: la stima $Q(s, a)$ viene aggiornata nella direzione dell'errore TD, cioè la differenza tra il target empirico

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

coerente con le equazioni di Bellman e la stima corrente $Q(S_t, A_t)$. Questo meccanismo consente all'algoritmo di ottimizzare iterativamente la funzione Q e, in condizioni adeguate — ad esempio con una sufficiente esplorazione dello spazio degli stati e delle azioni e con un opportuno tasso di apprendimento — di convergere alla funzione ottimale Q_*.

Inoltre, l'algoritmo di *Q-learning* è un algoritmo *off-policy*, il che significa che può aggiornare i valori $Q(s, a)$ nella tabella seguendo anche una politica differente da quella ottimale per la scelta delle azioni, ad esempio esplorando in maniera casuale, per raccogliere esperienza. Generalmente questo tipo algoritmi si avvale dell'utilizzo di un iperparametro $\epsilon \in [0, 1]$ per definire il bilanciamento tra esplorazione dello spazio delle azioni e sfruttamento dell'esperienza accumulata. Seguendo infatti una politica comportamentale *ϵ-greedy* l'agente ad ogni passo temporale seleziona l'azione con il valore $Q(s, a)$ più alto (sfruttamento) con probabilità $1 - \epsilon$, mentre con probabilità ϵ sceglie un'azione casuale (esplorazione). In questo modo l'agente esplora nuove strategie potenzialmente migliori, evitando di rimanere intrappolato in soluzioni subottimali, e garantendo al tempo stesso la convergenza verso la politica ottimale nel lungo periodo. Per completezza espositiva, lo pseudo-codice dell'algoritmo è riportato in Appendice A, Algoritmo 1.

Nonostante la sua semplicità ed efficacia in ambienti di piccole dimensioni, il *Q-learning* presenta stringenti limiti pratici che richiedono la necessità di modelli più complessi per essere superati. Con l'aumentare della dimensione dello spazio degli stati $\mathcal{S}$ e delle azioni $\mathcal{A}$, l'assenza di capacità di generalizzazione implica la necessità di esplorare e memorizzare la quasi totalità delle combinazioni possibili di stato e azione. Questo rende rapidamente impossibile rappresentare la funzione Q all'interno di una tabella finita, compromettendo l'efficacia e la scalabilità dell'algoritmo. Per superare queste limitazioni, algoritmi più sofisticati come il *Deep Q-Network* [27] sostituiscono la rappresentazione tabellare discreta con una rete neurale, capace di generalizzare l'esperienza dell'agente anche a stati mai visti direttamente.

2.2.2 Deep Q-Network

Il *Deep Q-Network* (DQN) è un'estensione del *Q-learning* che permette di affrontare ambienti caratterizzati da spazi stato-azione di grandi dimensioni o continui, dove la rappresentazione tabellare è impraticabile [27].

L'idea è quella di utilizzare al posto della tabella discreta un approssimatore di funzione continuo, tipicamente una rete neurale, in grado di stimare il valore $Q(s, a, \theta)$ per ciascuna azione a eseguita nello stato s in funzione di un insieme di parametri θ, ovvero i pesi della rete. Intuitivamente quindi si cerca la combinazione di parametri ottimali $\tilde{\theta}$ tale che la funzione approssimata converga al valore ottimale $Q(s, a, \tilde{\theta}) \rightarrow Q_*$.
In particolare, il ciclo di interazione agente-ambiente in un algoritmo DQN è sviluppato in questo modo. La rete riceve in input lo stato corrente s e in funzione di esso calcola i valori di tutte le possibili azioni $Q(s, a, \theta)\ \forall a \in \mathcal{A}(s)$. Noti questi valori viene selezionata una particolare azione, solitamente tramite una politica ϵ-*greedy*, e viene eseguita sull'ambiente. Come in ogni MDP, l'ambiente risponde restituendo una ricompensa r e lo stato successivo s', consentendo all'agente di ripetere il ciclo.
Ad ogni interazione, la transizione osservata viene memorizzata in una struttura chiamata *Buffer Replay*, che conserva un insieme di esperienze nella forma di tuple $\{s, a, r, s'\}$. Questa memoria consente di riutilizzare in seguito le esperienze accumulate per ottimizzare i parametri della rete, riducendo la dipendenza temporale tra le osservazioni e rendendo l'apprendimento più stabile ed efficiente.
Nel momento in cui l'agente ha acquisito esperienza sufficiente, e quindi un adeguato numero di elementi nel *Replay Buffer*, ha inizio il processo di ottimizzazione. Il DQN utilizza un approccio basato sull'errore TD, analogo a quello del *Q-learning*. In questo caso però la stima del target empirico coinvolge una rete secondaria — rete target (*target network*) — che viene aggiornata periodicamente con i valori stimati durante l'apprendimento.
Vengono campionati casualmente un insieme di elementi dal *Replay Buffer*, e per ciascuna transizione viene calcolato un valore target

$$y_{\text{target}} = r + \gamma \max_{a'} Q(s', a', \theta^-)$$

dove θ^- indica i pesi della *target network*. Successivamente, la rete principale aggiorna i propri parametri minimizzando la *loss function*, definita come la differenza quadratica media tra il valore target e la stima corrente.

$$\begin{aligned} L(\theta) &= \mathbb{E}\left[(y_{\text{target}} - Q(s, a, \theta))^2\right] = \\ &= \mathbb{E}\left[(r + \gamma \max_{a'} Q(s', a', \theta^-) - Q(s, a, \theta))^2\right] \end{aligned} \tag{2.15}$$

L'ottimizzazione di questa funzione consente alla rete di ridurre progressivamente la distanza tra le stime e i valori ottimali Q_*, migliorando la qualità della policy appresa. Il campionamento casuale dal *Replay Buffer* è essenziale per evitare una correlazione tra l'aggiornamento dei pesi e le esperienze di interazione più recenti, favorendo così un apprendimento stabile e volto alla convergenza.
Lo pseudo-codice dell'algoritmo è riportato in Appendice A, Algoritmo 1.

In sintesi, grazie alla capacità della rete neurale di generalizzare l'esperienza acquisita, il DQN rappresenta una valida alternativa per superare i limiti della rappresentazione tabellare, permettendo lo studio di problemi che coinvolgono ambienti con

spazi di stato e azione ad alta dimensionalità. Tuttavia, il *Deep Q-Network* presenta alcune criticità, tra cui la tendenza ad sovrastimare i *Q-value*, che può compromettere la stabilità e la qualità della politica appresa. Una soluzione a questo problema è rappresentata da una variante del DQN — il *Double Deep Q-Network* — ossia l'algoritmo implementato in questo progetto.

2.2.3 Double Deep Q-Network

Il *Double Deep Q-Network* (DDQN) è una variante del DQN progettata per ridurre il bias di sovrastima dei valori Q, che rappresenta un problema comune di questo algoritmo [28].

L'idea fondamentalmente è quella di separare la selezione dell'azione che massimizza Q dalla sua valutazione. In DDQN, la rete principale — quella in addestramento — viene utilizzata per scegliere l'azione con il massimo valore previsto nello stato successivo, mentre la rete target fornisce la valutazione di quell'azione. Formalmente si passa da un target per DQN come

$$y_{\text{targetDQN}} = r + \gamma \max_{a'} Q(s', a', \theta^-)$$

ad un target per DDQN simile a

$$y_{\text{targetDDQN}} = r + \gamma Q(s', \arg\max_{a} Q(s', a, \theta), \theta^-)$$

dove θ^- rappresenta i pesi della rete target e θ rappresenta i pesi della rete principale. Questa separazione riduce il rischio di sovrastima dei valori, favorendo un apprendimento più stabile e migliorando la qualità della politica derivata.
Per il resto, il ciclo di interazione agente-ambiente e l'utilizzo del *Replay Buffer* sono identici a quelli del DQN. Per ogni interazione le esperienze vengono accumulate sotto forma di tuple $\{s, a, r, s'\}$ e vengono campionate casualmente durante il processo di ottimizzazione. Anche in questo caso, la rete aggiorna i pesi minimizzando la *loss function* definita come lo scarto quadratico medio tra il nuovo target e la stima corrente, come nel DQN.

$$L(\theta) = \mathbb{E}\left[(r + \gamma Q(s', \arg\max_{a} Q(s', a, \theta), \theta^-) - Q(s, a, \theta))^2\right] \tag{2.16}$$

La *target network* viene dunque aggiornata con i pesi della rete principale secondo una frequenza prestabilita di passi temporali.

Grazie a questa modifica il DDQN rappresenta una soluzione più robusta rispetto al DQN. Mantenendo la capacità di generalizzazione della rete neurale, si conferma la scelta di algoritmo ottimale per applicazioni in ambienti con spazi di stato e azione di grandi dimensioni, come quelli considerati in questo progetto.

2.3 Reti Neurali artificiali

In questa sezione vengono introdotti i concetti teorici fondamentali necessari per comprendere il funzionamento delle reti neurali artificiali.
In generale, le reti neurali artificiali (*Artificial Neural Network*, ANN) sono modelli computazionali che si ispirano, in modo semplificato, al funzionamento del cervello

umano. Il loro obiettivo è quello di imparare le relazioni tra un insieme di dati di ingresso e i corrispondenti valori di uscita attraverso un processo di apprendimento basato sull'esperienza. In termini matematici, una rete neurale può essere vista come un insieme di trasformazioni parametriche non lineari che convertono un vettore di input in un vettore di output. Per questo motivo possono essere considerate come degli approssimatori di funzione, ovvero sistemi capaci di rappresentare — entro certi limiti — qualsiasi funzione continua. Tale caratteristica le rende dunque fondamentali in un contesto di RL *value-based* in cui si vuole stimare una funzione complessa come $Q(s, a)$.

2.3.1 Architettura e funzioni di attivazione

Una rete neurale è costituita da una serie di strati sequenziali di neuroni artificiali che operano in parallelo per trasformare i dati di ingresso e produrre un output.

Il neurone è l'unità di calcolo fondamentale di una rete neurale, ispirata al funzionamento dei neuroni biologici [29, 25]. Esso riceve uno o più valori in ingresso, ciascuno associato ad un peso che ne modula l'importanza. Alla somma pesata di questi valori si aggiunge un parametro bias, che serve per adattare la soglia di attivazione del neurone[4]. Infine, il valore di output è calcolato tramite l'impiego di particolari funzioni non lineari, chiamate funzioni di attivazione, necessarie per rappresentare relazioni complesse tra valori di ingresso e uscita. Formalmente dunque l'output è rappresentato da

$$y = f\left(\sum_{i=1}^{n} \omega_i x_i + b\right) \tag{2.17}$$

dove $x_1, x_2, \cdots, x_n$ rappresentano i valori di input per il neurone, $\omega_1, \omega_2, \cdots, \omega_n$ i rispettivi pesi, b il bias e $f(\cdot)$ la funzione di attivazione.
Come anticipato, una rete è costituita da una sovrapposizione di strati (*layer*). Un *layer* è un insieme di neuroni che lavorano in parallelo sugli stessi dati di input, ciascuno producendo il proprio output — sulla base dei propri parametri — che può essere passato allo strato successivo o interpretato come valore finale.

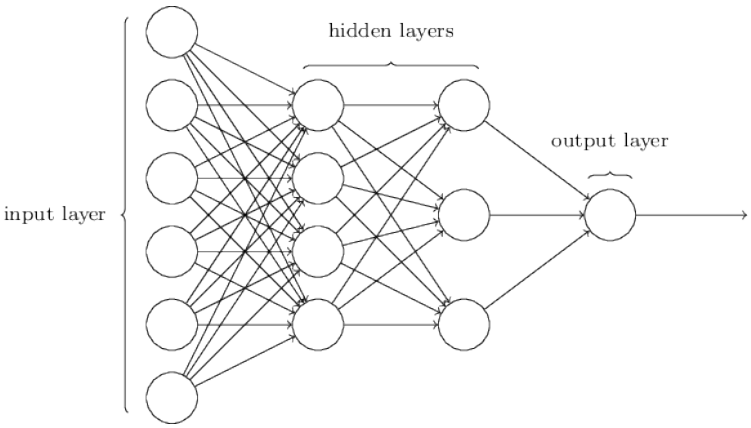


Figura 2.3: Esempio della struttura a strati di una rete neurale artificiale [30].

[4]In neuroscienza, una catena di neuroni interconnessi interagisce tramite invio/ricezione di neurotrasmettitori. Questi modificano il potenziale elettrico che, se supera una certa soglia, "eccita" il neurone permettendogli di inviare neurotrasmettitori ad altri neuroni.

In generale, la struttura di una rete neurale prevede almeno uno strato di input che riceve i dati grezzi dall'ambiente e uno strato di output che genera i valori finali, come ad esempio i valori Q nel caso del DQN. In mezzo possono essere presenti diversi strati nascosti (*hidden layer*) che costituiscono il cuore del funzionamento di una rete neurale: essi infatti elaborano le informazioni, permettendo alla rete di sfruttare rappresentazioni intermedie sempre più astratte[5] che consentono l'adattamento ad arbitrarie funzioni complesse. Intuitivamente, più una rete è profonda (ossia con maggior numero di strati nascosti), più è in grado di rappresentare funzioni articolate e complesse. Una rappresentazione schematica della stratificazione di una rete è presentata in fig. 2.3.

Esempi di funzioni di attivazione

Le funzioni di attivazione svolgono un ruolo cruciale: introducono nel modello la non linearità necessaria affinché la rete possa apprendere relazioni non lineari tra input e output. Senza di esse, una rete neurale, per quanto profonda, si comporterebbe come una semplice trasformazione lineare, incapace di risolvere problemi complessi. Alcuni esempi tra le principali funzioni di attivazione utilizzate sono:

- la sigmoide:
$$f(x) = \frac{1}{1 + e^{-x}} \tag{2.18}$$
- la tangente iperbolica:
$$f(x) = \tanh(x) \tag{2.19}$$
- la funzione ReLU (*Rectified Linear Unit*):
$$f(x) = \max(0, x) \tag{2.20}$$

Tra tutte, la funzione ReLU rappresenta oggi lo standard nelle reti neurali profonde, grazie alla sua semplicità ed efficienza. Pertanto, è la funzione di attivazione impiegata nelle reti implementate in questo progetto.

2.3.2 Addestramento di una rete

L'addestramento di una rete consiste nell'adattare progressivamente i pesi dei neuroni affichè essa impari a mappare i valori di ingresso nei valori di uscita desiderati. L'idea è quella di confrontare le previsioni della rete con un valore di riferimento (*target*) rappresentativo del comportamento ottimale, rispetto ad un campione di esempi di input. Una volta quantificata la discrepanza tra i due vettori di output mediante una funzione di errore (o *loss function*), l'obiettivo dell'addestramento è minimizzare tale valore, modificando i parametri della rete nella direzione opposta al gradiente[6] della loss. In particolare, questo processo si articola principalmente in quattro fasi distinte: *forward pass*, calcolo della *loss*, *backpropagation* e ottimizzazione [31].

[5] Il comportamento "interno" di una rete è spesso non interpretabile: gli strati di input e output rappresentano rispettivamente i livelli di codifica e decodifica dell'informazione.

[6] Matematicamente, se si immagina la funzione di perdita come una superficie nello spazio dei parametri, il gradiente rappresenta la direzione di crescita più rapida. Poiché l'obiettivo è ridurre la loss, è necessario muoversi nella direzione opposta a questo vettore, processo noto come *discesa del gradiente*.

Forward pass

Durante questa fase la rete calcola la propria stima dell'output $\hat{y}$ per ciascuno dei vettori x di ingresso. I dati grezzi vengono elaborati parallelamente da ciascun neurone dell'*input layer*, secondo il processo descritto in precedenza. Viene calcolata la somma pesata per ciascun dato, a cui viene aggiunto un bias, dopodiché questo valore passa per una funzione di attivazione e viene inviato ai neuroni dello strato successivo. In termini matematici [30]:

$$
\begin{aligned}
z^{(1)} &= W^{(1)}x + b^{(1)} \\
a^{(1)} &= f^{(1)}(z^{(1)}) \\
&\cdots \\
z^{(l)} &= W^{(l)}a^{(l-1)} + b^{(l)} \qquad (2.21)\\
a^{(l)} &= f^{(l)}(z^{(l)}) \qquad (2.22)\\
&\cdots \\
z^{(L)} &= W^{(L)}a^{(L-1)} + b^{(L)} \\
a^{(L)} &= f^{(L)}(z^{(L)}) = \hat{y}
\end{aligned}
$$

dove $W^{(l)}$, $b^{(l)}$ rappresentano rispettivamente la matrice con i pesi e il vettore con le soglie di uno strato l, con $l \in [1, L]$, mentre $a^{(l)}$ rappresenta l'output di ciascun *layer*, dopo il passaggio dei valori lineari $z^{(l)}$ per le rispettive funzioni di attivazione $f^{(l)}(\cdot)$. Infine $a^{(L)} = \hat{y}$ rappresenta il valore di uscita dell'*output layer* L, ossia la stima corrente della rete per il vettore di ingresso.

Loss function

A questo punto la rete confronta il proprio output $\hat{y}$ con un valore di riferimento y. Per fare ciò viene utilizzata una funzione di perdita (*loss function*) $\mathcal{L}(\hat{y}, y)$, il cui valore fornisce un indicatore di quanto la rete stia attualmente sbagliando. Si noti che, poiché la stima $\hat{y}$ dipende da tutti i parametri della rete, anche la funzione di loss è implicitamente funzione di essi:

$$\mathcal{L}(\hat{y}, y) = \mathcal{L}(\theta) \tag{2.23}$$

dove θ denota l'insieme di tutti i pesi e bias. Gli esempi più comuni di funzioni utilizzate per questo tipo di problemi sono la *Mean Squared Error* (MSE) e la *Mean Absolute Error* (MAE). Oltre a queste esistono diverse varianti, come ad esempio *Huber Loss*, anch'esse con il fine di quantificare la discrepanza tra stima e target, e utili per migliorare la qualità dell'addestramento.

Backpropagation

Noto un valore per la funzione di errore, viene implementato l'algoritmo di *backpropagation* per calcolare in maniera efficiente il vettore gradiente rispetto ai parametri della rete , ovvero (mantenendo lo stesso formalismo matematico introdotto in precedenza):

$$\nabla_\theta \mathcal{L} = \left(\cdots, \frac{\partial \mathcal{L}}{\partial W^{(l)}}, \frac{\partial \mathcal{L}}{\partial b^{(l)}}, \cdots, \right)^T \tag{2.24}$$

dove $W^{(l)}$ rappresenta la matrice con i pesi $\omega_{jk}^{(l)}$ per ciascun input k del neurone j nello strato l, e analogamente $b^{(l)}$ è il vettore dei bias $b_j^{(l)}$.
Questo algoritmo si basa sul teorema della catena (*chain rule*), secondo cui, data una funzione composta $f(g(x))$, la derivata rispetto a x è

$$\frac{\mathrm{d}f(g(x))}{\mathrm{d}x} = \frac{\partial f}{\partial g}\frac{\partial g}{\partial x} .$$

Applicando questo al comportamento della rete, in cui gli input di ciascuno strato corrispondono agli output dello strato precedente, si ottiene

$$\frac{\partial \mathcal{L}}{\partial a^{(l-1)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}}\frac{\partial a^{(l)}}{\partial a^{(l-1)}} \tag{2.25}$$

L'idea alla base della *backpropagation* è quindi calcolare l'errore a partire dall'ultimo strato e propagare tale informazione a ritroso fino al primo strato, da cui il nome *backpropagation*.
Introduciamo [30] l'errore per lo strato l, definito come

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} \tag{2.26}$$

in cui $z^{(l)}$ rappresenta il vettore di valori ottenuti linearmente in ingresso alla funzione di attivazione. Per lo strato di output L questo corrisponde a

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \odot f'^{(L)}(z^{(L)}) \tag{2.27}$$

dove il simbolo $\odot$ rappresenta il prodotto elemento per elemento, noto come prodotto di Hadamard. Si noti che ciascuno degli elementi di questo prodotto è noto: l'espressione della funzione di loss permette di ricavare il più delle volte in maniera elementare il valore del primo termine, mentre il secondo dipenderà dalla specifica funzione di attivazione, ma anche in questo caso il fatto che sia nota l'espressione analitica rende questo calcolo banale dal punto di vista computazionale.
A questo punto con pochi conti di algebra si può dimostrare [30] che l'equazione ricorsiva che lega l'errore di ciascuno strato al successivo è data da

$$\delta^{(l)} = \left(\left(W^{(l+1)}\right)^T \delta^{(l+1)}\right) \odot f'^{(l)}(z^{(l)}) \tag{2.28}$$

Infine, noti gli errori $\delta^{(l)}$ per ciascuno strato, le derivate della loss rispetto a ciascun parametro si ottengono tramite le seguenti equazioni.

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)}\left(a^{(l-1)}\right)^T \tag{2.29}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)} \tag{2.30}$$

Si noti che anche in questo caso, poiché il processo di *backpropagation* avviene in seguito a quello di *forward pass*, l'algoritmo dispone di tutte le informazioni necessarie per calcolare i gradienti in modo computazionalmente efficiente.

Ottimizzazione

L'ultima fase del processo di addestramento di una rete neurale consiste nell'ottimizzazione, ovvero nell'aggiornamento dei pesi e dei bias per ridurre l'errore della rete.

Esistono molteplici algoritmi di ottimizzazione, tra cui sicuramente il più semplice è la *gradient descent* (discesa del gradiente). Una volta calcolato il vettore gradiente della loss rispetto ai parametri, l'algoritmo aggiorna i pesi nella direzione opposta al gradiente, con lo scopo di minimizzare la discrepanza tra le stime della rete e i valori desiderati:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathcal{L}(\theta) \tag{2.31}$$

dove θ rappresenta l'insieme dei parametri (pesi e bias), $\nabla_\theta \mathcal{L}(\theta)$ è il gradiente della *loss function* e η è il tasso di apprendimento (*learning rate*), che determina la dimensione del passo di aggiornamento. Un *learning rate* troppo grande può rendere l'ottimizzazione instabile, mentre un valore troppo piccolo rischia di rallentare l'apprendimento. Algoritmi più avanzati, come *Adam*, combinano la discesa del gradiente con adattamenti del tasso di apprendimento per migliorare velocità e stabilità dell'addestramento.

2.3.3 Tipologie di rete

Fino ad ora si è parlato di reti neurali come strati sovrapposti di neuroni interconnessi, senza fare riferimento a particolari strutture. A seconda del dato di input grezzo che la rete deve elaborare e in funzione della struttura di output che si vuole generale è possibile distinguere diverse tipologie di reti neurali. In particolare, prima di procedere con gli ultimi elementi di *Reinforcement Learning*, è utile introdurre brevemente le due tipologie di architetture per reti neurali utilizzate in questo progetto.

Reti feed-forward (FFN)

Le FFN, note anche come reti *fully connected*, rappresentano la tipologia di rete neurale più semplice, analoga a quella considerata fino ad ora. In queste reti ciascun neurone è collegato a tutti i neuroni dello strato precedente e di quello successivo e l'informazione procede in un'unica direzione (dall'input verso l'output), senza nessuna retroconnessione. L'unica variabile strutturale necessaria per progettare una rete di questo tipo è il numero di neuroni e di strati (*hidden layers*) impiegati, che determinano la capacità del modello di rappresentare funzioni più o meno complesse. Tali reti sono particolarmente adatte all'elaborazione di dati vettoriali o scalari, dove la struttura spaziale non è importante.

Reti convoluzionali (CNN)

Le reti convoluzionali (*Convolutional Neural Network*, CNN) [31] rappresentano una estensione delle FFN, particolarmente adatta ad elaborare dati caratterizzati da strutture locali, come immagini, o — come nel caso di questo progetto — matrici di sindrome d'errore.
A differenza delle *fully connected* dove ogni neurone è connesso a tutti quelli del *layer* precedente, nelle reti CNN ciascun neurone è connesso unicamente ad un'area locale dell'input. Questa caratteristica riduce il numero di parametri e consente alla rete di estrarre pattern locali significativi (ad esempio bordi, angoli o strutture ricorrenti).

Formalmente, una rete CNN elabora strutture tridimensionali di dati, in cui le dimensioni rappresentano rispettivamente:

- il numero di canali (*channels*) ovvero, nel contesto di questo lavoro, il numero di matrici di sindrome sovrapposte;
- l'altezza e la larghezza spaziale dei dati, e quindi la dimensione della matrice.

In particolare, una CNN è in grado di elaborare strutture simili tramite l'applicazione di filtri, detti *kernel*, ciascuno definito da un insieme di pesi che rappresentano la regione locale analizzata per produrre un singolo valore in uscita. Il filtro scorre sull'input con un determinato passo, detto *stride*, generando in uscita una nuova mappa bidimensionale chiamata *feature map*. Ogni elemento di tale mappa rappresenta un neurone la cui attivazione misura quanto il pattern descritto dal filtro è presente in quella particolare regione dell'input.
L'output di un *layer* convoluzionale è quindi un insieme di *feature map* — in numero uguale ai filtri applicati nello strato — ciascuna di dimensione ridotta rispetto alla matrice iniziale. Questo tipo di dato può essere passato ad un nuovo strato di CNN per studiare pattern sempre più complessi, oppure può essere appiattito e passato a strati FFN per la fase decisionale finale della rete. Un esempio di struttura simile è presentato in fig. 2.4

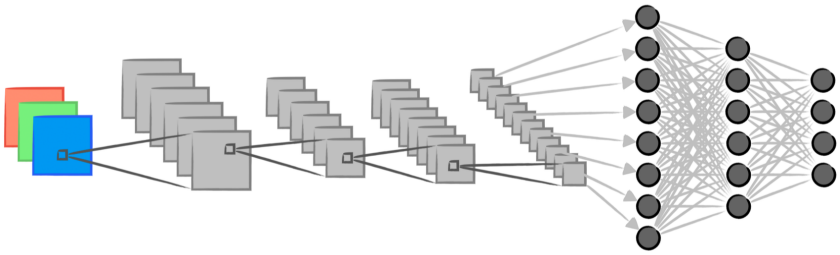


Figura 2.4: Esempio di una rete neurale composta da strati CNN e FFN [32].

2.4 Probabilistic Policy Reuse

Infine, per completare il quadro dei fondamenti teorici alla base di questo progetto, viene introdotta una variante sul paradigma di *Reinforcement Learning* classico volta ad accelerare l'allenamento di una *policy* basata su DDQN: il *Probabilistic Policy Reuse* (PPR) [10]. Questo approccio rappresenta un'implementazione del concetto di *continual learning*, ovvero un particolare aspetto di *Machine Learning* che specifica la capacità di un agente di apprendere da informazioni accumulate nel tempo. L'idea è infatti quella di riutilizzare comportamenti precedentemente appresi — sotto forma di politiche — per agevolare la risoluzione di un nuovo problema in un ambiente sconosciuto, ma che condivide alcune caratteristiche con quelli già esplorati.

2.4.1 Esplorazione tramite π-reuse

Formalmente, il DDQN appartiene alla famiglia degli algoritmi *off policy*, e cioè esso è in grado di apprendere anche eseguendo azioni che non seguono esattamente la politica in allenamento. Questa caratteristica consente di introdurre strategie di esplorazione,

come ϵ-*greedy*, che introducono un elemento di casualità per favorire la scoperta di nuove soluzioni. Allo stesso modo, è possibile estendere questo principio e introdurre una diversa strategia di esplorazione, nota come π-*reuse Exploration Strategy* [33], alla base dell'approccio di PPR.

Questo metodo ha come scopo quello di influenzare un nuovo processo di apprendimento con una *policy* passata. A tal proposito, definiamo π_{past} la politica che si vuole riutilizzare e π_{new} la politica che si vuole istruire per risolvere il nuovo problema. L'obiettivo di π-*reuse* è quello di bilanciare tre componenti: l'esplorazione casuale dello spazio delle azioni, lo sfruttamento della politica passata e lo sfruttamento della politica corrente. A questo proposito introduciamo un iperparametro $\psi \in [0, 1]$ con il compito di regolare la probabilità di riutilizzo di politiche passate in ciascun passo:

$$a = \begin{cases} \pi_{\text{past}}(s) & \text{con prob. } \psi \\ \epsilon\text{-}greedy\,(\pi_{\text{new}}(s)) & \text{con prob. } 1-\psi \end{cases} \tag{2.32}$$

Secondo questa strategia, ad ogni step l'agente con probabilità ψ esegue l'azione suggerita dalla politica passata, mentre con probabilità $1-\psi$ utilizza la classica strategia ϵ-*greedy* basata sulla nuova politica.

L'iperparametro ψ controlla quindi il bilanciamento tra riuso e apprendimento autonomo: valori alti favoriscono un uso intenso dei della politica passata, accelerando l'apprendimento iniziale ma rischiando di compromettere l'adattamento alle differenze che i due ambienti presentano. Un valore basso invece favorisce l'esplorazione autonoma e consente un miglior adattamento alle caratteristiche del nuovo ambiente.

2.4.2 Addestramento con Probabilistic Policy Reuse

L'addestramento di un agente tramite DDQN con l'aggiunta di *Probabilistic Policy Reuse* utilizza la strategia appena descritta di π-*reuse* estendendola ad una libreria di politiche pre-addestrate L:

$$L = \{\pi_1, \pi_2, \cdots, \pi_n\} \tag{2.33}$$

Consideriamo un problema episodico, come quello affrontato in questo progetto. In un contesto simile, il PPR consente di selezionare, all'inizio di ogni episodio, una *policy* π_i dalla libreria L e di accumulare esperienze implementando una strategia π-*reuse* con quella politica. Pertanto, per ogni interazione dell'episodio corrente, l'azione eseguita verrà scelta con probabilità ψ su suggerimento della politica pre-addestrata π_i, con probabilità $1-\psi$ mediante la strategia ϵ-*greedy* sulla politica π_{new} in fase di addestramento.

L'aspetto distintivo della PPR riguarda il meccanismo di selezione della politica da utilizzare in ciascun episodio. Questa selezione infatti può essere uniforme, assegnando la stessa probabilità a tutte le *policy* presenti nella libreria, oppure può seguire una distribuzione che privilegi le politiche con performance migliori, promuovendo il riutilizzo di strategie più efficaci.
L'approccio più comune prevede l'utilizzo di una distribuzione *Softmax*, regolata tramite un parametro di temperatura τ che varia durante la fase di addestramento:

$$\mathrm{P}(\pi_j) = \frac{e^{\tau W_j}}{\sum_{p=0}^{n} e^{\tau W_p}} \tag{2.34}$$

Nell'equazione (2.34) i termini W_j rappresentano la ricompensa media ottenuta nel tempo dalla politica π_j. Dato un episodio in cui è stata selezionata la *policy* j, poiché ogni azione — trattandosi di un MDP — viene valutata tramite una ricompensa, è possibile considerare la ricompensa cumulativa R dell'episodio come una misura della sua efficacia. Questa ricompensa viene mediata sul numero di utilizzi della politica, costruendo così la variabile discriminante per la distribuzione di probabilità.
Il parametro di temperatura τ viene inizializzato a 0, e progressivamente subisce un incremento di un fattore $\delta\tau$ per ogni episodio. In questo modo, la distribuzione risulta inizialmente uniforme, coerente con la mancanza di un'esperienza sufficiente per poter valutare l'utilizzo delle possibili soluzioni, e diventa progressivamente selettiva, fino a privilegiare le strategie migliori per il problema considerato.
Un estratto di pseudo-codice sintetico relativo all'apprendimento di DDQN con PPR è riportato in Appendice A, Algoritmo 3.

Capitolo 3

Progetto e risultati sperimentali

In questo capitolo viene presentato il lavoro svolto in questo progetto. L'obiettivo principale è stato lo sviluppo di un decoder basato sul paradigma di RL per la decodifica di sindromi generate da un *Surface Code*. In particolare, si è voluto mostrare come la capacità di generalizzazione di un decoder fondato su tecniche di *Deep Reinforcement Learning* — nello specifico un modello *Double Deep Q-Network* (DDQN) — consenta di gestire in modo intrinseco ed efficace il caso di sindromi *faulty*, ossia affette anche da errori di misura. Parallelamente, è stato studiato come l'utilizzo di strategie di *continual learning*, in particolare del metodo di *Probabilistic Policy Reuse* (PPR), possa migliorare la capacità di adattamento del decoder nel tempo, ponendo le basi per un sistema in grado di aggiornarsi *online* al variare del modello di rumore. Il confronto tra politiche addestrate nello stesso ambiente, con e senza l'utilizzo di PPR, ha confermato sperimentalmente come tale approccio consenta un più efficiente addestramento in termini temporali. Il lavoro prende ispirazione dalla proposta di Sweke et al. [9] per lo sviluppo del decoder basato su DDQN, e da quella di Matekole et al. [10] per l'integrazione di strategie di *continual learning* mediante PPR.

Il capitolo è articolato in tre sezioni: la prima descrive nel dettaglio l'ambiente di simulazione e l'architettura dell'agente sviluppato; la seconda illustra i metodi e le strategie di addestramento adottati, insieme alla gestione dei parametri; la terza presenta e analizza i risultati sperimentali, con particolare attenzione al contributo del metodo PPR e alle prestazioni del decoder nei diversi ambienti.

3.1 Setup sperimentale

In questa sezione vengono descritti l'ambiente di simulazione e l'architettura software utilizzati per lo sviluppo e l'addestramento del decoder. L'ambiente si basa sull'implementazione proposta da Sweke et al. [9], opportunamente adattata per poter essere utilizzata con le architetture del decoder e le funzioni di addestramento sviluppate in questo progetto. Tutto il codice è stato scritto in *Python*, facendo uso principalmente — oltre a librerie classiche come *Numpy* e *Matplotlib* — della libreria di *PyTorch* [34], impiegata per costruire modelli personalizzati di *Reinforcement Learning*, e della libreria *Gymnasium* [35] per quanto riguarda la gestione dell'ambiente di simulazione.

3.1.1 Ambiente di simulazione

L'ambiente di simulazione, tratto dal repository [36] relativo all'articolo di Sweke et al. [9] del 2020, fornisce una rappresentazione accurata del problema di decodifica di *Surface Code*. Originariamente sviluppato per un agente basato sulle librerie *TensorFlow/Keras RL*, l'ambiente seguiva le API e il funzionamento di *OpenAI Gym*. Poiché il progetto è stato implementato utilizzando *PyTorch* e la libreria *OpenAI Gym* — oggi deprecata — è stata sostituita con *Gymnasium*, le modifiche apportate all'ambiente originale del repository riguardano principalmente l'adattamento alla nuova API e la compatibilità con i modelli sviluppati in *PyTorch*.
L'ambiente è costituito da un modello personalizzato di *Gymnasium*, ossia una classe che definisce lo stato osservabile e i metodi *reset()* e *step()*, rispettivamente per inizializzare l'*environment* ed eseguire un'azione al suo interno.

Lo stato osservabile

Fissato il modello di errore da simulare, lo stato generico dell'ambiente consiste in una rappresentazione bidimensionale della sindrome associata a un *Surface Code*, affiancata da una memoria che traccia le correzioni (azioni) eseguite dal decoder in ciascun episodio. Considerando un *Surface code* di dimensione $d \times d$, le informazioni sono codificate in matrici $(2d + 1) \times (2d + 1)$, come mostrato in fig. 3.1 per il caso $d = 5$ utilizzato in questo progetto.

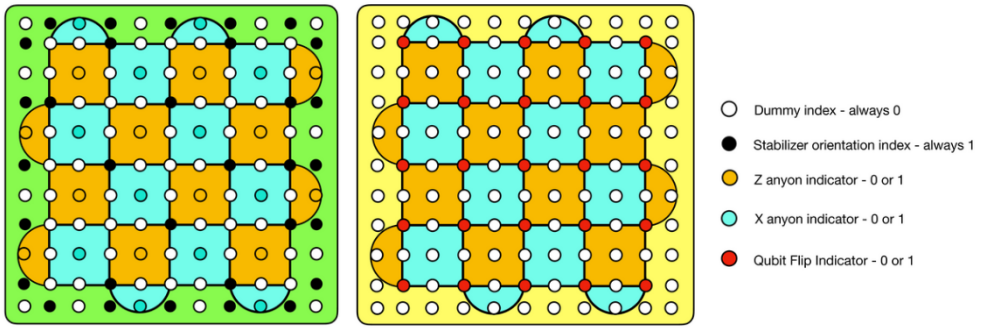


Figura 3.1: Matrici per la codifica di uno stato dell'*environment* [36].

In ciascuna griglia, in analogia a quanto discusso nella sezione 1.3, ogni vertice e ogni cella rappresentano rispettivamente i qubit fisici e gli stabilizzatori del codice. La griglia verde raffigura lo strato relativo alla sindrome, mentre quella gialla rappresenta la memoria delle azioni che, nel corso di un episodio, vengono applicate dall'agente ai singoli qubit. Le misure restituite da ciascuno stabilizzatore e la traccia degli operatori applicati sono codificate nelle matrici tramite valori booleani 0 o 1.

L'ambiente è progettato per simulare in maniera adeguata modelli di errore di tipo *bit-flip noise* e *depolarizing noise*[1] ai quali l'agente può rispondere applicando per ogni step esclusivamente un operatore X o Z su un singolo qubit fisico. Lo spazio delle azioni è quindi limitato al set di operazioni $\{X, Z\}$ per ciascun qubit: per rappresentare correttamente la cronologia e la natura delle correzioni effettuate, lo stato deve dunque includere strati di memoria distinti dedicati a ciascun tipo di operatore applicabile. Poiché gli operatori Y possono essere ottenuti come combinazione di X e Z, la loro

[1] Per semplicità rappresentato dall'applicazione casuale di operatori X, Y, e Z.

azione effettiva può essere implementata attraverso più passi, corrispondenti a successive interazioni agente-ambiente. Questo giustifica la restrizione sullo spazio delle azioni, permettendo di semplificare il problema di decodifica senza perdere coerenza con la dinamica temporale dell'ambiente. È importante sottolineare infatti che ogni step rappresenta una singola decisione dell'agente, mentre l'evoluzione temporale complessiva del sistema è organizzata in cicli discreti di tipo "errore–misura–correzione". In ciascun ciclo, l'ambiente simula l'introduzione di nuovi errori fisici, fornisce la sindrome corrispondente, e consente all'agente di accumulare progressivamente le azioni correttive che ritiene necessarie.

Inoltre, per simulare il problema in maniera più realistica, il codice genera *faulty syndromes* che, oltre agli errori fisici per ciascun qubit, includono anche gli errori di misura per ciascuno stabilizzatore. Di conseguenza, per permettere al decoder di imparare strategie adeguate anche in presenza di strumentazione non ideale, lo stato comprende una sovrapposizione temporale di strati di sindrome — il cosiddetto *syndrome volume* — di profondità arbitraria, fissata a 5 nel caso specifico di questo progetto.

Il metodo reset()

Ciascun episodio di interazione con questo ambiente ha inizio con la chiamata del metodo *reset()* che inizializza lo stato. In accordo con il modello di errore considerato e i parametri p_{phys} (probabilità di errore fisico) e p_{meas} (probabilità di errore di misura), il codice genera realisticamente strati temporalmente sequenziali di *faulty syndrome*, considerando l'eventuale accumulo di errori fisici. A questi strati si aggiungono i due livelli di memoria vuoti, costituendo così una struttura tridimensionale — 7 matrici 11×11 nel caso $d = 5$ — adatta ad essere elaborata dalle reti neurali convoluzionali dell'agente. Oltre alle sindromi affette da errori di misura che vengono fornite all'agente come osservazione parziale dell'ambiente, l'*environment* mantiene internamente uno stato nascosto (*hidden state*) che rappresenta la reale configurazione del qubit logico. Tale stato viene aggiornato a ogni interazione, riflettendo l'evoluzione effettiva del sistema.
Lo stato osservabile viene quindi passato all'agente che seleziona un'azione in accordo con la propria politica. Per agevolare l'addestramento, ad ogni step l'ambiente impone le seguenti restrizioni sulla scelta dell'azione:

1. le azioni devono essere coerenti con il modello di errore considerato (solo operatori X per il *bit-flip*, operatori X e Z per il *depolarizing*);

2. l'applicazione può avvenire solo su qubit adiacenti a stabilizzatori violati o già modificati nel corso dell'episodio.

Il metodo step()

Una volta che l'agente ha selezionato l'azione, essa viene eseguita sull'ambiente tramite il metodo *step()* articolato nelle seguenti operazioni:

- viene aggiornato lo strato di memoria relativo alla cronologia dell'operatore applicato, andando a marcare il qubit fisico coinvolto nella griglia;

- viene applicata l'azione allo *hidden-state*, modificando di conseguenza la reale configurazione del sistema;

- lo stato reale aggiornato viene quindi processato da un decoder "arbitro" (*referee decoder*) parte integrante dell'ambiente:
  - se il decoder **non riesce** a decodificare la sindrome reale, l'episodio termina e viene assegnata all'azione una ricompensa pari a 0;
  - se la sindrome reale risulta **ancora decodificabile**, l'episodio prosegue e viene assegnata una ricompensa pari a 1 nel caso in cui l'azione abbia ripristinato lo stato logico corretto, 0 altrimenti.
- infine, il metodo restituisce all'agente la ricompensa, lo stato corrente (aggiornato solo nella memoria delle azioni) e un segnale booleano che indica se lo stato è terminale.

Il *referee decoder*, fornito anch'esso nel repository, consiste in una rete neurale *feed forward* allenata in modo supervisionato per la classificazione dello stato. In linea di principio, qualsiasi algoritmo di decodifica di sindromi *perfette* — non soggette ad errori di misura — come MWPM può essere utilizzato come giudice per la valutazione dello stato reale. Il raggiungimento di una condizione terminale tramite un'azione comporta l'applicazione del metodo *reset()* e l'inizio di un nuovo episodio di decodifica.

La procedura di interazione appena descritta, contenuta nel metodo *step()*, si applica in modo analogo per ciascuna azione selezionata, fatta eccezione per le azioni di identità o le azioni ripetute. Queste ultime consentono all'agente di segnalare la propria convinzione di aver applicato tutte le correzioni necessarie per riportare il codice allo stato desiderato, il che conclude il ciclo "errore-misura-correzioni" in corso. In risposta, l'ambiente genera quindi una nuova *faulty syndrome* e ripristina la cronologia delle azioni. Qualora l'agente abbia selezionato l'identità prematuramente o in modo errato, la configurazione di errore residuo si somma a quella nuova[2], complicando eventualmente la decodifica e conducendo più rapidamente a una condizione terminale.

Tuttavia, è fondamentale che l'agente non sia vincolato a riportare il sistema all'interno del *code space* tra una misura di sindrome e la successiva. Gli algoritmi di *Reinforcement Learning* infatti ottimizzano il rendimento cumulativo e non solo la ricompensa immediata, il che consente all'agente di apprendere strategie che includono azioni il cui effetto positivo si manifesta solo nel lungo periodo. Nel contesto in analisi, in presenza di un volume di sindrome particolarmente rumoroso o ambiguo (ad esempio dovuto a molteplici errori di misura) l'agente, grazie a questa libertà, può decidere consapevolmente di effettuare solo una correzione parziale e richiedere una nuova misura di sindrome, nella quale la configurazione di errore potrebbe risultare più chiara. Questa caratteristica conferisce al decoder la possibilità di sviluppare strategie di correzione non convenzionali e sequenziali, potenzialmente più efficaci rispetto ai metodi di decodifica tradizionali (come MWPM), che non considerano esplicitamente questa dinamica temporale.

[2]La sindrome viene sempre generata in riferimento allo *hidden-state* che se dunque non è realmente ripristinato rischia di complicare ulteriormente la nuova configurazione di errore. Diversa è la situazione per il metodo *reset()*: in tal caso anche lo stato nascosto viene ovviamente ripristinato.

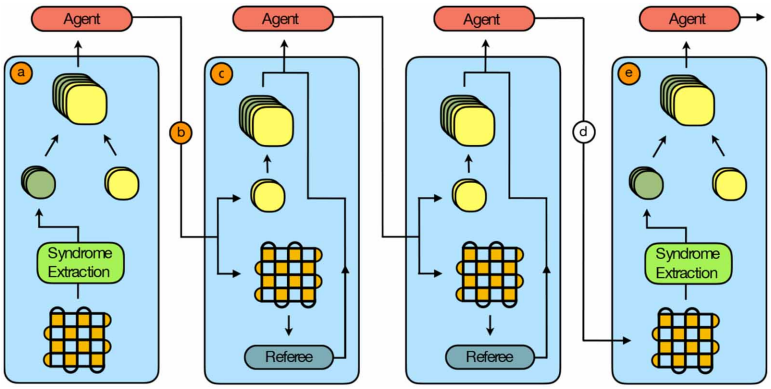


Figura 3.2: Processo ciclico di interazione con l'ambiente [9].

La figura 3.2 mostra graficamente il processo ciclico di interazione dell'agente con questo ambiente: il primo box azzurro rappresenta l'applicazione del metodo *reset()* mentre i successivi sono invece relativi al metodo *step()*. In particolare, l'ultimo box illustra il comportamento dell'ambiente in risposta all'azione identità (o azione ripetuta) scelta dall'agente, con conseguente generazione di nuova sindrome.

Qubit lifetime

L'obiettivo di un agente RL è quello di massimizzare la ricompensa cumulativa di ciascun episodio di interazione con l'ambiente. Nel contesto descritto, ciò equivale ad eseguire le correzioni necessarie a mantenere più a lungo possibile la coerenza logica del qubit codificato, ritardando il più possibile il verificarsi di una condizione terminale in cui il sistema non è più decodificabile.

Tale obiettivo può quindi essere espresso in termini di un parametro fondamentale dell'ambiente, ossia la *lifetime* (vita media) del qubit logico. Essa è definita come il numero medio di sindromi osservate dall'agente per episodio, e cioè il numero di cicli "errore-misura-correzione" che il sistema riesce a sostenere prima del fallimento logico. In questo senso, la *lifetime* rappresenta una misura discreta del tempo di coerenza del qubit, prima che il decoder *referee* determini che lo stato di errore raggiunto è irreversibile.
La vita media costituisce quindi una quantità direttamente correlata alla ricompensa cumulativa media per episodio, ma che consente di valutare in modo più concreto le performance ottenute da una determinata politica addestrata in uno specifico ambiente. Consideriamo, ad esempio, un ambiente caratterizzato da una probabilità di errore p_{err}: in media ci si aspetta che un qubit fisico rimanga coerente per circa

$$\frac{1}{p_{\text{err}}}$$

cicli prima di subire un errore. Pertanto questo valore rappresenta in prima approssimazione la *lifetime* del qubit fisico. La *logical lifetime* ottenuta testando una politica addestrata nello stesso ambiente (in modalità *greedy* su un campione di episodi simulati) può quindi essere confrontata con tale riferimento, fornendo una misura diretta dell'efficacia del processo di decodifica. Un decoder efficace è quindi quello per cui la *logical lifetime* risulta significativamente superiore a quella di un singolo qubit fisico, dimostrando la capacità del codice e della politica di mantenere l'informazione logica nonostante la presenza di rumore.

Ambienti simulati

Come anticipato, in questo progetto è stato considerato un *Surface Code* di dimensione 5×5 ($d = 5$), presentato all'agente sotto forma di un *syndrome volume* di profondità 5. In particolare, è stato adottato un modello di errore limitato al *bit-flip noise*, assumendo la stessa probabilità di errore per ciascun qubit fisico e per ciascuno stabilizzatore.
Gli ambienti simulati considerano probabilità di errore crescenti, definite come

$$p_{\text{err}} \equiv p_{\text{phys}} = p_{\text{meas}} \in \{0.001,\ 0.003,\ 0.005,\ 0.007,\ 0.009\} \tag{3.1}$$

3.1.2 Architettura del decoder

Delineate le principali caratteristiche e il funzionamento dell'ambiente utilizzato per la simulazione del problema di decodifica, vengono ora presentate l'architettura e le componenti sviluppate per la costruzione dell'agente DDQN. Per l'implementazione di tali componenti è stata adottata la libreria di *PyTorch*, ormai standard di riferimento per lo sviluppo di algoritmi di *Reinforcement Learning* come quello impiegato in questo progetto. In particolare, per la realizzazione dell'agente *Double Deep Q-Network* sono stati sviluppati modelli personalizzati per la rete neurale, la memoria di esperienza (*Replay Buffer*) e una classe agente dedicata, progettata per coordinare in modo efficiente l'interazione tra questi elementi. Estratti di codice relativi a questi modelli sono riportati in Appendice B.

La rete neurale

La rete neurale rappresenta la componente centrale che estende l'algoritmo di questo progetto al paradigma di *Deep Reinforcement Learning*. In particolare, essa funge da approssimatore della funzione valore di azione $Q(s, a)$, sfruttando la propria capacità di generalizzare l'esperienza acquisita a partire da un insieme limitato di interazioni. Le sue proprietà di elaborazione di dati complessi e strutturati, come le rappresentazioni multidimensionali dello stato fornite dall'ambiente adottato, la rendono uno strumento indispensabile per implementare un agente di RL in questo contesto.
A questo proposito, si è scelto di adottare — in analogia a quanto proposto da Sweke et al. [9] — una rete neurale composta da tre strati convoluzionali seguiti da un singolo tratto completamente connesso (*feed forward*) prima dello strato di output, che restituisce un valore Q per ciascuna delle azioni possibili nello spazio delle azioni $\mathcal{A}$.
Indicando uno strato convoluzionale tramite la terna $[n,\ w,\ s]$ dove:

- n rappresenta il numero di filtri
- w rappresenta la dimensione del kernel
- s rappresenta lo stride, ossia il passo con cui ciascun filtro si muove lungo la matrice

e uno strato *feed forward* con la coppia $[n,\ d]$ dove:

- n rappresenta il numero di neuroni
- d rappresenta il tasso di *dropout* (vedi sotto)

la struttura complessiva della rete neurale adoperata in questo progetto risulta quindi

$$\big[[64, 3, 2], [32, 2, 1], [32, 2, 1], [512, 0.2], [|\mathcal{A}|, 0]\big] \tag{3.2}$$

dove $|\mathcal{A}|$ indica la dimensione dello spazio delle azioni. Per garantire la capacità di approssimazione di funzioni non lineari, dopo ciascun *layer* è impiegata la funzione di attivazione ReLU, ad eccezione dello strato di output.
Il *dropout* è una tecnica di regolarizzazione utilizzata per rendere più robuste e generalizzabili le stime che una rete può produrre durante un addestramento. Durante ciascuna iterazione di training, il *dropout* disattiva casualmente una percentuale d dei neuroni di un *layer* (nel caso di questo progetto, $d = 0.2$ nel livello *feed forward*), impedendo alla rete di dipendere eccessivamente da specifiche connessioni e favorendo l'apprendimento di rappresentazioni più distribuite.
Un'immagine rappresentativa della struttura della rete neurale adottata — comprensiva di strati convoluzionali, appiattimento, livello completamente connesso e output — è presentata in fig. 3.3.

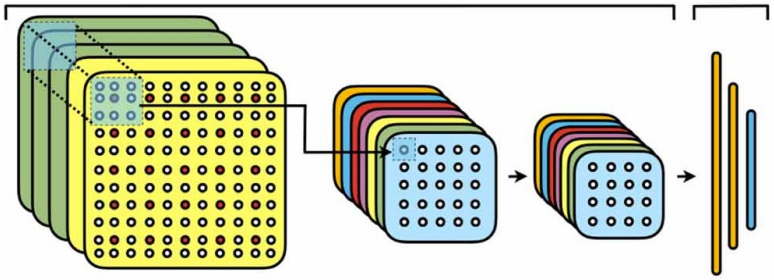


Figura 3.3: Struttura della rete neurale implementata [9].

Infine, per migliorare la stabilità e l'efficienza dell'apprendimento, la rete neurale impiegata adotta un'architettura di tipo *dueling* [37], particolarmente efficace nel contesto del *Reinforcement Learning*. Questa architettura nasce con l'obiettivo di separare la stima del valore complessivo di uno stato dalla stima dell'utilità relativa delle singole azioni, consentendo di apprendere in modo più efficiente anche quando le azioni disponibili producono effetti simili sulla ricompensa futura. In molti stati, infatti, più azioni possono avere un impatto quasi equivalente sul ritorno atteso: una rete tradizionale sarebbe comunque costretta a stimare un valore $Q(s, a)$ distinto per ciascuna, introducendo rumore e rallentando la convergenza. L'architettura *dueling* affronta questo problema introducendo due flussi distinti per l'elaborazione dell'output, volti a stimare rispettivamente:

- il valore dello stato $V(s)$ (flusso del valore);
- l'utilità relativa di ciascuna azione $A(s, a)$ (flusso del vantaggio).

Le due componenti vengono poi ricombinate per la stima dei valori Q secondo la relazione[3]

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')\right) \tag{3.3}$$

[3]Il termine di vantaggio viene normalizzato rispetto alla media per garantire che $V(s)$ rappresenti il valore assoluto dello stato, eliminando ambiguità numeriche.

ottenendo così una stima più stabile e un aggiornamento dei pesi più efficiente a ogni batch campionato. Un estratto di codice contenente la classe adoperata per costruire la rete neurale di questo progetto è inserito in Appendice B, Codice B.2.

Il Replay Buffer

Un elemento fondamentale per la stabilità di un apprendimento nel contesto del *Deep Q-Learning* è rappresentato dalla memoria di esperienza, o *Replay Buffer*.
Questa struttura ha il compito di memorizzare le transizioni osservate dall'agente durante l'interazione con l'ambiente, ossia le tuple nella forma[4]

$$(s,\ a,\ r,\ s', terminated)$$

dove s è lo stato corrente, a è l'azione selezionata, r ricompensa ottenuta, s' lo stato successivo e *terminated* — talvolta indicato come *done* — una variabile booleana per segnalare l'eventuale fine dell'episodio.
L'utilizzo di un *Replay Buffer* risulta fondamentale per spezzare la correlazione temporale che naturalmente si manifesta tra esperienze successive, consentendo alla rete di apprendere da campioni estratti in modo casuale. Durante il processo di ottimizzazione, infatti, non è possibile utilizzare l'intera esperienza accumulata, ma soltanto un sottoinsieme di transizioni selezionate, al fine di mantenere la computazione entro tempi ragionevoli. Tale limitazione può introdurre instabilità negli aggiornamenti dei pesi, in particolare quando la dimensione del campione è ridotta. L'impiego di una memoria di esperienze, da cui campionare in maniera casuale, consente invece di ridurre le dipendenze sequenziali tra dati successivi, garantendo aggiornamenti più stabili e una migliore convergenza dell'algoritmo. Inoltre, la possibilità di riutilizzare le stesse esperienze in più iterazioni rende l'addestramento più efficiente anche dal punto di vista statistico.

Nel progetto è stata implementata una classe personalizzata denominata *ReplayBuffer*, progettata per gestire in maniera efficiente il salvataggio e il campionamento delle esperienze. Il funzionamento di questo oggetto è basato su due parametri fondamentali: la dimensione della memoria (*buffer-size*) e la dimensione del campione estratto (*batch-size*). In particolare, la classe implementa due metodi principali:

- **push()**: consente di salvare una transizione. In particolare salva gli oggetti in formato *Numpy array* all'interno di una memoria circolare di dimensione massima pari a *buffer size*. Una volta raggiunta la capacità massima, le esperienze più vecchie vengono progressivamente sovrascritte.

- **sample()**: effettua il campionamento casuale di un mini-batch di dimensione fissa (*batch size*), restituendo i dati in formato tensoriale, compatibile con i modelli implementati in *PyTorch*.

La scelta dei parametri del buffer costituisce un compromesso tra efficienza di apprendimento e costo computazionale. Un valore maggiore di *buffer-size* consente di conservare un insieme più ampio e diversificato di esperienze, migliorando la capacità

[4] Le nuove API della classe *Gymnasium* in realtà prevedono che il metodo *step()* restituisca un ulteriore parametro, *truncated*, per segnalare la fine di un episodio prematura. Per coerenza con le nuove direttive della libreria è stato inserito anche questo valore, tuttavia mai utilizzato e dunque non salvato nel *Replay Buffer*.

di generalizzazione del modello; tuttavia, richiede maggiore memoria e tempi più lunghi per le operazioni di campionamento. Analogamente, un valore elevato di *batch-size* produce stime dei gradienti più stabili ma aumenta il tempo di elaborazione per ciascun aggiornamento. Per tutte le simulazioni svolte in questo progetto sono stati utilizzati rispettivamente i seguenti parametri:

$$\textit{buffer-size} = 50000 \tag{3.4}$$

$$\textit{batch-size} = 64 \tag{3.5}$$

Un estratto di codice contenente la classe adoperata per la gestione della memoria di esperienze è riportato in Appendice B, Codice B.1.

L'agente DDQN

L'agente rappresenta l'entità protagonista di un episodio nel paradigma di *Reinforcement Learning*, capace di prendere decisioni, compiere azioni, acquisire esperienza e apprendere strategie ottimali. Nel contesto trattato in questo elaborato, esso racchiude la logica di apprendimento del decoder, fungendo da coordinatore tra le varie componenti del modello — le reti neurali e la memoria di esperienze — per implementare efficacemente il processo di ottimizzazione.
È stata implementata una classe personalizzata denominata *PyTorchDDQNAgent*, ispirata all'architettura del *Double Deep Q-Network* (DDQN), il cui scopo è ottimizzare in maniera stabile e progressiva la funzione di valore d'azione $Q(s, a)$ rappresentata dalla rete neurale. In particolare, la classe gestisce e integra tutti gli elementi essenziali dell'algoritmo: la rete principale (*policy_net*) e la rete target (*target_net*) per la stima dei valori Q, la memoria di esperienze (*Replay Buffer*) per il campionamento casuale delle transizioni, e infine l'ottimizzatore con la funzione di perdita per l'aggiornamento dei pesi.

Il cuore del processo di apprendimento è implementato tramite il metodo *learn()*, che esegue un singolo passo di ottimizzazione di DDQN.
Dopo una prima fase iniziale di *warm-up*, necessaria affiché la memoria venga riempita con un numero sufficiente di esperienze, l'agente campiona un mini-batch di transizioni dalla memoria ed esegue il processo di ottimizzazione dei pesi secondo la logica prevista dall'algoritmo.
In particolare, per ciascuna transizione $(s, a, r, s', \textit{terminated})$ vengono calcolati:

- il valore corrente $Q_{\text{policy}}(s, a)$ tramite la rete principale (*policy_net*)
- il valore target, definito come[5]

$$y = r + \gamma\, Q_{\text{target}}\left(s', \arg\max_{a'} Q_{\text{policy}}(s', a')\right)(1 - \textit{terminated})$$

  dove γ rappresenta il fattore di sconto. In questo schema la selezione dell'azione avviene tramite la rete principale, mentre la valutazione del corrispondente valore Q è affidata alla rete target.

[5]Il parametro booleano *terminated* viene convertito in *float*. In questo modo, per gli stati terminali, il termine futuro viene annullato e la ricompensa non è più propagata.

Questo meccanismo per la stima del target — caratteristico del *Double DQN* — riduce la tendenza del DQN classico a sovrastimare i valori Q. Come discusso in sezione 2.2.3 infatti, l'algoritmo tradizionale impiega un'unica rete (*target_net*) sia per selezionare che per valutare le azioni, introducendo una correlazione che porta ad una sovrastima sistematica. L'introduzione della doppia rete nel calcolo di questo valore consente di disaccoppiare i due processi, mitigando questo effetto e migliorando così la stabilità dell'apprendimento.
A questo punto, noti questi valori, il processo prosegue secondo il normale protocollo di ottimizzazione dei pesi di una rete neurale. Per queste fasi — rispettivamente calcolo della loss, *backpropagation* e ottimizzazione — sono state utilizzate le funzioni native della libreria *PyTorch*.
In particolare come funzione di perdita si è scelto di utilizzare la funzione *SmoothL1Loss*, come proposto da Matekole et al. [10], simile alla funzione *Huber Loss*.

$$\mathcal{L} = \text{SmoothL1Loss}(Q_{\text{policy(s,a)}}, y) \tag{3.6}$$

Successivamente, viene eseguito il calcolo dei gradienti rispetto ai parametri della rete tramite il metodo *backward()*. È buona pratica, soprattutto nel contesto di RL, eseguire una procedura chiamata *gradient clipping*, implementata in questo caso mediante la apposita funzione fornita della libreria. In sostanza, qualora la norma complessiva dei gradienti superi una determinata soglia (pari a 1 nel presente caso), essi vengono riscalati proporzionalmente in modo che la loro norma totale coincida con tale valore limite. Questa tecnica consente di prevenire il fenomeno di *esplosione dei gradienti* tipico dei processi in cui il segnale di ricompensa può essere molto variabile, evitando che l'algoritmo compia aggiornamenti eccessivi dei pesi che potrebbero compromettere la stabilità dell'apprendimento.
Infine, viene eseguito un passo di aggiornamento dei parametri tramite l'ottimizzatore *Adam* della libreria, scelto per la sua capacità di adattare dinamicamente il tasso di apprendimento η inizialmente fornito, il che garantisce una convergenza più stabile ed efficiente.
Il metodo *learn()* restituisce dunque il valore della loss, utile per monitorare l'andamento dell'addestramento e valutare la capacità dell'agente di migliorare progressivamente la propria politica di decisione.
L'unica altra funzione della classe agente è *update_target_net()*, che aggiorna i pesi della rete target copiandoli dalla rete principale. Questo metodo viene chiamato con una frequenza fissa in termini di step, determinata da un parametro apposito all'interno della funzione di addestramento.
Oltre all'insieme delle istanze necessarie per eseguire un passo di ottimizzazione, l'agente viene inizializzato con i parametri fondamentali per questo processo, ovvero fattore di sconto γ e tasso di apprendimento η. Nel caso di questo progetto, i valori utilizzati per tutte le simulazioni eseguite sono riportati di seguito.

$$\gamma = 0.99 \tag{3.7}$$
$$\eta = 0.0001 \tag{3.8}$$

Un estratto di codice contenente la classe agente implementata è riportato in Appendice B, Codice B.3.

3.2 Addestramento dell'agente

Definiti gli elementi necessari per comprendere il funzionamento dell'ambiente e l'architettura con cui è stato sviluppato l'agente, in questa sezione vengono descritte le funzioni implementate per l'addestramento secondo il paradigma del *Reinforcement Learning*. In particolare, viene presentato il processo di addestramento basato sull'algoritmo *Double Deep Q-Network* (DDQN) e sulla sua estensione con *Probabilistic Policy Reuse* (PPR), implementato mediante funzioni che si interfacciano con le classi e le istanze precedentemente illustrate. Successivamente viene riassunto l'insieme dei parametri impiegati nel training, con particolare attenzione alle scelte relative agli iperparametri che regolano le strategie di esplorazione nei due casi. Infine, viene fornita una descrizione della funzione di test e dei criteri adottati la valutazione delle *policy* addestrate e delle relative prestazioni.

3.2.1 Funzioni di training DDQN e DDQN + PPR

Le funzioni di training costituiscono il cuore dell'implementazione del paradigma di *Reinforcement Learning* sviluppato in questo progetto. Esse sfruttano gli elementi precedentemente descritti — ambiente, agente e strutture di supporto — per definire il ciclo di interazione tipico di un *Markov Decision Process* (MDP), all'interno del quale l'agente apprende attraverso esperienza diretta. Il processo di addestramento è organizzato in episodi, durante i quali l'agente osserva lo stato dell'ambiente, seleziona un'azione, riceve una ricompensa e aggiorna la propria politica in funzione del feedback che ottiene.
In particolare sono state sviluppate due funzioni di training ispirate rispettivamente all'algoritmo *Double Deep Q-Network* e alla sua estensione con *Probabilistic Policy Reuse*. In Appendice A sono riportati gli pseudocodici generali di riferimento per entrambi gli algoritmi, utilizzati come base concettuale per lo sviluppo del codice effettivo. I due modelli condividono la stessa struttura generale, pur differendo in alcuni aspetti specifici e in particolare nella strategia di selezione dell'azione. Entrambi includono meccanismi aggiuntivi nella fase di fine episodio, pensati per monitorare l'andamento dell'apprendimento, ottimizzarne la durata e permettere l'analisi a posteriori.

Struttura e logica generale (DDQN)

Di seguito viene presentata la struttura generale delle funzioni di training adoperate, in particolare riferita al caso del DDQN tradizionale.
La funzione prende come input le varie istanze presentate, tra cui in particolare l'agente (*agent*) e l'ambiente (*env*), insieme a tutti i parametri necessari, forniti per mezzo di un dizionario apposito. Dopo quindi un primo momento in cui vengono inizializzate tutte le variabili, viene introdotto il ciclo principale su cui è costruito l'addestramento, iterato su un numero massimo di step di interazione predefinito. Di seguito è riportato un estratto di codice con riferimento ai metodi delle istanze precedentemente illustrate: esso rappresenta il funzionamento schematico delle funzioni implementate.

```
# Alla funzione vengono passati agent, env e tutti i parametri necessari
state, info = env.reset()

for step in range(max_timestep):
    # Selezione dell'azione epsilon-Greedy (DDQN)
    action = select_action_eps_greedy(state)

    next_state, reward, done, info = env.step(action)

    memory.push(state, action, reward, next_state, done)

    state = next_state

    if len(memory) > learning_starts:
        agent.learn()

    if step % freq_target_update == 0:
        agent.update_target_net()


    if done:
        # ... Gestione fine episodio (metriche, log, checkpoint, early-stop) ...
        state, info = env.reset()
```

In riferimento allo schema proposto, si noti che:

- Ogni episodio ha inizio con il metodo *env.reset()* che fornisce lo stato iniziale.
- Per ogni step del ciclo principale viene selezionata un'azione per lo stato corrente. Nello schema sopra, questa fase è rappresentata dalla chiamata della funzione fittizia *select_action_eps_greedy()*, che racchiude il comportamento generale di una politica ϵ-*Greedy*. Si noti però che questo è un esempio semplificato: nel codice effettivo la scelta dell'azione impiega alcuni passaggi preliminari e una funzione completa, che gestisce tutti i dettagli di aggiornamento e calcolo degli ϵ dinamici.
- L'azione scelta è applicata all'ambiente tramite il metodo *env.step()*, producendo la nuova transizione.
- La transizione viene salvata nel *Replay Buffer* tramite *memory.push()*.
- Se la memoria è contiene abbastanza dati, viene eseguito un passo di ottimizzazione della rete tramite *agent.learn()*.
- Con frequenza prefissata, la rete target è aggiornata con *update_target_net()*. In particolare, nelle simulazioni svolte, si è scelto di usare

$$freq_target_update = 5000$$

- La fine di un episodio è segnalata dalla variabile booleana *done*, la quale attiva le procedure di monitoraggio, registrazione dei log, salvataggio dei checkpoint e la logica di *early stopping*. Successivamente, lo stato dell'ambiente viene ripristinato tramite *env.reset()* per avviare un nuovo episodio.

Estensione con PPR

La funzione di training sviluppata per operare con l'estensione al paradigma di *continual learning* PPR è basata sulla stessa struttura generale, modificata principalmente in due aspetti:

1. Il processo di selezione per ciascun episodio di una politica dalla libreria di esperienze in input.

2. La strategia di selezione dell'azione tramite π-*reuse*.

In particolare, lo schema precedente viene implementato con le seguenti strutture. All'inizio di ciascun episodio, insieme al reset dell'ambiente viene selezionata una politica dalla libreria *policy_library* secondo il seguente schema:

```
# ... all'inizio di ciascun episodio, dopo env.reset() ...
    # calcolo le probabilità softmax P
    P = softmax(W, current_tau)
    # selezione dell'indice k relativo ad una policy
    k = np.random.choice(len(W), p=P)
    # inizializzo la policy di reuse
    Li = policy_library[k-1] if k > 0 else None     # k=0 corrisponde alla policy
    attuale
```

dove L_i rappresenta la rete con i pesi relativi alla k-esima *policy* estratta, mentre W è un vettore contenente le ricompense medie ottenute per ciascun episodio di utilizzo di una determinata politica, aggiornato progressivamente secondo

```
# alla fine di ciascun episodio, dopo done=True ...
    # aggiorno le rewards cumulative (W) e l'utilizzo (U) della policy estratta k
    W[k] = (W[k] * U[k] + episode_reward) / (U[k] + 1)
    U[k] += 1
    # aggiorno la temperatura per la prossima estrazione di policy
    current_tau += tau_increment
```

Il parametro τ rappresenta la variabile di temperatura implementata per bilanciare il peso delle ricompense medie ottenute dalle varie politiche riutilizzate all'interno della distribuzione Softmax, necessario per garantire un'esplorazione casuale della libreria che con il tempo si riduce. Questo parametro è inizializzato a 0 e ad ogni episodio viene aggiornato con un incremento $\delta\tau$, il cui valore scelto è riportato di seguito.

$$\delta\tau = 0.0001 \tag{3.9}$$

La selezione di una politica per ciascun episodio è propedeutica all'implementazione della strategia di π-*reuse* per la selezione dell'azione, che può essere schematizzata nel seguente estratto di codice.

```
    action = None
    if k > 0:
        if np.random.rand() < current_psi:
            # Selezione dell'azione greedy per la politica riusata
            action = select_action_greedy(Li, state)
    if action is None:
        # Selezione dell'azione epsilon-Greedy sulla politica corrente
        action = select_action_eps_greedy(state)
```

Anche in questo caso, si noti che la rappresentazione schematica fa uso di funzioni fittizie per rappresentare lo sfruttamento di una politica passata piuttosto che l'esplorazione casuale. Estratti di codice più concreti sono riportati in Appendice B, mentre la gestione degli iperparametri ϵ e ψ sarà trattata in seguito.

Dinamica di fine episodio

Infine, come anticipato, le funzioni di training sviluppate includono una serie di accorgimenti progettati per migliorare l'efficienza dell'addestramento e consentirne un monitoraggio continuo, sia durante l'esecuzione (*online*) sia a posteriori.
In particolare, al termine di ciascun episodio vengono calcolate alcune metriche fondamentali, come la media mobile della ricompensa cumulativa per episodio e la *lifetime* media. Tali valori, insieme ad altri indicatori registrati durante il ciclo di interazione — ad esempio la funzione di perdita (*loss*) e gli iperparametri dinamici come ϵ e ψ — vengono salvati in un file di *history* aggiornato ad ogni episodio, così da consentire un'analisi quantitativa post-training. In parallelo, con una frequenza prefissata in termini di numero di episodi, le principali metriche vengono stampate a terminale, fornendo un riscontro immediato sull'andamento dell'addestramento.
Inoltre, le funzioni gestiscono il salvataggio automatico dei pesi della rete ogni qualvolta viene registrato un nuovo valore massimo della *lifetime* media, garantendo così la disponibilità del miglior modello addestrato oltre a quello finale. A questo proposito, viene anche implementato un meccanismo di *early stopping*: una volta raggiunta una soglia obiettivo — solitamente fissata in corrispondenza della *lifetime* media attesa per un singolo qubit fisico nel modello di rumore considerato — l'addestramento viene interrotto anticipatamente dopo che tale metrica non migliora in un numero predefinito di episodi (*patience*). Ciò consente di ridurre i tempi computazionali e, al contempo, di evitare un sovra-addestramento della rete, assicurando la conservazione del modello ottimale.
Alcuni riferimenti più tecnici ed estratti di codice sono riportati in Appendice B.

3.2.2 Parametri e impostazioni di addestramento

I parametri utilizzati per l'addestramento coincidono in gran parte con quelli già presentati nella descrizione delle funzioni e delle istanze sviluppate per la realizzazione di questo progetto. Per ciascuna simulazione è stato predisposto un dizionario contenente tutti i valori di configurazione impiegati nella sessione di training, così da consentirne un accesso immediato all'interno di ogni funzione o classe. Durante l'inizializzazione delle procedure di logging, una copia del dizionario viene inoltre salvata nella directory dedicata alla simulazione, garantendo la tracciabilità completa dei parametri sperimentali. Una panoramica completa di tutti i parametri utilizzati, con i rispettivi valori e descrizioni, è riportata in Appendice B.
Tra questi rivestono un ruolo di particolare importanza gli iperparametri ϵ e ψ, volti rispettivamente al bilanciamento tra i comportamenti di esplorazione e sfruttamento, e alla regolazione del riuso delle *policy* considerate.

Gestione dinamica di ϵ e ψ

Come spesso accade nel contesto di algoritmi *off-policy* di *Reinforcement Learing*, si è scelto di adottare una gestione dinamica degli iperparametri comportamentali ϵ e ψ. In particolare, per agevolare il processo di apprendimento e favorire un equilibrio progressivo tra esplorazione e sfruttamento, è pratica comune — nel caso di algoritmi come il DDQN — utilizzare valori dipendenti dal tempo, aggiornati secondo una strategia di *annealing* più o meno graduale. Questo approccio consente all'agente di privilegiare

l'esplorazione o il riutilizzo nelle fasi iniziali dell'addestramento, riducendo progressivamente tali comportamenti man mano che viene acquisita esperienza, così da poter sfruttare in modo più efficace le transizioni memorizzate e favorire una convergenza più stabile della politica.
Per quanto riguarda la strategia di esplorazione ϵ-*Greedy* si è scelto di utilizzare — per semplicità del modello e per un primo approccio sperimentale[6] — un decadimento lineare del valore di ϵ. In particolare

$$\epsilon(t) = \begin{cases} \epsilon_{\max} + \frac{\epsilon_{\text{final}} - \epsilon_{\max}}{t_{\exp}} t & \text{se } t < t_{\exp} \\ \epsilon_{\text{final}} & \text{altrimenti} \end{cases} \tag{3.10}$$

dove t rappresenta il numero di step eseguiti dall'inizio dell'addestramento e, con riferimento ai valori adottati in questo progetto:

- $\epsilon_{\max} = 1.0$ rappresenta il valore iniziale (massima esplorazione),
- $\epsilon_{\text{final}} = 0.02$ è il valore minimo raggiunto al termine della fase di esplorazione,
- $t_{\exp} = 75{,}000$ indica il numero di step su cui è applicato il decadimento lineare.

La gestione della strategia di esplorazione ϵ-*Greedy* con questo modello di annullamento del parametro è implementata nel codice tramite un'apposita istanza della classe *LinearAnnealedEpsGreedyPolicy*. Quest'ultima, mediante metodi dedicati e in accordo con le regole definite dall'ambiente, gestisce autonomamente sia la scelta dell'azione casuale o deterministica, sia l'aggiornamento progressivo del valore di ϵ a ogni step. I dettagli relativi a questa classe sono riportati in Appendice B, codice B.4.
Questa configurazione è stata determinata in fase di addestramento del modello DDQN, e successivamente mantenuta invariata anche per l'estensione DDQN + PPR, al fine di garantire la coerenza del confronto sperimentale.

Per quanto concerne invece la strategia di π-*reuse* implementata nella funzione con PPR, si è scelto — al fine di una prima sperimentazione — di adottare anche in questo caso un decadimento lineare per il parametro ψ. Il valore di questo parametro controlla la probabilità di riutilizzare una politica preesistente in modo puramente *Greedy*, piuttosto che adoperare la strategia ϵ-*Greedy* sulla rete corrente.
Si noti quindi che, per quanto entrambi i parametri contribuiscano all'evoluzione della meccanica decisionale dell'agente, il loro ruolo è differente: mentre ϵ regola un comportamento di tipo *intra-policy*, volto al bilanciamento tra esplorazione e sfruttamento all'interno della singola politica, ψ agisce invece a livello *inter-policy*, controllando la propensione al riutilizzo di politiche preaddestrate rispetto all'apprendimento di una nuova.

In maniera del tutto analoga al caso precedente, la funzione di aggiornamento del parametro ψ è definita come

$$\psi(t) = \begin{cases} \psi_{\max} + \frac{\psi_{\text{final}} - \psi_{\max}}{t_{\text{reuse}}} t & \text{se } t < t_{\text{reuse}} \\ \psi_{\text{final}} & \text{altrimenti} \end{cases} \tag{3.11}$$

dove t rappresenta il numero di step dall'inizio dell'addestramento e, con riferimento ai valori adottati nel presente progetto:

[6] La notevole durata delle sessioni di training non ha permesso di sperimentare diverse configurazioni del comportamento dei parametri. Si è pertanto optato per un modello lineare semplice, lasciando spazio a possibili approfondimenti futuri.

- $\psi_{\text{max}} = 0.85$ indica l'intensità iniziale del riuso di politiche pregresse;
- $\psi_{\text{final}} = 0.02$ è il valore minimo raggiunto al termine della fase di *annealing*;
- $t_{\text{reuse}} = 175{,}000$ è il numero di step su cui è applicato il decadimento lineare.

Questa configurazione consente una transizione più graduale rispetto al caso dell'esplorazione. Poiché i due parametri agiscono in modo annidato, una scelta di questo tipo garantisce un riutilizzo più intenso delle esperienze pregresse nelle prime fasi dell'addestramento, a discapito dell'esplorazione casuale del nuovo ambiente. Lo scopo prefissato infatti era quello di dimostrare che, nel contesto di ambienti simili, tecniche di *continual learning* come la PPR costituiscono un approccio efficace per favorire il trasferimento di conoscenza tra scenari affini, riducendo i tempi di convergenza e migliorando la stabilità dell'apprendimento senza compromettere la capacità di adattamento.
La gestione del decadimento del parametro ψ secondo questo modello è implementata all'interno del ciclo di interazione di DDQN + PPR, tramite un'apposita funzione dedicata (*linear_update_psi()*). Un estratto del codice relativo a tale funzione è riportato in Appendice B, codice B.5.

3.2.3 Funzione di test e valutazione della policy

Infine, per la valutazione delle politiche apprese durante le simulazioni, è stata implementata una specifica funzione di test. La struttura di questo modello è pressoché quella della funzione di training precedentemente descritta, con però una differenza fondamentale nella strategia di selezione delle azioni. Durante la fase di test infatti, le azioni vengono scelte in maniera puramente deterministica — ovvero *Greedy* — rispetto alla politica da valutare; in questo modo, l'agente mette in atto esclusivamente le conoscenze apprese, consentendo di analizzare in modo diretto le prestazioni del modello attraverso le metriche prodotte nel corso della simulazione.

Fornito un ambiente coerente con quello di addestramento della politica che si vuole testare, la funzione simula un numero prestabilito di episodi in cui viene messa in pratica la strategia acquisita. Per ogni episodio vengono registrate come metriche la ricompensa cumulativa e la *lifetime* del qubit logico, che successivamente vengono restituite come output della funzione oppure salvate in un file di log, a seconda delle indicazioni fornite in ingresso. Dalle metriche ottenute è quindi possibile ricavare in particolar modo la *lifetime* media, utilizzata come principale parametro di valutazione della *policy*. In particolare, al fine di garantire un valore statisticamente significativo, ciascuna politica è stata testata su un numero di episodi tale da garantire l'analisi di almeno 2×10^6 sindromi da parte del decoder, in linea con quanto proposto da Sweke et al. [36]. Questo valore può essere confrontato con la vita media prevista per un singolo qubit fisico sottoposto alle stesse condizioni di rumore, mettendo in evidenza l'effettivo incremento, in termini di tempo (*discreto*) di coerenza, che un *Surface Code* munito di un decoder simile è in grado di fornire.

3.3 Risultati sperimentali

Infine, in quest'ultima sezione vengono presentati i risultati sperimentali ottenuti. L'obiettivo è quello di mostrare, attraverso un'analisi delle metriche raccolte durante le fasi di addestramento e di test delle politiche, le potenzialità di un decoder come quello sviluppato in questo progetto e, in particolare, il contributo che un approccio basato sul *continual learning* come PPR può offrire in termini di adattabilità a un nuovo ambiente. A questo proposito, dopo una descrizione introduttiva delle simulazioni svolte, vengono presentati i grafici relativi al confronto tra i due metodi di addestramento sviluppati e, successivamente, un'analisi dei valori di *logical lifetime* ottenuti dai test sulle politiche.

Le simulazioni sono state condotte in ambienti affetti da errori di tipo *bit-flip*, caratterizzati da un tasso di errore sulla misura uguale a quello fisico ($p_{\text{err}} \equiv p_{\text{phys}} = p_{\text{meas}}$). In una fase iniziale, sono stati addestrati agenti DDQN su modelli di errore con probabilità crescenti, rispettivamente

$$p_{\text{err}} \in \{0.001,\ 0.003,\ 0.005,\ 0.007,\ 0.009\}$$

utilizzando i metodi e i parametri descritti in precedenza. L'insieme delle politiche generate è stato successivamente impiegato per costruire le librerie di esperienza necessarie per implementare la strategia di π-*reuse*. In seguito, sono stati simulati nuovamente gli stessi ambienti — ad eccezione del primo — per l'addestramento tramite DDQN con il supporto del metodo PPR, secondo lo schema presentato in tab. 3.1.

Ambiente	**Metodo DDQN**	**Metodo DDQN + PPR**	
Prob. errore p_{err}	**Politica π generata**	**Libreria $\mathcal{L}$ usata**	**Politica π_{PPR} generata**
0.001	$\pi_{0.001}$	—	—
0.003	$\pi_{0.003}$	$\{\pi_{0.001}\}$	$\pi_{\text{PPR }0.003}$
0.005	$\pi_{0.005}$	$\{\pi_{0.001}, \pi_{0.003}\}$	$\pi_{\text{PPR }0.005}$
0.007	$\pi_{0.007}$	$\{\pi_{0.001}, \pi_{0.003}, \pi_{0.005}\}$	$\pi_{\text{PPR }0.007}$
0.009	$\pi_{0.009}$	$\{\pi_{0.001}, \pi_{0.003}, \pi_{0.005}, \pi_{0.007}\}$	$\pi_{\text{PPR }0.009}$

Tabella 3.1: Schema del procedimento sperimentale: generazione e riutilizzo delle *policy*.

Si noti che ciascuna politica addestrata tramite con DDQN + PPR ha quindi beneficiato del supporto di una libreria di strategie relative ad ambienti di difficoltà inferiore, secondo la logica dei metodi di *continual learning*.
L'insieme delle metriche raccolte durante ciascun addestramento ha permesso di costruire grafici relativi alla *lifetime* media — calcolata come media mobile su una finestra di 500 episodi[7] — in funzione degli episodi e dei *timestep* di interazione, utili per monitorare l'adattamento dell'agente all'ambiente.

[7] Si noti che, al fine di mantenere monitorata anche la fase iniziale di ciascun training, la media nei primi episodi è calcolata su un intervallo minore di esperienze, il che giustifica alcuni comportamenti instabili osservati in alcuni grafici.

Ambiente $p_{err} = 0.001$

Il primo ambiente simulato è quello con tasso di errore $p_{\text{err}} = 0.001$, utilizzato principalmente come caso di riferimento e per la costruzione della prima libreria di esperienze necessaria agli esperimenti successivi basati su PPR.

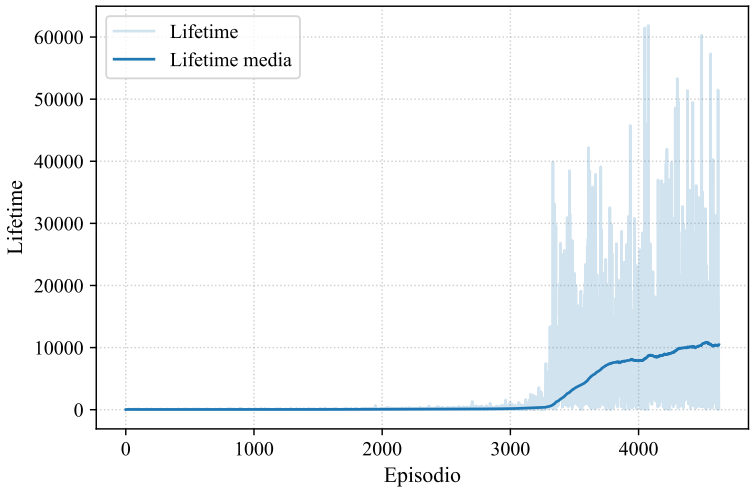


Figura 3.4: Grafico *lifetime* media per episodio, ambiente $p_{\text{err}} = 0.001$.

Il grafico in fig. 3.4 mostra l'andamento della *lifetime* media in funzione dell'episodio di addestramento. Analogamente, è possibile studiare l'adattamento del decoder al modello di rumore in termini di interazioni agente-ambiente, ossia *timestep*: il grafico in fig. 3.5 mostra il caso relativo al primo ambiente simulato.

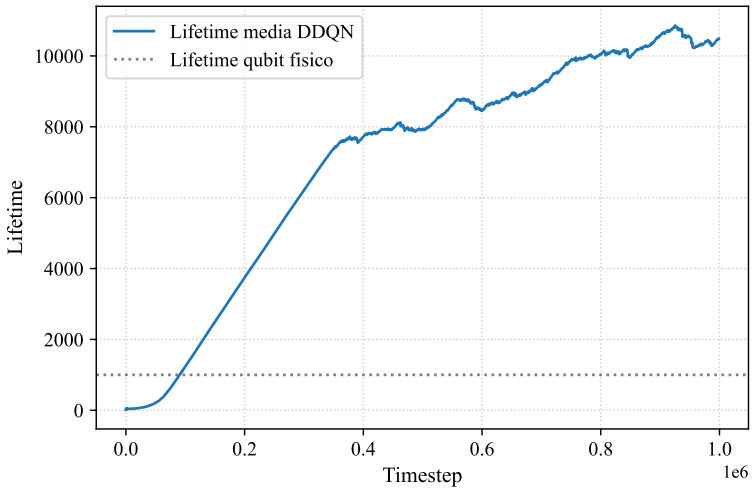


Figura 3.5: Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.001$.

Entrambi i grafici manifestano il comportamento tipico di un agente di *Reinforcement Learning*, evidenziando chiaramente gli effetti della dipendenza dalla strategia di esplorazione adottata. Dopo una prima fase fortemente esplorativa, in cui l'agente esegue scelte sostanzialmente casuali che limitano la durata degli episodi, esso inizia progressivamente a sfruttare le conoscenze acquisite, selezionando azioni via via più efficaci, fino circa a stabilizzarsi su un valore elevato di ricompensa media in termini di *lifetime*, specie se confrontato con il valore previsto per un singolo qubit fisico.

3.3.1 Confronto tra DDQN e DDQN + PPR

Per evidenziare il miglior adattamento del decoder addestrato con il metodo PPR, sono stati confrontati grafici analoghi a quelli presentati in precedenza, ottenuti implementando differenti strategie di addestramento sullo stesso ambiente. In particolare, si è scelto di analizzare l'andamento medio della *lifetime* in funzione dei *timestep* di addestramento. A differenza del numero di episodi, i *timestep* rappresentano il conteggio effettivo delle interazioni agente–ambiente e possono quindi essere interpretati come una variabile temporale discreta, più adatta a valutare la rapidità di adattamento del decoder nel tempo. Ulteriori grafici di supporto sono inseriti in Appendice C.

Ambiente $\mathbf{p_{err}} = \mathbf{0.003}$

Nel caso dell'ambiente con $p_{\text{err}} = 0.003$, il confronto tra i due metodi è riportato in fig. 3.6.

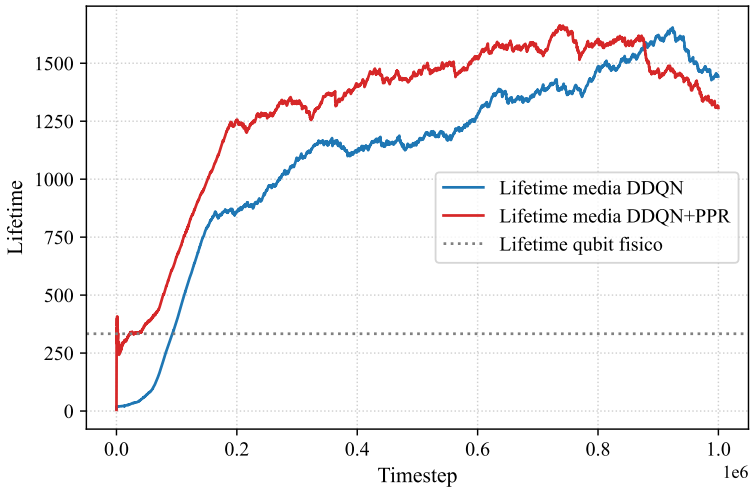


Figura 3.6: Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.003$.

L'immagine mostra chiaramente come il metodo DDQN + PPR mantenga un livello di ricompensa — in termini di vita media — maggiore durante tutta la durata del processo di addestramento, in particolare raggiungendo il picco in circa 2×10^5 step in meno rispetto al DDQN tradizionale, confermando le ipotesi per questo tipo di ambiente.

Ambiente $\mathbf{p_{err}} = \mathbf{0.005}$

Il secondo ambiente studiato è quello con tasso di errore 0.005, per il quale il confronto tra i due metodi è inserito in fig. 3.7.
Si osservi che il numero di timestep dell'addestramento DDQN risulta inferiore al milione fissato come soglia massima: ciò è dovuto all'attivazione del meccanismo di *early stopping* previsto dalla funzione di training. Per garantire un confronto equo, basato su un numero adeguato di interazioni agente–ambiente, tale meccanismo è stato disabilitato durante l'addestramento dell'agente con supporto PPR.
Anche in questo caso, livelli di *lifetime* media ottimale vengono raggiunti dall'agente DDQN + PPR in un numero minore di *timestep* rispetto al DDQN tradizionale, sebbene il vantaggio risulti meno marcato rispetto all'ambiente precedente. Inoltre, si osserva

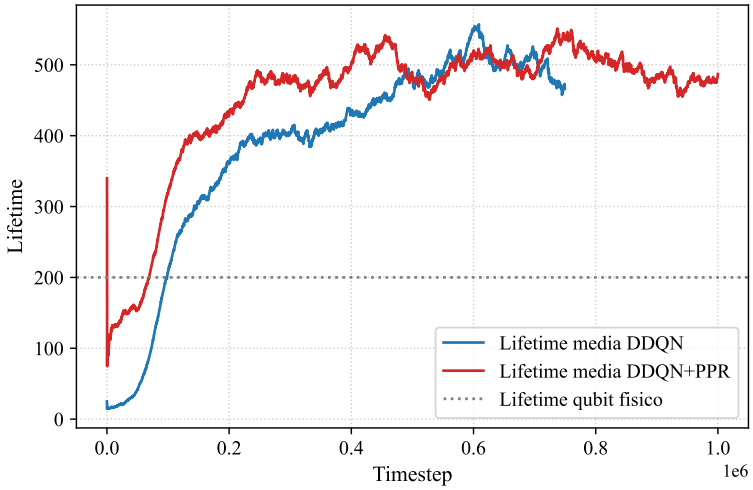


Figura 3.7: Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.005$.

come, all'aumentare del tasso di errore dell'ambiente, i livelli di prestazione ottimale tendano a ridursi, avvicinandosi alla *lifetime* media di un singolo qubit fisico ed evidenziando il progressivo aumento della difficoltà del compito di decodifica, dovuto anche soprattutto alla presenza di sindromi via via più rumorose, affette da un maggior numero di errori di misura sugli stabilizzatori.

Ambiente $\mathbf{p_{err} = 0.007}$

Nel caso dell'ambiente con $p_{\text{err}} = 0.007$, il confronto tra i due metodi è riportato in fig. 3.8.

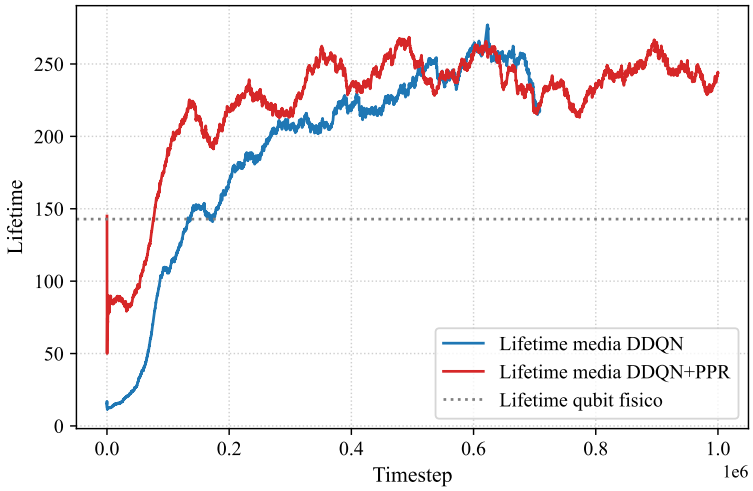


Figura 3.8: Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.007$.

In questo ambiente, le prestazioni complessive risultano inferiori rispetto ai casi precedenti, indicando una maggiore difficoltà di adattamento dell'agente. In particolare, l'agente DDQN + PPR mostra una notevole instabilità nei livelli di *lifetime* raggiunti,

probabilmente a causa dell'aumento del numero di politiche presenti in libreria[8] e del riutilizzo di strategie talvolta subottimali. Analizzando la scelta delle politiche riutilizzate in ciascun episodio (cfr. Appendice C, fig. C.1), si osserva come l'agente prediliga complessivamente il riuso della politica sviluppata per l'ambiente con $p_{\text{err}} = 0.003$, piuttosto che quella più simile di $p_{\text{err}} = 0.005$. Questo rappresenta un comportamento anomalo dell'agente, in contrasto con le simulazioni svolte negli altri ambienti, che può esser attribuito o ad una *policy* per l'ambiente precedente meno generalizzabile o, verosimilmente, inferiore rispetto alla media di prestazione attesa.
Complessivamente, questo risultato conferma il vantaggio di un approccio come PPR, ma evidenzia il fatto che per garantire un adattamento più efficace è necessario interfacciarsi con una libreria di esperienze pregresse qualitativamente adeguata.

Ambiente $\mathbf{p_{err} = 0.009}$

Infine, l'ultimo ambiente simulato è quello con tasso di errore 0.009, per il quale il confronto tra i due metodi è inserito in fig. 3.9.

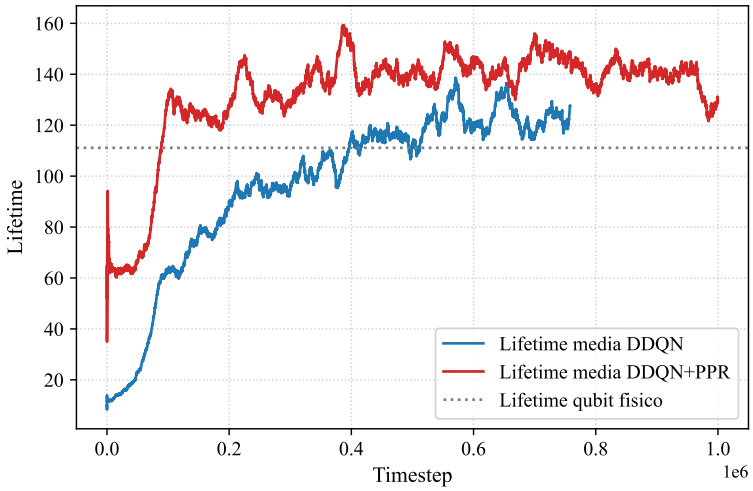


Figura 3.9: Grafico *lifetime* media per *timestep*, ambiente $p_{\text{err}} = 0.009$.

In quest'ultimo grafico il vantaggio dovuto al meccanismo di PPR è netto: l'agente raggiunge in fretta un alto livello di *lifetime* media, mantenendo prestazioni migliori rispetto al DDQN tradizionale per tutta la durata dell'addestramento.

In conclusione, l'analisi dei grafici mostra chiaramente le potenzialità di un approccio di *continual learning* come PPR nel fornire supporto alla politica in fase di addestramento. I risultati ottenuti evidenziano inoltre come il vantaggio ottenibile sia strettamente correlato alla qualità delle esperienze pregresse, sia in termini di ricompensa attesa che di capacità di generalizzazione del problema. Infine, si segnala che, come prima sperimentazione, è stato adottato un modello relativamente semplice per la gestione dinamica degli iperparametri. È probabile che l'utilizzo di un modello più articolato, talvolta sviluppato *ad hoc* per ciascun problema di decodifica, possa generare politiche più efficienti e garantire un vantaggio maggiore in termini di risparmio di risorse e tempo di addestramento.

[8] Un numero maggiore di politiche comporta l'esigenza di maggior esplorazione della libreria, con conseguente oscillazione delle prestazioni.

3.3.2 Analisi della lifetime logica

Per ciascuna delle nove politiche ottenute — cinque tramite DDQN tradizionale e quattro tramite DDQN + PPR — sono stati condotti test nello stesso ambiente utilizzato durante l'addestramento, mediante l'apposita funzione. Questo ha permesso di analizzare il comportamento del decoder sviluppato e di valutarne le prestazioni effettive in termini di *logical lifetime*, confrontando i risultati con la vita media di un singolo qubit soggetto agli stessi modelli di rumore.

Inoltre, l'analisi delle metriche acquisite durante ciascun test ha permesso di studiare la distribuzione di *lifetime* per un qubit logico protetto da un *Surface Code* che si avvale di un decoder come quello sviluppato. Per ogni test, tale distribuzione è risultata conforme al modello *esponenziale*, o per la precisione *geometrico* se si considera il tempo discreto dettato dai cicli di correzione. Questi modelli infatti descrivono, rispettivamente in maniera continua e discreta, la vita media di un singolo qubit in presenza di errori come quelli simulati, e cioè basati su eventi poissoniani (indipendenti e rari) caratterizzati da un tasso di probabilità fisso. La distribuzione ottenuta costituisce quindi un risultato consistente con le aspettative se si considera che un errore logico, ossia una condizione terminale, è determinato dall'accumularsi di singoli eventi di errore fisico, e pertanto ne eredita la caratteristica di evento poissoniano. Il codice di correzione sviluppato ha permesso dunque di ridurre il tasso di fallimento logico — ovvero aumentare il valore di vita media previsto — ma non ha alterato la forma fondamentale della distribuzione di *lifetime*.
In fig. 3.10 è riportato, a titolo esemplificativo, l'istogramma della distribuzione della *logical lifetime* ottenuta nel test della politica PPR addestrata per l'ambiente con $p_{\text{err}} = 0.009$.

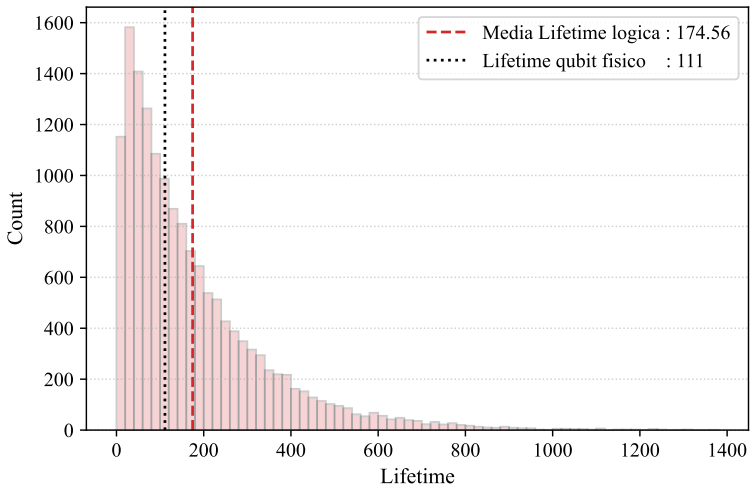


Figura 3.10: Distribuzione della *logical lifetime* per la politica PPR addestrata sull'ambiente $p_{\text{err}} = 0.009$.

I test sono stati eseguiti per ciascuna politica su un numero di episodi sufficiente a garantire una stima statisticamente affidabile dei risultati. I valori medi di *logical lifetime* ottenuti per ciascun ambiente simulato — rispettivamente con addestramento DDQN e DDQN + PPR — sono riportati in tab. 3.2, insieme alla previsione teorica

della vita media di un singolo qubit fisico, approssimata come

$$\frac{1}{p_{\text{err}}}$$

Ambiente $\mathbf{p_{err}}$	**Qubit fisico** **lifetime**	**DDQN** **logical lifetime**	**DDQN + PPR** **logical lifetime**
0.001	1000	16723	—
0.003	333	1794	2066
0.005	200	653	611
0.007	142	340	321
0.009	111	163	175

Tabella 3.2: Confronto dei valori di *lifetime* media per ambiente.

Dal punto di vista delle prestazioni in fase di test, i modelli ottenuti con i due approcci di addestramento risultano sostanzialmente equivalenti, senza che emerga una differenza significativa tra DDQN tradizionale e DDQN + PPR. Tuttavia non è da escludere l'ipotesi che, ottimizzando gli iperparametri e quindi soprattutto aumentando la qualità delle esperienze accumulate nella libreria, il modello DDQN + PPR possa raggiungere risultati superiori rispetto al metodo tradizionale.
Inoltre si osserva come l'incremento di *logical lifetime* rispetto alla previsione per un singolo qubit fisico diminuisca progressivamente all'aumentare della difficoltà dell'ambiente, evidenziando la crescente complessità del problema di decodifica, soprattutto a causa della presenza di sindromi sempre più ambigue e affette da errori di misura. A questo proposito, il grafico in fig. 3.11 mostra l'andamento della *logical lifetime* media in funzione del tasso di errore p_{err} per i due modelli di addestramento, evidenziando questo limite pratico alle prestazioni del decoder.

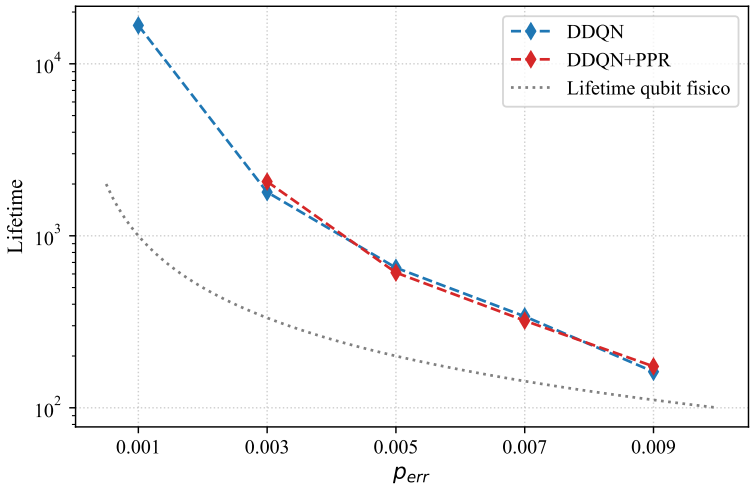


Figura 3.11: Andamento della *logical lifetime* media all'aumentare del tasso di errore p_{err}.

In conclusione, i risultati ottenuti costituiscono una prima stima relativa al potenziale che un decoder basato su *Deep Reinforcement Learning* come quello sviluppato

può offrire. Si osserva come, pur rimanendo lontani dal massimo potenziale teorico di protezione che un simile *Surface Code* è in grado di fornire[9], questo tipo di architettura abbia comunque garantito un aumento significativo della vita media rispetto a un singolo qubit fisico per tutti gli ambienti simulati. È inoltre importante sottolineare che l'analisi considera l'eventualità di errori di misura nella sindrome, fornendo risultati e stime concrete e realistiche, più vicine a scenari sperimentali rispetto a molti studi teorici basati su algoritmi classici.

[9]In presenza di un decoder ideale, il tasso di errore logico scala secondo valori $O(p_{\text{err}}^{d})$ con d profondità del codice.

Conclusioni

La *Quantum Error Correction* (QEC) rappresenta un elemento fondamentale per la futura realizzazione di computer quantistici su larga scala in grado di operare in regime *fault-tolerant*, requisito fondamentale per ottenere la condizione di vantaggio quantistico. Tra i diversi approcci proposti, il *Surface Code* si distingue come un candidato promettente per l'implementazione di queste tecniche in contesti reali, grazie alla sua buona scalabilità — che ciò nonostante rimane un elemento critico — e alla compatibilità con le architetture hardware attualmente più mature. In questo lavoro è stato sviluppato e analizzato un decoder basato sul paradigma di *Reinforcement Learning* (RL), in particolare sull'algoritmo *Double Deep Q-Network* (DDQN), progettato per la decodifica di sindromi di *Surface Code* affette da errori di misura. Parallelamente è stato studiato come, nel contesto di ambienti simili, l'utilizzo di modelli di *continual learning* come *Probabilistic Policy Reuse* (PPR) possa migliorare l'adattamento dell'agente al problema, riducendo significativamente i costi di addestramento in termini di risorse e tempo.

I risultati sperimentali mostrano le potenzialità che un decoder di questo tipo può offrire. In primo luogo, il confronto tra i grafici relativi alla ricompensa media — in termini di *lifetime* del codice — in funzione del numero di interazioni agente-ambiente (*timestep*) ha permesso di dimostrare la miglior velocità di adattamento del modello DDQN + PPR. Gli stessi livelli prestazionali sono infatti stati raggiunti con un numero di *timestep* inferiore del 20 – 30% rispetto al DDQN tradizionale, dimostrando l'efficacia di un approccio di *continual learning* applicato a questo contesto. Inoltre, l'analisi delle scelte di riuso delle *policy* della libreria ha evidenziato come questo risultato possa essere influenzato particolarmente dalla qualità e generalizzabilità delle esperienze pregresse, in termini di efficienza delle strategie apprese precedentemente. In fase di test, per ogni ambiente simulato, l'architettura sviluppata si è dimostrata in grado di aumentare significativamente la vita media rispetto a quella di un singolo qubit fisico, raggiungendo buoni livelli di *logical lifetime* del codice. Ad esempio, considerando valori teorici di *lifetime* pari a circa 1000 cicli di sindrome per l'ambiente con probabilità $p_{\text{err}} = 0.001$ e 111 per $p_{\text{err}} = 0.009$, il decoder ha raggiunto valori medi ottimali di *logical lifetime* pari rispettivamente a 16723 e 175. Questa riduzione progressiva del miglioramento ottenuto all'aumentare del tasso di errore è coerente con la crescente complessità del problema di decodifica e rappresenta uno dei principali limiti di questo modello. Tali risultati sono lontani dalla potenziale capacità teorica massima di un *Surface Code* dotato di un decoder perfetto, tuttavia rappresentano stime concrete e realistiche: l'analisi infatti tiene conto dell'eventualità di errori di misura nella sindrome, condizione che riflette più fedelmente i possibili scenari sperimentali.

Nel complesso, i dati ottenuti rappresentano un primo risultato promettente, che conferma la validità dei metodi proposti e che potrà essere ottimizzato sotto molteplici aspetti. In primo luogo, si osserva che le simulazioni sono state svolte su un hardware

classico in cui l'algoritmo di *Machine Learning* operava senza l'ausilio di infrastrutture di *High Performance Computing* (HPC), il che ha imposto dei limiti alla potenza di calcolo e delle restrizioni nella sperimentazione dei parametri. Il modello può essere infatti ottimizzato nel valore e nella combinazione relativa dei parametri di apprendimento (*learning rate*, *batch size*, *buffer size*, ecc.), al fine di garantire una miglior convergenza delle funzioni $Q(s,a)$ delle reti neurali e consolidare al meglio le strategie imparate. Questo porterebbe a politiche più robuste, con un possibile incremento dei valori medi prestazionali sia per l'algoritmo DDQN che per DDQN + PPR.
Inoltre, si evidenzia che per un primo approccio si è scelto di mantenere una gestione dinamica semplificata degli iperparametri ϵ e ψ, implementando un decadimento lineare con pendenze fisse, ma che il significato di questi valori è ben più profondo. In un contesto reale di applicazione, ψ bilancia il riuso delle politiche preaddestrate, ovvero determina il meccanismo di adattamento ai cambiamenti nel rumore del dispositivo. Il suo valore andrebbe dunque scelto in base alla somiglianza tra il regime di rumore da mitigare e quelli affrontati in precedenza, tenendo conto anche delle condizioni fisico-ambientali a cui può essere sottoposto un qubit (radiazione esterna, variazioni di temperatura, campi elettromagnetici, ecc.) o del deterioramento della tecnologia hardware con cui è realizzato. Diversamente, ϵ regola il classico bilanciamento RL tra esplorazione e sfruttamento all'interno della *policy* in addestramento. Il suo valore andrebbe determinato in funzione della quantità di esplorazione necessaria — anche in relazione alla difficoltà del problema — per l'apprendimento di una politica ottimale, evitando di cadere in soluzioni subottimali e garantendo una minima esplorazione residua che renda le strategie apprese sufficientemente robuste anche a piccole fluttuazioni del modello di errore, che è stato considerato fisso nelle simulazioni di questo progetto. A questo proposito, un possibile sviluppo futuro potrebbe riguardare la progettazione di una funzione di test che implementi un meccanismo analogo al PPR, ma orientato non alla fase di addestramento, bensì alla selezione dinamica della politica più adatta durante l'applicazione pratica, una sorta di *Probabilistic Policy Selection*. Un approccio di questo tipo potrebbe risultare particolarmente utile in ambienti in cui il modello di errore evolve nel tempo, permettendo di individuare in modo adattivo la *policy* più appropriata tra quelle disponibili. Questa selezione dinamica potrebbe avvenire in funzione della stima del regime di rumore, la quale potrebbe essere ottenuta attraverso metodi probabilistici o mediante l'impiego di reti neurali dedicate alla classificazione in tempo reale del modello di errore corrente.

In prospettiva, l'approccio sviluppato potrebbe beneficiare dell'impiego di risorse HPC a supporto dei metodi di *Machine Learning* presentati e, successivamente, dell'utilizzo di qubit superconduttivi reali: dapprima per condurre studi e simulazioni più coerenti con i modelli di rumore effettivi a cui tali dispositivi sono soggetti, e in particolare per approfondire la definizione di strategie ottimali di gestione e adattamento degli iperparametri; in un secondo momento, per una possibile implementazione sperimentale concreta, oltre la simulazione.

Appendice A

Algoritmi di Reinforcement Learning

In questa sezione vengono inseriti estratti di pseudo-codice relativi agli algoritmi di RL presentati in questo elaborato. Mentre *Q-learning* è inserito per completezza espositiva, gli algoritmi *(Double) Deep Q-Network* e *DDQN con Probabilistic Policy Reuse* costituiscono un riferimento sintetico alle procedure effettivamente implementate per l'addestramento dell'agente.

Algoritmo 1: Q-Learning tabellare

given: Un ambiente $\mathcal{E}$
given: Tasso di Apprendimento α, Fattore di Sconto γ
given: Politica di esplorazione ϵ (con annealing)
given: Massimo numero di passi eseguibili K (*maxtimesteps*)
init: La tabella $Q(s, a)$ con valori arbitrari (es. zero)
init: stato inziale s
(1) **for** *step* $= 1, 2, \ldots, K$ **do**
(2) Esegui reset dell'ambiente $\mathcal{E}$
(3) Osserva e inizializza lo stato iniziale s
(4) **while** *lo stato s non è terminale* **do**
Selezione Azione con ϵ-Greedy
(5) **if** *rand()* $< \epsilon$ **then**
(6) Seleziona casualmente un'azione a
(7) **else**
(8) Seleziona l'azione a con il valore Q più alto:
$a \leftarrow \text{argmax}_{a'} Q(s, a')$
(9) Esegui l'azione a sull'ambiente $\mathcal{E}$
(10) Osserva il nuovo stato s' e la ricompensa r
Aggiorna il valore della tabella
(11) $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha\,(r + \gamma \max_{a'} Q(s', a'))$
(12) $s \leftarrow s'$
(13) Aggiorna ϵ (Decadimento dell'esplorazione)
return: La tabella Q aggiornata che approssima Q^*

Algoritmo 2: (Double) Deep Q-Network

given: Ambiente $\mathcal{E}$
given: Rete Policy Q con parametri θ
given: Rete Target $\hat{Q}$ con parametri θ^-
given: Memoria Replay Buffer $\mathcal{D}$
given: Tasso di Apprendimento η, Fattore di Sconto γ
given: Massimo numero di passi eseguibili K (*max_timestep*)
given: Frequenza di aggiornamento target f_{target}
given: Politica di esplorazione ϵ (con annealing)
init: Parametri θ arbitrari
init: $\theta^- \leftarrow \theta$ (copia i parametri)
(1) **for** *step* $= 1, 2, \ldots, K$ **do**
(2) Esegui reset dell'ambiente $\mathcal{E}$
(3) Osserva e inizializza lo stato iniziale s
(4) **while** *lo stato s non è terminale* **do**
Selezione Azione con ϵ-Greedy
(5) **if** *rand*() $< \epsilon$ **then**
(6) Seleziona un'azione a casuale
(7) **else**
(8) Seleziona l'azione a con il valore Q più alto:
$a \leftarrow \text{argmax}_{a'} Q(s, a'; \theta)$
(9) Esegui l'azione a nell'ambiente $\mathcal{E}$
(10) Osserva ricompensa r e stato successivo s'
(11) Memorizza la transizione (s, a, r, s') in $\mathcal{D}$
Apprendimento della rete
(12) **if** *dimensione*($\mathcal{D}$) *sufficiente* **then**
(13) Ottimizza la rete policy Q *# questo processo sfrutta Q, $\hat{Q}$, $\mathcal{D}$, γ, η*
Aggiornamento Rete Target
(14) **if** *step* $\bmod f_{target} = 0$ **then**
(15) $\theta^- \leftarrow \theta$
(16) $s \leftarrow s'$
(17) Aggiorna ϵ (Decadimento dell'esplorazione)
return: Parametri finali θ della rete Q

Il processo di ottimizzazione è quello descritto in sezione 2.3.2 e comprende le fasi di campionamento dal *Replay Buffer* $\mathcal{D}$, calcolo della loss, *backpropagation* e aggiornamento dei parametri. Si noti che la principale differenza tra gli algoritmi DQN e DDQN risiede nel calcolo della *loss*; pertanto, questo pseudocodice rappresenta in modo esemplificativo anche l'estensione al *Double Deep Q-Network*.

Algoritmo 3: (Double) Deep Q-Network con Probabilistic Policy Reuse (PPR)

given: Ambiente $\mathcal{E}$
given: Rete Policy $Q(\theta)$, Rete Target $\hat{Q}(\theta^-)$, Libreria di policy $L = \{\pi_1, \cdots, \pi_n\}$
given: Memoria Replay Buffer $\mathcal{D}$, Tasso di Apprendimento η, Fattore di Sconto γ
given: Parametro di temperatura τ e incremento $\delta\tau$
given: Massimo numero di passi eseguibili K (*max_timestep*)
given: Frequenza di aggiornamento target f_{target}
given: Politica di esplorazione ϵ (con annealing)
given: Politica di riuso ψ (con annealing)
init: Vettore ricompense, $W_i = 0 \; \forall i \in [0, n]$ *# i=0 corrisponde a Q*
init: Vettore contatori utilizzo delle policy π_i, $U_i = 0 \; \forall i \in [0, n]$
(1) **for** *step* $= 1, 2, \ldots, K$ **do**
(2) Esegui reset dell'ambiente $\mathcal{E}$, dello stato s e della ricompensa R dell'episodio
Selezione della policy valida per l'episodio
(3) Calcola vettore probabilità softmax $p(W, \tau)$
(4) Estrai un indice k dalla distribuzione p
(5) **while** *lo stato s non è terminale* **do**
(6) **if** *k==0* **then**
la policy estratta è Q
(7) Seleziona l'azione a tramite ϵ-*Greedy* sulla policy Q
(8) **else**
Seleziona Azione tramite π-reuse
(9) **if** *rand()* $< \psi$ **then**
(10) Seleziona l'azione a ottimale per la policy π_k
(11) **else**
(12) Seleziona l'azione a tramite ϵ-*Greedy* sulla policy Q
(13) Esegui l'azione a nell'ambiente $\mathcal{E}$
(14) Osserva ricompensa r e stato successivo s'
(15) Memorizza la transizione (s, a, r, s') in $\mathcal{D}$
(16) **if** *dimensione*$(\mathcal{D})$ *sufficiente* **then**
(17) Ottimizza la rete policy Q *# questo processo sfrutta Q, $\hat{Q}$, $\mathcal{D}$, γ, η*
(18) **if** *step* $\bmod f_{target} = 0$ **then**
(19) $\theta^- \leftarrow \theta$
(20) $s \leftarrow s'$
(21) Aggiorna ϵ (Decadimento dell'esplorazione)
(22) Aggiorna ψ (Decadimento del riuso)
(23) Aggiorna la ricompensa cumulativa dell'episodio R
(24) $W_k \leftarrow \frac{W_k U_k + R}{U_k + 1}$
(25) $U_k \leftarrow U_k + 1$
(26) $\tau \leftarrow \tau + \delta\tau$
return: Parametri finali θ della rete Q

Appendice B

Implementazione e frammenti di codice

L'obiettivo di questa sezione è fornire alcuni dettagli tecnici ed estratti di codice relativi al progetto. Per ragioni di spazio, non è possibile riportare integralmente le funzioni sviluppate: per una visione completa si rimanda quindi al repository del progetto [38].

Architettura del decoder

Di seguito vengono riportati alcuni estratti di codice relativi alle istanze utilizzate, in particolare relativi alla memoria buffer, alle reti neurali e all'agente.

```python
class ReplayBuffer:
    """Memoria per le transizioni (s,a,r,s',terminated) con una capacità limitata"""
    def __init__(self, capacity):
        self.capacity = capacity
        self.buffer = []
        self.position = 0

    def push(self, state, action, reward, next_state, terminated):
        """salva una transizione nella memoria"""
        # se il buffer non è pieno viene aggiunto
        if len(self.buffer) < self.capacity:
            self.buffer.append(None)
        # altrimenti sovrascrivo la posizione più vecchia
        self.buffer[self.position] = (state, action, reward, next_state, terminated)
        self.position = (self.position + 1) % self.capacity

    def sample(self, batch_size):
        """campiona un batch casuale di transizioni e le converte in tensori pytorch"
    ""
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, terminateds = zip(*batch)
        # conversione in tensori pytorch
        state_tensor = torch.from_numpy(np.array(states)).float()
        action_tensor = torch.tensor(actions, dtype=torch.long).unsqueeze(1)
        reward_tensor = torch.tensor(rewards, dtype=torch.float32).unsqueeze(1)
        next_state_tensor = torch.from_numpy(np.array(next_states)).float()
        terminated_tensor = torch.tensor(terminateds, dtype=torch.float32).unsqueeze
    (1)

        return state_tensor, action_tensor, reward_tensor, next_state_tensor,
    terminated_tensor

    def __len__(self):
        return len(self.buffer)
```

Codice B.1: Implementazione della Memoria di Replay (ReplayBuffer)

```python
class ConvNetRL(nn.Module):
    """classe che definisce la rete neurale convoluzionale per l'agente DDQN"""
    def __init__(self, cc_layers, ff_layers, input_shape, num_actions, dueling=True):
        super(ConvNetRL, self).__init__()

        self.conv_layers = nn.ModuleList()
        in_channels = input_shape[0]
        for filt, ksize, stride in cc_layers:
            self.conv_layers.append(nn.Conv2d(in_channels, filt, kernel_size=ksize,
    stride=stride))
            in_channels = filt

        self.flatten = nn.Flatten()

        # calcolo della dimensione di input fully connected generato dalla cc
    precedente
        with torch.no_grad():
            dummy_input = torch.zeros(1, *input_shape)
            x = dummy_input
            for conv in self.conv_layers:
                x = F.relu(conv(x))
            fc_input_dim = x.view(1, -1).shape[1]

        # fully connected + dropout
        self.fc_layers = nn.ModuleList()
        self.dropouts = nn.ModuleList()
        in_dim = fc_input_dim
        for neurons, dropout_rate in ff_layers:
            self.fc_layers.append(nn.Linear(in_dim, neurons))
            self.dropouts.append(nn.Dropout(dropout_rate))
            in_dim = neurons

        # procedura dueling
        self.dueling = dueling

        if self.dueling:
            self.advantage_layer = nn.Linear(in_dim, num_actions)
            self.value_layer = nn.Linear(in_dim, 1)
        else:
            self.out = nn.Linear(in_dim, num_actions)

    def forward(self, x):
        # strati convoluzionali e flatten
        for conv in self.conv_layers:
            x = F.relu(conv(x))
        x = self.flatten(x)

        # strati fully connected
        for fc, drop in zip(self.fc_layers, self.dropouts):
            x = F.relu(fc(x))
            x = drop(x)

        if self.dueling:
            value = self.value_layer(x)         # V(s)
            advantage = self.advantage_layer(x) # A(s,a)

            # ricombinazione dueling: Q = V(s) + [A(s,a) - mean(A(s,a))]
            q_values = value + (advantage - advantage.mean(dim=1, keepdim=True))
            return q_values
        else:
            return self.out(x)
```

Codice B.2: Implementazione della rete neurale convoluzionale

```python
class PyTorchDDQNAgent:
    def __init__(self, policy_net, target_net, optimizer, memory, all_configs, device
    ):
        self.policy_net = policy_net
        self.target_net = target_net
        self.device = device
        self.optimizer = optimizer
        self.memory = memory
        self.criterion = nn.SmoothL1Loss()

        self.gamma = all_configs["gamma"]
        self.batch_size = all_configs["batch_size"]
        self.learning_starts = all_configs["learning_starts"]
        self.target_update_freq = all_configs["target_network_update_freq"]

    def learn(self):
        """
        esegue un singolo passo di ottimizzazione DDQN.
        """
        if len(self.memory) < self.learning_starts or len(self.memory) < self.
    batch_size:
            return None # ancora in fase di riscaldamento o buffer non pieno

        # campionamento dal buffer di replay
        state_batch, action_batch, reward_batch, next_state_batch, terminated_batch =
     self.memory.sample(self.batch_size)

        state_batch = state_batch.to(self.device)
        action_batch = action_batch.to(self.device)
        reward_batch = reward_batch.to(self.device)
        next_state_batch = next_state_batch.to(self.device)
        terminated_batch = terminated_batch.to(self.device)

        # calcolo Q(s, a) corrente (valori stimati)
        current_q_values = self.policy_net(state_batch).gather(1, action_batch)

        # calcolo di max(Q_target(s', a')) (valori target)
        with torch.no_grad():
            actions_from_policy_net = self.policy_net(next_state_batch).argmax(1).
    unsqueeze(1)
            next_q_values = self.target_net(next_state_batch).gather(1,
    actions_from_policy_net)

            # calcolo del target Q-value
            target_q_values = reward_batch + self.gamma * next_q_values * (1 -
    terminated_batch)

        # calcolo della loss
        loss = self.criterion(current_q_values, target_q_values)

        # backward pass e aggiornamento
        self.optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.policy_net.parameters(), 1.0)
        self.optimizer.step()

        return loss.item()

    def update_target_net(self):
        """aggiorna i pesi della rete target."""
        self.target_net.load_state_dict(self.policy_net.state_dict())
```

Codice B.3: Implementazione della classe relativa all'agente DDQN

Funzioni e istanze relative alla gestione degli iperparametri

In questa sezione sono riportate le istanze e le funzioni relative alla gestione dinamica degli iperparametri ϵ e ψ durante il training. In particolare, entrambi i parametri sono soggetti a un decadimento lineare, con pendenze differenti: tale scelta consente una fase iniziale caratterizzata da una più ampia esplorazione e riuso delle azioni, assicurando in seguito la stabilizzazione della politica man mano che i valori dei parametri si riducono. Nel caso di ϵ, l'aggiornamento del valore è direttamente gestito dallo stesso metodo chiamato per la scelta di un'azione secondo ϵ-*Greedy*.

```python
class LinearAnnealedEpsGreedyPolicy:
    """gestisce la logica di esplorazione epsilon-greedy con decay lineare"""
    def __init__(self, value_max, value_min, nb_steps, masked_greedy):
        self.value_max = value_max      # max_eps (1.0)
        self.value_min = value_min      # final_eps (0.02)
        self.nb_steps = nb_steps        # exploration_fraction (75000)
        self.masked_greedy = masked_greedy
        self.current_epsilon = value_max
        self.slope = (value_min - value_max) / nb_steps

    def update_epsilon(self, step_count):
        """aggiorna epsilon linearmente"""
        if step_count < self.nb_steps:
            self.current_epsilon = self.value_max + self.slope * step_count
        else:
            self.current_epsilon = self.value_min

    def select_action(self, q_values, legal_actions, is_training, step_count):
        """sceglie un'azione in base alla policy corrente"""

        if is_training:
            self.update_epsilon(step_count)
            epsilon = self.current_epsilon
        else:
            # GreedyPolicy
            epsilon = 0.0

        num_actions = q_values.size(-1)

        if is_training and np.random.rand() < epsilon:
            # esplorazione: selezione casuale tra azioni legali
            return random.choice(list(legal_actions))
        else:
            # sfruttamento: selezione greedy
            if self.masked_greedy or not is_training:
                # applica il masking se richiesto o per il test
                mask = torch.ones(num_actions, dtype=torch.bool).to(q_values.device)
                illegal_actions = [a for a in range(num_actions) if a not in
    legal_actions]
                mask[illegal_actions] = False

                masked_q_values = q_values.masked_fill(~mask, -1e9)
                return masked_q_values.argmax().item()
            else:
                # senza masking
                return q_values.argmax().item()
```

Codice B.4: Implementazione della classe relativa alla policy di esplorazione e gestione dell'annealing lineare di ϵ

Si noti che — sebbene non sia stato utilizzato in fase di allenamento — la funzione *select_action()* permette di escludere le azioni non consentite anche nel caso di selezione

Greedy durante l'addestramento. Diverso il caso del test, in cui tale maschera viene implementata di default.
Nel caso di ψ invece l'aggiornamento viene implementato manualmente nella funzione di training con PPR attraverso la seguente funzione, basata sui parametri forniti nel dizionario *all_configs* contenente tutte le impostazioni dell'addestramento.

```
def linear_update_psi(step_count, all_configs):
    """aggiorna il valore di psi (parametro che gestisce la dinamica di reuse) con
     decadimento lineare"""

    slope = (all_configs["final_psi"] - all_configs["max_psi"]) / all_configs["
     ppr_fraction"]
    if step_count < all_configs["ppr_fraction"]:
        current_psi = all_configs["max_psi"] + slope * step_count
    else :
        current_psi = all_configs["final_psi"]
    return current_psi
```

Codice B.5: Gestione di annealing lineare di ψ

Estratti di codice delle funzioni di training

A causa della lunghezza complessiva del codice relativo alle funzioni di training, non è possibile riportarlo integralmente in questo elaborato. In questa sezione vengono quindi presentati alcuni frammenti rappresentativi, focalizzati sui passaggi più significativi dei modelli sviluppati e, in particolare, sui punti in cui essi differiscono. Per una visione completa della struttura e della logica implementativa si rimanda agli algoritmi illustrati in Appendice A e al repository del progetto [38].

Strategia di selezione dell'azione

I seguenti frammenti di codice mostrano la differenza nelle modalità di selezione dell'azione per ciascun episodio rispettivamente delle funzioni DDQN e DDQN + PPR. Si noti che entrambi gli estratti fanno riferimento per la scelta delle azioni ϵ-*Greedy* al metodo *select_action()* consultabile nel codice B.4.

```
# ... all'inizio di ciascun interazione agente-ambiente ...
    # selezione dell'azione (epsilon-greedy)
    state_tensor = torch.from_numpy(current_state).float().to(device).unsqueeze(0)
    q_values = policy_net(state_tensor).squeeze(0)
    action = policy.select_action(
        q_values=q_values,
        legal_actions=env.legal_actions,
        is_training=True,
        step_count=step_count
    )
# ...
```

Codice B.6: Estratto per la scelta dell'azione nella funzione di training con DDQN

```
# ... all'inizio di ciascun interazione agente-ambiente ...
    state_tensor = torch.from_numpy(current_state).float().to(device).unsqueeze(0)
    action = None
    if k > 0:
        if np.random.rand() < current_psi:
            # seleziono l'azione greedy dalla policy Li
            with torch.no_grad():
                q_values_Li = Li(state_tensor).squeeze(0)
            action = q_values_Li.argmax().item()
            reuse_action_count += 1

            # aggiorno psi ed eps per questo step in cui avviene reuse
            current_psi = linear_update_psi(step_count, all_configs)
            policy.update_epsilon(step_count)
    # se non è ancora stata scelta un'azione o se k==0 (policy corrente)
    if action is None:
        # seleziono l'azione secondo eps-greedy sulla policy corrente
        q_values = policy_net(state_tensor).squeeze(0)
        action = policy.select_action(
            q_values=q_values,
            legal_actions=env.legal_actions,
            is_training=True,
            step_count=step_count
            )
        # aggiorno solo psi: eps è aggiornato in "select_action"
        current_psi = linear_update_psi(step_count, all_configs)
# ...
```

Codice B.7: Estratto di codice per la scelta dell'azione nella funzione di training con PPR

Dinamiche di fine episodio

La gestione di fine episodio implementata nelle funzioni di training sviluppate include una logica completa per il calcolo delle metriche, la registrazione dei log, il monitoraggio a terminale, la creazione di checkpoint e il meccanismo di *early stopping*. Il calcolo e la registrazione delle metriche, così come la stampa periodica a terminale, sono realizzati in modo intuitivo mediante funzioni della libreria *NumPy* e tramite funzioni di supporto (*helper*) progettate per creare in modo efficiente la struttura di directory dedicata alla simulazione e i relativi file aggiornati a ogni episodio. Tali funzioni non sono riportate in questo appendice per ragioni di spazio. Si noti che le metriche e i parametri registrati risultano sostanzialmente identici nelle due versioni delle funzioni di training, fatta eccezione per alcune variabili specifiche della versione estesa DDQN + PPR.
Di seguito è riportato un frammento di codice tratto dalle implementazioni reali, relativo alla gestione del salvataggio dei checkpoint e all'attivazione della logica di *early stopping*.

```
# ... durante la dinamica di fine episodio sancita da done=True ...

    # early stopping e checkpointing
    if current_lifetime_avg > best_avg_lifetime and len(all_lifetimes) >= all_configs
    ["rolling_average_length"]:
        # checkpoint
        torch.save(policy_net.state_dict(), best_model_path)
        if best_avg_lifetime != -float('inf'):
            print(f"Nuovo Best Model! Avg Lifetime: {current_lifetime_avg:.2f}")

        best_avg_lifetime = current_lifetime_avg
        patience_counter = 0

    elif best_avg_lifetime >= all_configs['success_threshold']:
        patience_counter += 1
        # early stopping
        if patience_counter >= all_configs["stopping_patience"]:
            print(f"--- Stopping early. Best Avg Lifetime: {best_avg_lifetime:.2f}")
            training_completed = False
            break
# ...
```

Codice B.8: Estratto di codice relativo alla gestione di checkpoint e early-stopping

Parametri di addestramento

Di seguito è riportata una tabella con tutti i parametri — e i rispettivi valori — implementati per l'addestramento. Nel codice sono forniti alle varie istanze/funzioni attraverso un dizionario generale denominato *all_configs*. Si noti che i parametri con valore "$\cdots$" sono tali perché dipendenti dall'ambiente simulato e pertanto variabili nel corso dell'esperienza.

Parametro	**Valore**	**Descrizione**
d	5	Dimensione del *Surface Code*
volume_depth	5	Profondità del *syndrome volume*
p_{phys}	$\cdots$	Probabilità errore singolo qubit fisico
p_{meas}	$\cdots$	Probabilità errore misura
max_timesteps	1 000 000	Numero massimo totale di step
rolling_avg_len	500	Finestra per la media mobile delle metriche
learning_starts	1000	Numero step per *warm up*
train_freq	1	Freq. (step) di aggiornamento dei pesi
print_freq	100	Freq. (episodi) di stampa dei log a terminale
success_threshold	$\cdots$	Soglia *lifetime* minima per *early-stopping*
stopping_patience	1000	Tolleranza (episodi) prima di early stopping
batch_size	64	Dimensione del batch campionato
buffer_size	50 000	Dimensione memoria Replay Buffer
γ	0.99	Fattore di sconto
optimizer	Adam	Ottimizzatore utilizzato
learning_rate	1×10^{-4}	Tasso di apprendimento (Adam)
target_net_update_freq	5000	Freq. (step) di aggiornamento rete target
max_eps / final_eps	1.0 / 0.02	Intervallo di esplorazione ϵ-greedy
exploration_fraction	75 000	Fase di riduzione di ϵ
max_psi / final_psi	0.85 / 0.02	Intervallo di probabilità di riuso (PPR)
ppr_fraction	175 000	Fase di riduzione di ψ
τ	0	Valore di temperatura iniziale per softmax
$\delta\tau$	0.0001	Incremento per il parametro τ

Tabella B.1: Elenco completo dei parametri di configurazione utilizzati nel progetto.

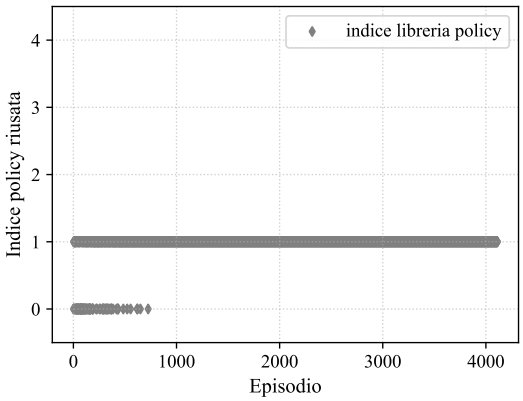
Appendice C

Grafici ulteriori

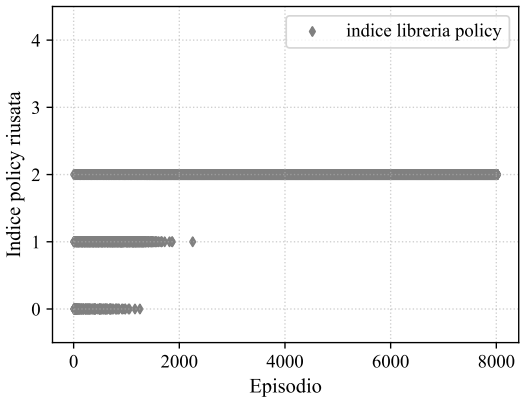
In questo Appendice vengono inseriti ulteriori grafici a supporto dei risultati sperimentali.

Riuso delle politiche nel meccanismo di PPR

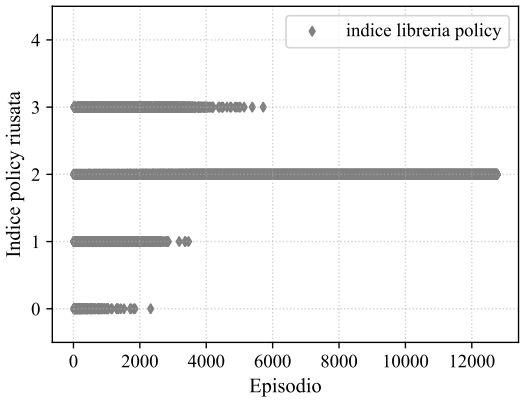
Le immagini seguenti mostrano la selezione della politica utilizzata per la strategia di π-*reuse* per ciascun episodio di addestramento DDQN + PPR. In particolare, i grafici tracciano l'indice delle politiche estratte tramite Softmax dalle librerie correnti per ciascuna simulazione considerata, in funzione del numero di episodio. Per ogni grafico, l'indice $k = 0$ si riferisce alla politica che l'agente sta attivamente addestrando, mentre gli indici $k > 0$ rappresentano le politiche pre-addestrate, ordinate in base al tasso di errore p_{err} crescente dell'ambiente di origine (ad esempio, per l'ambiente $p_{\text{err}} = 0.007$, l'indice 0 è relativo alla strategia in sviluppo, mentre 1, 2 e 3 sono rispettivamente le politiche generate con DDQN sugli ambienti con $p_{\text{err}} = 0.001,\ 0.003,\ 0.005$).



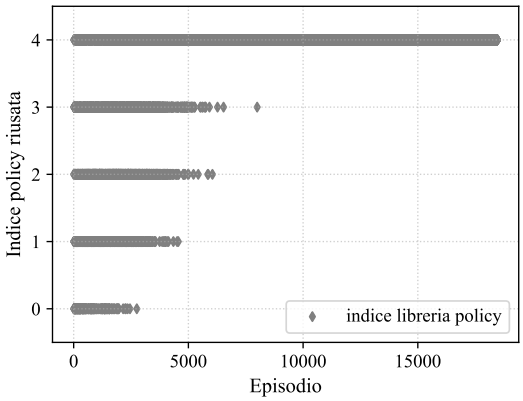
(a): PPR su ambiente $p_{\text{err}} = 0.003$.



(b): PPR su ambiente $p_{\text{err}} = 0.005$.



(c): PPR su ambiente $p_{\text{err}} = 0.007$.



(d): PPR su ambiente $p_{\text{err}} = 0.009$.

Figura C.1: Grafici relativi alle distribuzioni delle scelte di *policy reuse* durante l'addestramento con DDQN + PPR per ciascun ambiente.

Ciascun addestramento è caratterizzato da una fase preliminare di esplorazione delle politiche contenute nella libreria, in accordo con una distribuzione di probabilità inizialmente uniforme. Con il passare degli episodi, la distribuzione Softmax tende a favorire le strategie che hanno ricevuto maggior ricompensa, e cioè solitamente quelle originate in ambienti più affini al problema di decodifica in analisi. Il caso di $p_{\mathbf{err}} = 0.007$ è anomalo in questo senso: la distribuzione tende a favorire il riutilizzo della politica originata in $p_{\mathbf{err}} = 0.003$ invece di $p_{\mathbf{err}} = 0.005$, probabilmente a causa di una minore generalizzabilità o di una qualità inferiore di quest'ultima.

Bibliografia

[1] Richard P. Feynman. «Simulating Physics with Computers». In: *International Journal of Theoretical Physics* 21.6-7 (1982), pp. 467–488. DOI: 10.1007/BF02650179.

[2] David Deutsch. «Quantum Theory, the Church–Turing Principle and the Universal Quantum Computer». In: *Proceedings of the Royal Society A: Mathematical and Physical Sciences* 400.1818 (1985), pp. 97–117. DOI: 10.1098/rspa.1985.0070.

[3] P.W. Shor. «Algorithms for quantum computation: discrete logarithms and factoring». In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.

[4] Lov K. Grover. *A fast quantum mechanical algorithm for database search*. 1996. arXiv: quant-ph/9605043 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9605043.

[5] John Preskill. «Quantum Computing in the NISQ era and beyond». In: *Quantum* 2 (ago. 2018), p. 79. ISSN: 2521-327X. DOI: 10.22331/q-2018-08-06-79. URL: http://dx.doi.org/10.22331/q-2018-08-06-79.

[6] A.Yu. Kitaev. «Fault-tolerant quantum computation by anyons». In: *Annals of Physics* 303.1 (gen. 2003), pp. 2–30. ISSN: 0003-4916. DOI: 10.1016/s0003-4916(02)00018-0. URL: http://dx.doi.org/10.1016/S0003-4916(02)00018-0.

[7] Austin G. Fowler et al. «Surface codes: Towards practical large-scale quantum computation». In: *Phys. Rev. A* 86 (3 set. 2012), p. 032324. DOI: 10.1103/PhysRevA.86.032324. URL: https://link.aps.org/doi/10.1103/PhysRevA.86.032324.

[8] Giacomo Torlai e Roger G. Melko. «Neural Decoder for Topological Codes». In: *Phys. Rev. Lett.* 119 (3 lug. 2017), p. 030501. DOI: 10.1103/PhysRevLett.119.030501. URL: https://link.aps.org/doi/10.1103/PhysRevLett.119.030501.

[9] Ryan Sweke et al. «Reinforcement learning decoders for fault-tolerant quantum computation». In: *Machine Learning: Science and Technology* 2.2 (dic. 2020), p. 025005. ISSN: 2632-2153. DOI: 10.1088/2632-2153/abc609. URL: http://dx.doi.org/10.1088/2632-2153/abc609.

[10] Elisha Siddiqui Matekole et al. *Decoding surface codes with deep reinforcement learning and probabilistic policy reuse*. 2022. arXiv: 2212.11890 [quant-ph]. URL: https://arxiv.org/abs/2212.11890.

[11] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge University Press, 2010.

[12] T. D. Ladd et al. «Quantum computers». In: *Nature* 464.7285 (2010), pp. 45–53. DOI: 10.1038/nature08812. URL: https://www.nature.com/articles/nature08812.

[13] John Preskill. «Reliable quantum computers». In: *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 454.1969 (gen. 1998), pp. 385–410. ISSN: 1471-2946. DOI: 10.1098/rspa.1998.0167. URL: http://dx.doi.org/10.1098/rspa.1998.0167.

[14] Daniel Gottesman. *An Introduction to Quantum Error Correction and Fault-Tolerant Quantum Computation*. 2009. arXiv: 0904.2557 [quant-ph]. URL: https://arxiv.org/abs/0904.2557.

[15] Barbara M. Terhal. «Quantum error correction for quantum memories». In: *Reviews of Modern Physics* 87.2 (apr. 2015), pp. 307–346. ISSN: 1539-0756. DOI: 10.1103/revmodphys.87.307. URL: http://dx.doi.org/10.1103/RevModPhys.87.307.

[16] Daniel Gottesman. *Stabilizer Codes and Quantum Error Correction*. 1997. arXiv: quant-ph/9705052 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9705052.

[17] Peter W. Shor. «Scheme for reducing decoherence in quantum memory». In: *Physical Review A* 52.4 (1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493. URL: https://doi.org/10.1103/PhysRevA.52.R2493.

[18] Arthur Pesah. *The stabilizer trilogy I — Stabilizer codes*. Gen. 2023. URL: https://arthurpesah.me/blog/2023-01-31-stabilizer-formalism-1/ (visitato il giorno 22/10/2025).

[19] Dorit Aharonov e Michael Ben-Or. *Fault-Tolerant Quantum Computation With Constant Error Rate*. 1999. arXiv: quant-ph/9906129 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9906129.

[20] Robert Raussendorf e Jim Harrington. «Fault-Tolerant Quantum Computation with High Threshold in Two Dimensions». In: *Phys. Rev. Lett.* 98 (19 mag. 2007), p. 190504. DOI: 10.1103/PhysRevLett.98.190504. URL: https://link.aps.org/doi/10.1103/PhysRevLett.98.190504.

[21] Rajeev Acharya, Google Quantum AI et al. «Suppressing quantum errors by scaling a surface code logical qubit». In: *Nature* 614.7949 (2023), pp. 676–681. DOI: 10.1038/s41586-022-05434-1.

[22] Hussain Anwar et al. «Fast decoders for qudit topological codes». In: *New Journal of Physics* 16.6 (giu. 2014), p. 063038. DOI: 10.1088/1367-2630/16/6/063038. URL: https://doi.org/10.1088/1367-2630/16/6/063038.

[23] Arthur Pesah. *An interactive introduction to the surface code*. Mag. 2023. URL: https://arthurpesah.me/blog/2023-05-13-surface-code/ (visitato il giorno 23/10/2025).

[24] Austin G. Fowler. *Minimum weight perfect matching of fault-tolerant topological quantum error correction in average $O(1)$ parallel time*. 2014. arXiv: 1307.1740 [quant-ph]. URL: https://arxiv.org/abs/1307.1740.

[25] Richard S. Sutton e Andrew G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018. ISBN: 0262039249.

[26] Hannibunny. *Q-Learning: FrozenLake*. 2025. (Visitato il giorno 27/10/2025).

[27] Volodymyr Mnih et al. «Human-level control through deep reinforcement learning». In: *Nature* 518 (2015), pp. 529–533. URL: https://api.semanticscholar.org/CorpusID:205242740.

[28] Hado van Hasselt, Arthur Guez e David Silver. «Deep reinforcement learning with double Q-Learning». In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. AAAI'16. Phoenix, Arizona: AAAI Press, 2016, pp. 2094–2100.

[29] Zhi-Hua Zhou e Shaowu Liu. *Machine Learning*. Springer Singapore, 2021.

[30] Michael A. Nielsen. *Neural Networks and Deep Learning*. Online. Determination Press, 2015. URL: https://neuralnetworksanddeeplearning.com/ (visitato il giorno 29/10/2025).

[31] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[32] Ex10si0n Yan / Aspires. *Neural Networks Series II: Forming Vision – How a Convolutional Neural Network Learns*. https://www.aspires.cc/conv-neural-networks/. Immagine: struttura di rete convoluzionale, consultato il 29-10-2025. 2023.

[33] Fernando Fernández e Manuela Veloso. «Probabilistic policy reuse in a reinforcement learning agent». In: *Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems*. AAMAS '06. Hakodate, Japan: Association for Computing Machinery, 2006, pp. 720–727. ISBN: 1595933034. DOI: 10.1145/1160633.1160762. URL: https://doi.org/10.1145/1160633.1160762.

[34] Adam Paszke et al. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». In: *Advances in Neural Information Processing Systems* 32 (2019).

[35] Mark Towers et al. *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. https://gymnasium.farama.org/. Farama Foundation, ultimo accesso: 6 novembre 2025. 2023.

[36] R. Sweke et al. *DeepQ-Decoding: Decoders for fault tolerant quantum computation via deepQ reinforcement learning*. https://github.com/R-Sweke/DeepQ-Decoding. GitHub repository, accesso: 2 novembre 2025. 2018.

[37] Ziyu Wang et al. *Dueling Network Architectures for Deep Reinforcement Learning*. 2016. arXiv: 1511.06581 [cs.LG]. URL: https://arxiv.org/abs/1511.06581.

[38] Simone Razzetti. *RL-QEC: RL-based decoder for Surface Codes in Quantum Error Correction*. `https://github.com/qismib/rl-qec`. Repository GitHub del progetto di tesi. QISMIB, Università degli Studi di Milano-Bicocca. 2025.

Ringraziamenti

Desidero esprimere la mia gratitudine al mio relatore, Prof. Andrea Giachero, al mio correlatore, Dott. Danilo Labranca, e al mio relatore esterno, Dott. Michele Grossi. In particolare, ringrazio il Prof. Andrea Giachero per avermi introdotto al mondo delle tecnologie quantistiche e del Quantum Computing, sia attraverso il corso di Elementi – per il quale ringrazio anche il Prof. Stefano Sanguinetti – sia grazie all'opportunità di questo progetto. Un ringraziamento speciale va inoltre al Dott. Michele Grossi, per la preziosa guida e il supporto nel corso di questo lavoro, senza il quale la realizzazione della tesi non sarebbe stata possibile.

Poiché sono convinto che le prossime righe saranno quelle più lette dell'intero elaborato, mi sforzo di scriverle meglio che posso, scusandomi in anticipo con tutte le persone che dimenticherò di menzionare.

In maniera simpatica e un po' non convenzionale, ringrazio la biblioteca *Il Pertini* di Cinisello, punto di riferimento e ritrovo per le lunghe giornate di studio.
In primo luogo mi piacerebbe ringraziare tutti gli insegnanti e professori che ho incontrato durante il mio percorso – accademico e non – e che, oltre alle nozioni trasmesse, sono stati in grado di trasmettermi la passione per questa materia.
Un ringraziamento speciale va alla mia *grande* famiglia, fonte di immenso *caotico* affetto e presenza immancabile da sempre nella mia vita. In particolare, vorrei ringraziare i miei genitori per il sostegno che non mi è mai mancato e per gli insegnamenti davvero importanti che mi hanno trasmesso.
Allargando l'orizzonte, vorrei estendere la mia gratitudine ai miei parenti e quindi alle mie nonne, ai miei zii e cugini e a tutti coloro che, per il tempo o i misteri della vita, purtroppo non possono esser più qui oggi.
Desidero inoltre ringraziare tutti gli amici che hanno contribuito a rendere più leggero e piacevole questo percorso. Amici di vecchia data – immancabile Giuse – i miei compagni del liceo, i miei ex compagni di squadra e soprattutto i miei attuali compagni di corso, tra cui non posso non menzionare Manu, Mirco, Ruggio, il Matti e il mitico Diego. Sarebbe bello poterli citare tutti, ma non basterebbero le pagine: mi ritengo davvero fortunato.
Ultima, ma non per importanza, non posso non ringraziare Alice per il suo sostegno, la sua pazienza e per i suoi sorrisi che rendono unico ogni passo del mio percorso.